StarForth Developer Guide

Building, Extending, and Porting the StarForth VM

Robert A. James

Contents

Part I

Getting Started

Chapter 1

Installation and Build

1.1 Installation

StarForth builds from source on Linux (x86_64, ARM64) and experimentally on macOS and Windows (WSL2). No binary packages are distributed at this time.

1.1.1 Prerequisites

Required

- GCC $\geq$ 7.0 or Clang $\geq$ 6.0
- GNU Make
- glibc (standard builds)

Optional

- `texinfo` — GNU info documentation (`sudo apt-get install texinfo`)
- `dpkg-dev` — Debian package generation (`sudo apt-get install dpkg-dev`)
- `rpm-build` — RPM package generation (`sudo dnf install rpm-build`)

1.1.2 Building from Source

The standard optimized build:

```
make fastest
```

This produces the maximum-performance binary with ASM optimisations, direct threading, and LTO enabled. Table 1.1 lists all available build targets.

Target	Description	Performance tier
`make fastest`	Maximum performance build	Tier 5 (highest)
`make pgo`	Profile-guided optimisation	Tier 6 (5–15% above fastest)
`make all`	Standard optimised build	Tier 4
`make debug`	Debug build with symbols	Tier 1
`make minimal`	Minimal/embedded build	Tier 3

Table 1.1: StarForth build targets.

1.1.3 Installation Methods

System-Wide Installation (Recommended)

```
make fastest
sudo make install
```

Installs to the following paths:

- Binary: `/usr/local/bin/starforth`
- Config: `/usr/local/etc/starforth/init.4th`
- Man page: `/usr/local/share/man/man1/starforth.1`
- Documentation: `/usr/local/share/doc/starforth/`
- Info: `/usr/local/share/info/starforth.info`

Custom Prefix

```
make fastest
make install PREFIX=$HOME/.local
```

Debian Package

```
make deb
sudo dpkg -i ../starforth_1.1.0-1_amd64.deb
```

RPM Package

```
make rpm
sudo rpm -ivh ~/rpmbuild/RPMS/x86_64/starforth-1.1.0-1.x86_64.rpm
```

1.1.4 Platform-Specific Notes

x86_64 Linux

Standard installation works without modification:

```
make fastest
sudo make install
```

ARM64 (Raspberry Pi 4 and compatible)

Native build on ARM64:

```
make rpi4
sudo make install
```

Cross-compile from x86_64:

```
sudo apt-get install gcc-aarch64-linux-gnu
make rpi4-cross
scp build/starforth user@arm64-device:~/
```

macOS (Experimental)

```
xcode-select --install
make fastest CC=clang
sudo make install
```

Windows (WSL2)

```
# Inside WSL2 (Ubuntu):
make fastest
make install PREFIX=$HOME/.local
echo 'export␣PATH=$HOME/.local/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

1.1.5 Verifying the Installation

```
starforth --version
starforth --run-tests
starforth
```

Quick functional check:

```
starforth -c ':␣HELLO␣."␣Hello,␣World!"␣CR␣;␣HELLO␣BYE'
starforth --benchmark 10000
```

1.1.6 Uninstallation

```
# From source install:
sudo make uninstall

# From Debian package:
sudo dpkg -r starforth

# From RPM package:
sudo rpm -e starforth
```

1.1.7 Troubleshooting

gcc: command not found

```
# Debian/Ubuntu:
sudo apt-get install build-essential

# Fedora/RHEL:
sudo dnf groupinstall "Development␣Tools"
```

starforth: command not found after installation

```
which starforth
export PATH=$HOME/.local/bin:$PATH
```

Cannot find init.4th

```
export STARFORTH_INIT=/usr/local/etc/starforth/init.4th
```

1.1.8 Building the Documentation

Generate the PDF manual (LaTeX → PDF):

```
make book
```

Generate HTML documentation (single-page and multi-page):

```
make book-html
```

1.2 Quick Start

Build the fastest binary for the current platform:

```
make fastest
```

1.2.1 Common Build Commands

Command	What it does	When to use
`make fastest`	Maximum speed build	Production, benchmarking
`make fast`	Fast without LTO	Development, debugging
`make debug`	Debug build (-g -O0)	Debugging with GDB
`make benchmark`	Run full benchmark suite	Performance testing
`make help`	List all options	Reference

Table 1.2: Common StarForth build commands.

1.2.2 Platform-Specific Builds

x86_64 Linux

```
make fastest                # Auto-detects x86_64; builds with ASM optimisations
./build/starforth
```

Raspberry Pi 4 (native)

```
make fastest                # Auto-detects ARM64; optimised for Cortex-A72
./build/starforth
```

Raspberry Pi 4 (cross-compile from x86_64)

Requires `gcc-aarch64-linux-gnu`:

```
make rpi4-cross             # Builds ARM64 binary with inline ASM
scp build/starforth pi@raspberrypi.local:~/
```

The resulting binary is statically linked and requires no runtime dependencies on the target device.

1.2.3 Performance Modes

Build targets are ranked by output speed:

1. `make pgo` — Profile-Guided Optimisation; 5–15% faster than `fastest`; requires two build passes (3–5 minutes).
2. `make fastest` — Recommended for production; ASM optimisations, direct threading, LTO.
3. `make fast` — ASM + direct threading; no LTO; easier to instrument.
4. `make turbo` — ASM optimisations only; no direct threading.
5. `make all` — Standard build (`-O2`).
6. `make debug` — No optimisations (`-O0`); full debug symbols.

1.2.4 INIT System

At startup StarForth automatically loads `./conf/init.4th`, which defines the foundational word set. No configuration is required:

```
./build/starforth
ok>         # Words from init.4th are immediately available
```

`init.4th` defines words, installs a dictionary fence to protect them from `FORGET`, and zeros blocks for user use.

1.2.5 Testing and Benchmarking

```
make bench          # Quick benchmark (1 million operations)
make benchmark      # Full benchmark suite
make test           # Run full test suite (936+ tests)
```

Typical performance for 1 million stack operations:

Build type	x86_64	ARM64 (Raspberry Pi 4)
Debug	∼800 ms	∼1200 ms
Standard	∼250 ms	∼380 ms
Fastest	∼60 ms	∼95 ms
PGO	∼50 ms	∼80 ms

Table 1.3: StarForth benchmark timings (1M stack operations).

1.2.6 Flags Enabled by `make fastest`

1.2.7 Advanced Build Options

Custom compiler flags:

```
make fastest CFLAGS="$(make␣-s␣print-cflags)␣-DCUSTOM_FLAG"
```

Static binary:

```
make fastest LDFLAGS="-static␣-s"
```

Alternate compiler:

```
make fastest CC=clang
```

Flag	Effect
`-O3`	Maximum compiler optimisation
`-march=native` (x86_64)	Use all host CPU features
`-march=armv8-a+crc+simd` (ARM64)	ARMv8 with NEON
`-DUSE_ASM_OPT=1`	Assembly optimisations
`-DUSE_DIRECT_THREADING=1`	Direct-threaded interpreter
`-flto`	Link-Time Optimisation
`-funroll-loops`	Loop unrolling
`-finline-functions`	Aggressive function inlining
`-fomit-frame-pointer`	Free register for execution

Table 1.4: Compiler flags active in `make fastest`.

1.2.8 Troubleshooting

Illegal instruction error The host CPU does not support a required optimisation. Use a conservative baseline:

```
make fastest CFLAGS="$(BASE_CFLAGS) -O3 -march=x86-64 -DUSE_ASM_OPT
    =1"
```

Build fails outright Verify the standard build first, then re-enable optimisations incrementally:

```
make clean && make all
make clean && make fast
make clean && make fastest
```

Slow performance Confirm optimisation symbols are present in the binary:

```
file build/starforth
nm build/starforth | grep vm_push_asm
```

1.3 Developer Environment

This chapter sets up a working StarForth development environment, including the Isabelle/HOL toolchain required to regenerate the formal-verification documentation.

1.3.1 Required Tools

The core build needs only a C toolchain and Git:

```
sudo apt-get update
sudo apt-get install build-essential gcc make git
```

Several optional tools support documentation, supply-chain, and packaging workflows: Doxygen for API documentation, `syft` for SBOM generation, and `fpm` for building DEB and RPM packages.

```
sudo apt-get install doxygen

wget https://github.com/anchore/syft/releases/latest/download/
    syft_linux_amd64.tar.gz
```

```
tar -xzf syft_linux_amd64.tar.gz
sudo mv syft /usr/local/bin/

sudo apt-get install ruby-dev build-essential
sudo gem install fpm
```

1.3.2 Isabelle/HOL

Isabelle is required to generate the formal-verification documentation. Download Isabelle2025, unpack it, and place its `bin` directory on the `PATH`:

```
cd ~/bin
wget https://isabelle.in.tum.de/dist/Isabelle2025_linux.tar.gz
tar -xzf Isabelle2025_linux.tar.gz
```

Add the binaries to `~/.bashrc`, reload the shell, and verify:

```
export PATH="$PATH:$HOME/bin/Isabelle2025/bin"
source ~/.bashrc
isabelle version    # expects: Isabelle2025
```

The build system locates Isabelle on the `PATH` via a single Makefile variable, `ISABELLE ?= isabelle`. Isabelle is deliberately excluded from the repository through `.gitignore` (`/tools/Isabelle2025/`) and must never be committed. With it installed, three targets become available: `make isabelle-build` verifies all theories, `make isabelle-check` runs a faster syntax-only pass, and `make docs-isabelle` produces the audit-ready documentation.

1.3.3 Continuous Integration

CI pipelines must install Isabelle as a build step. A GitHub Actions job installs the C toolchain, fetches and unpacks Isabelle, appends its `bin` directory to `$GITHUB_PATH`, builds with `make fastest`, runs `make test`, and generates the verification docs with `make docs-isabelle` before uploading them as artifacts. The GitLab equivalent performs the same steps in a `before_script` stage.

Docker and Performance

In Docker, Isabelle additionally requires a JDK (17 or later):

```
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y \
    build-essential gcc make wget openjdk-17-jdk \
    && rm -rf /var/lib/apt/lists/*
RUN wget -q https://isabelle.in.tum.de/dist/Isabelle2025_linux.tar.gz && \
    tar -xzf Isabelle2025_linux.tar.gz && rm Isabelle2025_linux.tar.gz
ENV PATH="/Isabelle2025/bin:${PATH}"
RUN isabelle version
```

Full theory verification takes five to ten minutes. Cache the Isabelle installation between builds, use `isabelle-check` for fast pre-commit syntax checks, and reserve the full `isabelle-build` for the main branch and release tags.

1.3.4 Quick Start

```
git clone <repo-url>
cd StarForth
make fastest
make test
make docs-isabelle
ls -la docs/src/isabelle/
```

1.3.5 Formal Verification

StarForth carries machine-checked correctness proofs in Isabelle/HOL. The theory files live under `docs/src/internal/formal/*.thy`, organized into the `VM_Formal` and `Physics_Formal` sessions, and `make docs-isabelle` renders them into `docs/src/isabelle/`. The session covers core VM semantics (`VM_Core.thy`), stack operations and runtime (`VM_Stacks.thy`, `VM_StackRuntime.thy`), word and register definitions (`VM_Words.thy`, `VM_DataStack_Words.thy`, `VM_ReturnStack_Words.thy`, `VM_Register.thy`), and the physics state machine and observation model (`Physics_StateMachine.thy`, `Physics_Observation.thy`). All theories are fully verified.

1.3.6 Contributing

When a change affects VM semantics: update the relevant Isabelle theories, run `make isabelle-build` to verify the proofs, regenerate documentation with `make docs-isabelle`, and include the regenerated docs in the pull request.

1.4 Getting Started: Orientation

The getting-started material orients new users and developers. The developer setup and contribution workflow are covered in the Developer Environment chapter. Three quick-start guides walk through running experiments:

- Factorial Design of Experiments — the 2^7 factorial harness.
- Heartbeat logging experiments.
- Physics-optimization experiments.

From here, readers proceed to the experimental protocols to run their own experiments, the architecture documentation to understand the system internals, and the source-code reference for detailed technical material.

Chapter 2

Build System Reference

2.1 Build Options Reference

StarForth exposes a comprehensive set of build-time configuration options controlling optimizations, feature selection, platform targeting, and debugging. All options are passed as preprocessor defines via `CFLAGS` or as Makefile variables. Architecture detection runs automatically at compile time through `arch_detect.h`.

2.1.1 Performance Optimization Options

`USE_ASM_OPT`

Enables hand-optimized, architecture-specific assembly implementations for performance-critical operations. Type: integer (0 or 1). Default: 0.

Affected routines include:

- `vm_push` / `vm_pop` — data stack operations
- `vm_rpush` / `vm_rpop` — return stack operations
- `vm_add_check_overflow` / `vm_sub_check_overflow` — arithmetic with overflow detection
- `vm_mul` / `vm_div` — multiply and divide
- `vm_strcmp_asm` / `vm_memcpy_asm` — string and memory operations
- `vm_min_asm` / `vm_max_asm` — conditional-move optimizations
- `vm_abs_asm` — absolute value (ARM64)
- `vm_clz` / `vm_ctz` — count leading/trailing zeros (ARM64)
- `vm_prefetch` / `vm_prefetch_stream` — cache prefetching (ARM64)

Typical performance impact: ≈20–25% on x86_64; ≈15–20% on ARM64. Enabled automatically by the `fastest`, `fast`, `turbo`, and `pgo` targets.

`USE_DIRECT_THREADING`

Switches the inner interpreter from indirect threading (the FORTH-79 default) to direct threading using GCC computed-goto (`&&label` / `goto *ptr`). In direct-threaded mode each word's code field points directly to native code, eliminating one indirection level per dispatch. Type: integer (0 or 1). Default: 0.

Requirements: GCC or Clang; must be combined with `USE_ASM_OPT=1`. Full support is provided for x86_64 (`vm_inner_interp_asm.h`) and ARM64 (`vm_inner_interp_arm64.h`).

Typical performance impact: ≈30–40% over indirect threading when combined with `USE_ASM_OPT=1`. Enabled by `fastest`, `fast`, and `pgo` targets.

`NDEBUG`

Standard C99 macro. When defined, all `assert()` calls compile to nothing, and StarForth additionally suppresses debug logging in hot paths. Default: undefined (assertions active). Typical impact: ≈5–10% speedup.

`STARFORTH_PERFORMANCE`

Experimental flag that trades safety for throughput. Reduces stack-bounds checking in hot paths (DUP, SWAP, etc.) and applies `UNLIKELY()` branch hints, assuming well-formed Forth input.

Caution: stack overflows may go undetected. Use only on code that has been exhaustively tested. Typical additional gain: ≈5–8% over standard `-O3`. Relevant sources: `src/word_source/stack_words.c:5` (DUP), `src/word_source/stack_words.c:117` (SWAP).

2.1.2 Architecture Options

Architecture detection is automatic via `arch_detect.h`; manual overrides are available for cross-compilation scenarios.

`ARCH_X86_64`

Targets x86-64 (AMD64 / Intel 64). Auto-detected from compiler predefined macros (`__x86_64__`, `_M_X64`, `__amd64__`). Enables x86_64 assembly optimizations, SSE4.2 string instructions, and the x86_64 direct-threading header.

`ARCH_ARM64`

Targets AArch64 (ARMv8-A). Auto-detected from `__aarch64__`, `_M_ARM64`, and `__arm64__`. Enables NEON SIMD instructions, ARM64-specific instructions (CLZ, CTZ, RBIT, REV), ARM64 assembly optimizations, and ARM64 direct threading.

Native ARM64 build: `make rpi4`. Cross-compile from x86_64: `make rpi4-cross`.

`ARCH_NAME`

Human-readable string set automatically: `"x86_64"`, `"ARM64"`, or `"Unknown"`. Displayed by `make help`.

On an unrecognized architecture the build emits a compiler warning and falls back to pure-C implementations; no assembly optimizations are loaded.

2.1.3 Platform / Target Options

`STARFORTH_MINIMAL`

Enables minimal/freestanding build suitable for bare-metal or microkernel environments. When defined: ANSI color codes are suppressed in logging; I/O is reduced to minimal output; the build links with `-nostdlib -ffreestanding`. Enabled by `make minimal` and `make l4re`.

`L4RE_TARGET`

Targets the L4Re microkernel operating system. Enables the L4Re block-storage backend, ROMFS file access in place of POSIX, and implies `STARFORTH_MINIMAL=1`. Status: partially implemented (ROMFS stub present). See `src/word_source/starforth_words.c:225` and `include/blkio_fa`

2.1.4 Debugging and Profiling Options

`DEBUG`

Debug builds compile with `-O0 -g -DDEBUG`: no optimization, full debugging symbols, all assertions enabled, verbose logging throughout. Enabled by `make debug`.

`PROFILE_ENABLED`

Activates StarForth's built-in profiler. Type: integer (0 or 1). Default: 0. The profiler tracks per-word execution counts, call-graph relationships (caller/callee), stack operation counts, and total instruction counts. Overhead: ≈10–15%.

```
./build/starforth --profile 2       # enable at depth 2
./build/starforth --profile-report # print results
```

See `include/profiler.h`.

`STRICT_PTR`

Controls the Forth address–to–C pointer mapping. Type: integer (0 or 1). Default: 1.

- `STRICT_PTR=1` — Forth addresses are real pointers (`cell_t = uintptr_t`).
- `STRICT_PTR=0` — Forth addresses are indices/offsets (segmented memory models).

Disable only for benchmarking or non-hosted targets. See `src/vm_api.c:202` and `Makefile:37`.

2.1.5 Block System Options

- `BLKCFG_DEFAULT_FBS` — default Forth block size in bytes (default: 1024). See `include/blkcfg.h`.
- `BLKCFG_PATH_MAX` — maximum path length for block-storage files (default: 4096).
- `BLK_FORTH_SYS_RESERVED` — number of system-reserved RAM blocks, hidden from user access (default: 32, blocks 0–31).
- `BLK_DISK_SYS_RESERVED` — number of system-reserved disk blocks (default: 32, PBN 1024–1055).

2.1.6 Memory Configuration Options

- `DATA_STACK_SIZE` — data stack depth in cells (default: 256). See `include/vm.h`.
- `RETURN_STACK_SIZE` — return stack depth in cells (default: 256).
- `STARFORTH_STATE_BYTES` — total state-buffer size for minimal builds (default: 8192). See `src/main.c:40-42`.
- `SF_FC_BUCKETS` — number of forget-chain hash buckets; defined in `vm.h`.

2.1.7 Feature Flags

`STARFORTH_ANSI`

Enables ANSI escape sequences for terminal control in the block editor (LIST, EDIT): screen clear (`\x1b[2J`), cursor positioning, and colored line numbers. See `src/word_source/editor_words.c:74,188,20`

`VM_HAS_CURRENT_ENTRY`

Enables the `vm->current_entry` field, which tracks the currently-compiling word. Required for the SEE decompiler, recursive word definitions, and dictionary introspection. Default: defined (enabled).

2.1.8 Build System Variables

- `CC` — C compiler. Default: `gcc`. Also accepts `clang` and `aarch64-linux-gnu-gcc`.
- `CFLAGS` — compiler flags. Extends Makefile `BASE_CFLAGS`.
- `LDFLAGS` — linker flags. Default includes `-Wl,-gc-sections -s -flto=auto -fuse-linker-plugin -static`.
- `MINIMAL` — integer (0 or 1). Setting `MINIMAL=1` enables `STARFORTH_MINIMAL` and adds `-nostdlib -ffreestanding`.
- `ASM` — integer (0 or 1). When set to 1, the build emits `.s` assembly listings alongside `.o` object files.

2.1.9 Build Configuration Matrix

Table 2.1: Common build configurations

Target	USE_ASM_OPT	USE_DIRECT_THREADING	NDEBUG	Opt level	Use case
`debug`	0	0	No	`-O0 -g`	Developme
`all`	1	0	No	`-O2`	Default bu
`turbo`	1	0	Yes	`-O3 -flto`	Fast, no di
`fast`	1	1	Yes	`-O3`	Fast, reada
`fastest`	1	1	Yes	`-O3 -flto`	Maximum
`pgo`	1	1	Yes	`-O3 -fprofile-use`	Absolute fa
`minimal`	0	0	No	`-O2`	Embedded
`l4re`	0	0	No	`-O2`	L4Re micr
`profile`	0	0	No	`-O1 -g`	Profiling

2.1.10 Optimization Level Guidelines

- **`-O0 -g` (development)** — fastest compile, slowest runtime (≈10× below optimized); full symbols, all assertions. Use for initial development and crash debugging.
- **`-O2` (balanced)** — moderate compile overhead; 3–4× runtime gain over `-O0`. Default production-test builds.
- **`-O3` (high performance)** — aggressive inlining and vectorization; 5–6× over `-O0`. Production releases.

- `-O3 -flto -march=native -fprofile-use` **(maximum)** — multi-stage build; 7–8× over `-O0`. Release and benchmark artifacts. Use `make pgo`.

2.1.11 Usage Examples

Maximum performance native build

```
make clean && make pgo
```

Produces a PGO-optimized binary with assembly optimizations, direct threading, and link-time optimization.

Debug build with profiler

```
make clean && make profile
./build/starforth --profile 3 --profile-report
```

Cross-compile for Raspberry Pi 4

```
make rpi4-cross
scp build/starforth pi@raspberrypi.local:~/
```

Produces a statically linked ARM64 binary optimized for Cortex-A72.

Minimal embedded build

```
make minimal
```

Produces a freestanding binary with no libc dependencies.

AMD Zen 3 custom build

```
make clean
make CFLAGS="$(BASE_CFLAGS) -O3 -march=znver3 \
     -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -DNDEBUG" \
     LDFLAGS="-flto -s"
```

Generate assembly listings

```
make asm
less build/stack_management.s
```

2.1.12 Compiler Support

- **GCC** — recommended. Minimum GCC 7.0; GCC 11.0+ preferred for improved PGO and ARM64 code generation.
- **Clang** — minimum Clang 10.0; computed-goto and all assembly optimizations supported.
- **ARM64 cross-compilation from x86_64:**

```
sudo apt-get install gcc-aarch64-linux-gnu
make CC=aarch64-linux-gnu-gcc rpi4-cross
```

Static linking (`-static`) is recommended for cross-compiled binaries.

2.1.13 Performance Tuning Checklist

For maximum throughput:

- Use `make pgo` (profile-guided optimization).
- Enable `USE_ASM_OPT=1` and `USE_DIRECT_THREADING=1`.
- Define `NDEBUG`.
- Pass `-O3 -flto -march=native`.
- Add `-fno-plt -fno-semantic-interposition` to reduce indirection.
- Add `-funroll-loops -finline-functions` for code expansion.
- Link statically (`-static`) to eliminate PLT overhead.
- Strip symbols (`-s`) to reduce binary size.

Expected result: ≈7–8× faster than a debug build; ≈1.2–1.5× faster than a plain `-O3` build.

2.1.14 Troubleshooting

"Unknown architecture" warning The build falls back automatically to pure-C implementations. Assembly optimizations are disabled. To add support for a new architecture, extend `arch_detect.h`.

Direct threading shows no performance gain Verify that both `USE_DIRECT_THREADING=1` and `USE_ASM_OPT=1` are set, that the target is x86_64 or ARM64, and that the compiler is GCC or Clang. At runtime, `WORDS-INFO` reports whether direct threading is active.

PGO coverage mismatch warnings Run `make clean && make pgo` to force a full clean two-stage rebuild. Stale coverage data from a prior instrumentation pass causes these warnings.

2.2 Platform Build Guide

StarForth builds for both POSIX hosts and the L4Re microkernel from a single codebase, selected by one Makefile switch. The POSIX path is the default; the L4Re path is enabled with `L4RE=1`.

```
# POSIX build (Linux/macOS/BSD) -- default
make
make fastest
make pgo

# L4Re/StarshipOS build
make L4RE=1

# Clean switch between platforms
make clean && make L4RE=1
```

File	Purpose	Platform
`include/platform_time.h`	Abstraction API	All
`src/platform/platform_init.c`	Platform selector	All
`src/platform/time_posix.c`	POSIX implementation	POSIX
`src/platform/time_l4re.c`	L4Re implementation	L4Re

Table 2.2: Platform abstraction source files.

2.2.1 Platform Abstraction Files

The platform layer isolates timing and clock functionality behind a small set of files. The remainder of the VM is untouched by the platform choice.

Only three core files consume the abstraction: `src/log.c` (timestamps via `sf_realtime_ns()` and `sf_format_timestamp()`), `src/profiler.c` (monotonic timing via `sf_monotonic_ns()`), and `src/main.c` (calls `sf_time_init()` at startup). All other VM core files, word implementations, and the block I/O system are unchanged.

2.2.2 L4Re Time Backend

The L4Re RTC server provides hardware RTC access (x86 I/O ports, ARM PL031, I2C chips), nanosecond precision via the CPU timestamp counter, an IPC interface for getting and setting time, and suspend/resume handling that refreshes the offset on wakeup. The C API lives in `librtc.a`:

```
1 l4_uint64_t l4rtc_get_timer(void);
2 int l4rtc_get_offset_to_realtime(l4_cap_idx_t server, l4_uint64_t *
     ns);
3 int l4rtc_set_offset_to_realtime(l4_cap_idx_t server, l4_uint64_t ns
     );
```

StarForth's L4Re backend behaves as follows. Initialization attempts to obtain the `"rtc"` capability from the L4Re environment, queries the RTC offset by IPC when found, and falls back to a zero offset (epoch time) otherwise. Monotonic time always works without an RTC, reading `l4_kip_clock_ns(l4re_kip())` directly from the Kernel Info Page. Real time returns the RTC offset plus the KIP clock, or zero when no RTC is available. Setting time computes a new offset from the desired time minus uptime and writes it via IPC, which requires write permission on the `"rtc"` capability.

L4Re's libc already provides working `clock_gettime`, `time`, `localtime`, and `strftime`. The platform abstraction is retained anyway for direct control over the RTC capability, explicit handling of a missing RTC, a portable API across all platforms, and to avoid a libc dependency in minimal builds.

2.2.3 Testing

For the POSIX build, confirm timestamped log output and a non-zero RTC check:

```
1 make clean && make
2 ./build/starforth --log-info
```

For the L4Re build inside the StarshipOS tree, `make l4` selects the L4Re backend automatically; running under `scripts/runos.sh` should show working profiler timing (always available via the KIP clock) and working timestamps, which may report 1970 if no RTC server is present. A syntax-checking script can validate both backends without a full L4Re tree by compiling `time_l4re.c` with `-fsyntax-only` (a fatal error on missing L4Re headers is expected) and compiling the POSIX backend and selector cleanly.

2.2.4 Troubleshooting

- *undefined reference to sf_monotonic_ns* — ensure `time_posix.c` and `platform_init.c` are compiled.
- *implicit declaration of sf_time_init* — add `#include "platform_time.h"`.
- *fatal error: l4/re/env.h: No such file or directory* — expected outside the L4Re tree; build through the StarshipOS build system.
- *RTC server not found* — start the RTC server in the loader script and provide the `rtc` capability.
- *Timestamps show 1970-01-01* — the RTC server is not supplying an offset; check the vbus configuration and hardware RTC.

The result is a complete platform abstraction layer that builds on POSIX and L4Re alike, with zero runtime overhead from inline functions and a clean vtable-style separation. Switching platforms requires only `make L4RE=1`; the core VM is unchanged.

2.3 INIT Tools: Disk-Image Synchronization

StarForth ships a toolchain that maintains a bidirectional flow between `./conf/init.4th` — the version-controlled source of truth — and the disk images (`.img` files) tracked through Git LFS. The arrangement supports collaborative development: developers edit blocks natively inside a disk image using `LIST`, `EDIT`, and friends; tools extract the annotated blocks into `init.4th`; those changes are reviewed in Git pull requests; and approved changes are applied back to disk images.

2.3.1 The Two Core Tools

extract-init

`tools/extract-init` (C) scans a disk image for blocks marked with a `(- ... )` header and writes them to an `init.4th` file.

```
./tools/extract-init --img=disks/example.img --out=conf/init-4.4th \
    [--fbs=1024] [--start=0] [--end=-1] \
    [--loose] [--require-close] [--max=N]
```

- `-img=PATH` — disk image to scan (required).
- `-out=PATH` — output `init.4th` file (required).
- `-fbs=N` — Forth block size in bytes (default 1024).
- `-start=N`, `-end=N` — block range; `-1` reads to end of file.
- `-loose` — permit a BOM or leading whitespace before the `(-` header.
- `-require-close` — require the closing `)` in the same block.
- `-max=N` — stop after extracting N blocks.

The detection algorithm reads each block sequentially, checks for `(-` at the start (tolerating a BOM or whitespace under `-loose`), optionally verifies a closing `)` when `-require-close` is set, and extracts matching blocks. Output is a header section followed by `Block N` segments.

apply-init

`tools/apply-init` (C) parses an `init.4th` file and writes its `Block N` sections to the corresponding blocks of a disk image.

```
./tools/apply-init --img=disks/example.img --in=conf/init-4.4th \
    [--fbs=1024] [--start=0] [--end=N] \
    [--clip] [--dry-run] [--verify] [--verbose]
```

Only lines between a `Block N` marker and the next marker (or EOF) are written to block N; everything outside a block section, including top-level comments, is ignored. The `-start` and `-end` options act as write guards, `-clip` truncates oversized blocks, `-dry-run` prints the plan without writing, and `-verify` re-reads each block after writing for a byte-for-byte comparison.

2.3.2 Interactive Wrappers

Two shell wrappers add interactive prompts over the C tools. `scripts/update-init.sh` lists the disk images under `./disks/`, prompts for an image and extraction parameters, displays the command, and runs `extract-init` after confirmation — the natural step after editing blocks in an image. `scripts/apply-init.sh` mirrors this for `apply-init`, used after merging a pull request that changed `init.4th`.

2.3.3 Workflow Patterns

Extract from disk to Git. After working in a disk image, extract the marked blocks, review the diff, and commit:

```
./scripts/update-init.sh        # select disk, confirm extraction
git diff conf/init-4.4th
git add conf/init-4.4th
git commit -m "Add␣new␣INIT␣words:␣FOO␣and␣BAR"
```

Apply from Git to disk. After pulling a merged change, apply it to the local image and verify in the REPL:

```
git pull origin master
./scripts/apply-init.sh         # select disk, confirm application
```

Automation. For CI/CD the C tools run unattended with the full set of flags, for example an apply guarded to blocks at or above 1024, clipping oversized content, with read-back verification and verbose output.

2.3.4 The (- Metadata Marker

The `(-` word is dual-purpose: at runtime it behaves as a Forth comment (like `(` but with a dash), and to the tooling it marks an extraction-worthy block.

```
Block 2048
(- String utilities: COUNT, COMPARE, SEARCH )
: COUNT  DUP 1+ SWAP C@ ;
: COMPARE ... ;
```

The dash distinguishes deliberate metadata from the ordinary `(` comments that fill normal blocks, avoids false positives, and is trivially greppable with `grep "(-"`. The implementation lives in `src/word_source/starforth_words.c` and is registered in both the FORTH and STARFORTH vocabularies:

```
void starforth_word_paren_dash(VM *vm) {
    /* Consume input from "(- " to first ")" */
    int depth = 1;
    while (vm->input_pos < vm->input_length && depth > 0) {
        char c = vm->input_buffer[vm->input_pos++];
        if (c == '(') depth++;
        else if (c == ')') depth--;
    }
    if (depth > 0) {
        log_message(LOG_WARN, "(- comment not terminated");
    }
    log_message(LOG_DEBUG, "(- comment parsed (init.4th metadata
       marker)");
}
```

2.3.5 Safety Features

Four mechanisms protect against accidental data loss. *Guard rails* (`-start` / `-end`) refuse writes outside an allowed block range, protecting the boot area (blocks 0–1023) and system ranges. *Verification* (`-verify`) seeks back, re-reads the full block, and compares it against what was written, catching silent write failures. *Dry run* (`-dry-run`) prints the write plan without touching the disk. *Content-length checks* reject blocks longer than the Forth block size by default; `-clip` truncates instead (not recommended for code).

2.3.6 Building, Naming, and Testing

The interactive scripts rebuild the binaries when the source is newer; manual builds use a C99 compiler with POSIX 64-bit file offsets:

```
gcc -std=c99 -O2 -Wall -Wextra -Werror \
    -o tools/extract-init tools/extract_init.c
gcc -std=gnu99 -D_FILE_OFFSET_BITS=64 -O2 -Wall -Wextra -Werror \
    -o tools/apply-init tools/apply_init.c
```

Disk images follow the convention `<hostname>-<username>-<version>.img`; images go to Git LFS while `conf/init.4th` stays in plain Git as a diffable text file. A round-trip test — apply `init.4th` to a disk, extract it back, and diff (ignoring the metadata header) — should reproduce the original. The block size must match across extract, apply, and the VM's expectation.

Chapter 3

Documentation and API Tooling

3.1 The Documentation System

StarForth generates its API documentation directly from source, driven by Doxygen and rendered into several formats. A single command builds everything:

```
make docs
```

The build emits AsciiDoc API documentation under `docs/src/appendix/`, one flat file per source module. These files are produced on demand by `./scripts/generate-doxygen-appendix.sh` and rebuilt periodically by the Jenkins pipeline. To inspect the result, run the generator and read the index:

```
./scripts/generate-doxygen-appendix.sh
cat docs/src/appendix/index.adoc
```

3.1.1 Installation Requirements

Core functionality requires Doxygen and Graphviz. On the major distributions:

```
sudo apt-get install doxygen graphviz    # Ubuntu/Debian
brew install doxygen graphviz            # macOS
sudo dnf install doxygen graphviz        # Fedora/RHEL
```

PDF output additionally requires a TeX Live installation (`texlive-latex-base` and `texlive-latex-ext` on Debian, or MacTeX/BasicTeX on macOS). AsciiDoc and Markdown conversion requires Pandoc, with Asciidoctor optional for HTML rendering. Verify the toolchain before building — Doxygen should report version 1.9.0 or later:

```
doxygen --version
dot -V
pandoc --version    # optional
pdflatex --version  # optional
```

3.1.2 Documentation Targets

Target	Description	Requirements
`make docs`	Generate all formats	doxygen, graphviz (+ optional)
`make docs-html`	HTML only (fast)	doxygen, graphviz
`make docs-pdf`	PDF only	doxygen, graphviz, pdflatex
`make docs-open`	Generate HTML and open browser	doxygen, graphviz
`make docs-clean`	Remove generated docs	none

3.1.3 Documenting Code

Documentation lives beside the code it describes. The workflow is short: read the style guide, copy the template at `examples/doxygen_example.h`, then annotate. A typical function comment carries a brief, a detailed body, parameters, contracts, and an example:

```
/**
 * @brief Push value onto data stack
 *
 * @details
 * Adds a value to the top of the data stack. Stack overflow
 * is checked and vm->error is set if stack is full.
 *
 * @param vm Pointer to VM instance
 * @param value Value to push
 *
 * @pre vm must be initialized
 * @pre vm->dsp < STACK_SIZE-1
 * @post vm->dsp incremented by 1
 * @post On error: vm->error is set
 *
 * @note This is a hot-path function - optimized for speed
 * @warning Always check vm->error after calling
 *
 * @see vm_pop()
 */
void vm_push(VM *vm, cell_t value);
```

After annotating, rebuild and clear any warnings:

```
make docs-html
cat docs/src/appendix/doxygen_warnings.log
```

3.1.4 Coverage Priorities

The target is 100% coverage of the public API headers. Documentation effort is tiered by audience:

- **High priority** — user-facing API: `include/vm.h`, `include/word_registry.h`, `include/log.h`, `include/io.h`.
- **Medium priority** — developer API: word implementation headers under `src/word_source/include/`, `include/profiler.h`, `include/vm_debug.h`.
- **Low priority** — internal implementation files and test infrastructure headers.

3.1.5 Continuous Integration

The documentation build is CI-friendly. A GitHub Actions job installs Doxygen and Graphviz, runs `make docs-html`, fails the build if `doxygen_warnings.log` is non-empty, and deploys the generated HTML to GitHub Pages on the master branch.

3.1.6 Output Formats

Each format serves a distinct audience. HTML is best for interactive browsing and search; PDF for printing and offline reference; AsciiDoc for downstream conversion; Markdown for GitHub and wikis; and man pages for command-line reference, installable into the system man path.

3.1.7 Troubleshooting

The common failures are missing tools. A `command not found` for `doxygen`, `dot`, or `pdflatex` means the corresponding package is absent — install it, or fall back to `make docs-html` since PDF and Pandoc-based outputs are optional. Missing graphs indicate Graphviz is not installed. Warnings about undocumented functions are resolved by annotating the offending code per the style guide. "No such file or directory" when opening docs means they have not been generated yet.

3.1.8 Customization

Edit `Doxyfile` to adjust the project name and version (`PROJECT_NAME`, `PROJECT_NUMBER`), input file set (`INPUT`, `FILE_PATTERNS`), output formats, and diagram options (`CALL_GRAPH`, `CALLER_GRAPH`). Custom Markdown pages are added by appending them to `INPUT`. HTML appearance is themed through `HTML_EXTRA_STYLESHEET` and `HTML_COLORSTYLE`.

External projects can link against StarForth documentation by referencing the generated tag file `docs/src/appendix/starforth.tag` in their own Doxyfile via `TAGFILES`.

3.2 Doxygen Style Guide

StarForth generates its API documentation from Javadoc-style comments embedded in the source. This guide defines the house style for those comments. Doxygen renders them into HTML, PDF, AsciiDoc, Markdown, and Unix man pages; a single comment therefore feeds every audience, so it pays to write each one with care. Generate with `make docs` for all formats, `make docs-html` for a fast HTML-only pass, or `make docs-open` to build and open in a browser.

3.2.1 Comment Syntax by Construct

File Headers

Every header and source file opens with an `@file` block carrying a brief, a detailed description, and provenance:

```
/**
 * @file vm.h
 * @brief StarForth Virtual Machine Core API
 *
 * @details
 * Detailed description of what this file contains and its purpose.
 *
 * @author R. A. James (rajames)
 * @date 2025-08-15
 * @version 1.0.0
 * @copyright CC0-1.0 (Public Domain)
 */
```

Functions

Functions carry `@brief`, `@param`, `@return`, and an optional `@details`. Substantial functions also document their contracts (`@pre`, `@post`), call out hazards with `@note` and `@warning`, cross-reference relatives with `@see`, and supply a runnable example:

```
/**
 * @brief Initialize the StarForth virtual machine
 *
```

```
 * @details
 * Allocates VM memory, initializes stacks, sets up the dictionary,
 * and registers the FORTH-79 standard word set.
 *
 * @param vm Pointer to uninitialized VM structure
 *
 * @pre vm must point to valid memory
 * @post vm->memory is allocated (VM_MEMORY_SIZE bytes)
 * @post vm->error is 0 on success, 1 on failure
 *
 * @warning Do not use the VM if vm->error is non-zero after init
 * @see vm_cleanup()
 *
 * @par Example:
 * @code
 * VM vm;
 * vm_init(&vm);
 * if (vm.error) return 1;
 * vm_interpret(&vm, "42 . CR");
 * vm_cleanup(&vm);
 * @endcode
 */
void vm_init(VM *vm);
```

Types, Structures, and Members

Typedefs and structures take `@typedef` or `@struct` with a brief and details; each member is documented inline. Note platform caveats and thread safety where relevant — the `VM` structure, for instance, is documented as *not* thread-safe, requiring one instance per thread or external locking.

```
/**
 * @typedef cell_t
 * @brief Forth cell type (signed 64-bit integer)
 *
 * @note Size is platform-dependent: sizeof(signed long)
 */
typedef signed long cell_t;

typedef struct VM {
    /** @brief Data stack (1024 cells) */
    cell_t data_stack[STACK_SIZE];

    /**
     * @brief Data stack pointer (index of top element)
     * @details dsp == -1 means empty; STACK_SIZE-1 means full
     */
    int dsp;
} VM;
```

Enums and Macros

Enumerations use `@enum` with inline member documentation; macros use `@def`. Document the meaning and any layout consequence — for example, `VM_MEMORY_SIZE` is annotated with the 2 MB dictionary / 3 MB user block split.

3.2.2 Special Tags

Related functions are organized with `@defgroup` and `@ingroup`, bracketed by `@{` and `@}`. Cross-references use `@see`; examples use `@code` / `@endcode`. Contract and lifecycle tags — `@pre`, `@post`, `@note`, `@warning`, `@bug`, `@todo`, `@deprecated` — carry the operational caveats a caller must know.

3.2.3 Quality Guidelines

Write documentation that earns its place:

- Document every public API function; supply examples for the complex ones; state pre- and post-conditions; flag dangerous operations with `@warning`; cross-reference with `@see`; keep `@brief` to a single line and push depth into `@details`.
- Do not document private static functions unless they are genuinely intricate, restate the function name, write obvious comments ("`@brief Get value`" for `getValue()`), use vague phrasing, or let the comment drift out of sync with the code.

A complete worked header lives at `examples/doxygen_example.h`.

3.2.4 Checking and Coverage

After annotating, rebuild and clear the warning log:

```
make docs-html
cat docs/api/doxygen_warnings.log
```

The frequent offenders are undocumented functions, missing `@param` entries, missing `@return` on non-void functions, and broken `@see` targets. The coverage goal is 100% across the public headers in `include/`, the word-source headers in `src/word_source/include/`, and the key implementation files in `src/`.

3.2.5 IDE Integration

VS Code generates templates through the "Doxygen Documentation Generator" extension; CLion and IntelliJ have built-in support triggered by typing `/**` and Enter; Vim uses the DoxygenToolkit plugin.

3.3 Doxygen Quick Reference

A one-page cheat sheet for annotating StarForth code. The full conventions live in the Doxygen Style Guide; this card is the working reference.

3.3.1 Essential Commands

```
make docs           # all formats (HTML, PDF, AsciiDoc, MD, man)
make docs-html      # HTML only (fastest)
make docs-open      # generate and open in browser
make docs-clean     # remove generated docs
```

3.3.2 Comment Skeletons

A file header carries `@file`, `@brief`, `@author`, and `@date`. A function carries `@brief`, one `@param` per argument, and `@return`. A struct uses `@struct` with inline member comments (`/**< ... */`); an enum uses `@enum` likewise; a macro uses `@def`; a typedef uses `@typedef`.

```
/**
 * @brief One-line description
 * @param name Parameter description
 * @return Return value description
 */
type function(type name);
```

3.3.3 Common Tags

Tag	Purpose	Example
`@brief`	Short description	`@brief Initialize VM`
`@details`	Detailed description	`@details Allocates memory...`
`@param name`	Parameter	`@param vm VM instance pointer`
`@return`	Return value	`@return 0 on success`
`@retval value`	Specific return	`@retval 0 Success`
`@see`	Cross-reference	`@see vm_cleanup()`
`@note`	Important note	`@note Thread-safe`
`@warning`	Warning	`@warning May block`
`@bug`	Known bug	`@bug Issue #42`
`@todo`	Future work	`@todo Add optimization`
`@deprecated`	Deprecated	`@deprecated Use foo() instead`

3.3.4 Conditions, Examples, and Grouping

Invariants use `@pre`, `@post`, and `@invariant`. Examples sit inside `@code` / `@endcode` under a `@par Example:`. Related functions are bracketed by `@defgroup ... @{` and `@}`. Within comments, Markdown-style formatting works: hyphen bullets and numbered lists for sequences, `##` headings for sections, and `*italic*`, `**bold**`, and `` `code` `` for emphasis.

3.3.5 Two Templates

A simple function needs only brief, params, and return. A complex one earns the full treatment:

```
/**
 * @brief Short description
 *
 * @details
 * Detailed explanation of what this function does and why.
 *
 * @param vm VM instance pointer
 * @param value Input value
 *
 * @return Result value
 * @retval 0 Success
 * @retval -1 Error
 *
 * @pre vm must be initialized
 * @post vm->state is updated
 *
```

```
 * @warning Potential issue to be aware of
 * @see related_function()
 *
 * @par Example:
 * @code
 * int result = my_func(&vm, 42);
 * if (result < 0) handle_error();
 * @endcode
 */
int my_func(VM *vm, int value);
```

3.3.6 Best Practices

Document every public function, keep the brief to one line, push depth into `@details`, supply examples for complex functions, cross-reference with `@see`, document all parameters and return values, and flag hazards with `@warning` and `@note`. Do not document the obvious, restate the function name, omit parameter or return descriptions, write vague prose, use opaque names in examples, or let documentation rot when the code changes.

3.3.7 Checking Your Work

Generate, inspect the warning log, then view:

```
make docs-html
cat docs/api/doxygen_warnings.log
make docs-open
```

Warning	Fix
“Member X is not documented”	Add `@param X description`
“No documentation for function”	Add `@brief` comment
“Return value not documented”	Add `@return description`
“Invalid cross-reference”	Check the `@see` target exists

3.3.8 Effort Estimates

A simple function takes 2–5 minutes; a complex function with an example, 10–15; a struct with ten fields, 10–15; and a complete 20-function header, one to two hours. IDE template generators are available for VS Code (the “Doxygen Documentation Generator” extension), CLion (built in), and Vim (the DoxygenToolkit plugin with `:Dox`).

Part II

VM Architecture

Chapter 4

System Overview

4.1 Architecture Overview

StarForth is a FORTH-79 compliant virtual machine built around a physics-driven adaptive runtime, formally proven to achieve 0% algorithmic variance across experimental runs. Its architecture is organized into three layers — the VM core, a hardware abstraction layer, and platform implementations — coordinated by seven physics feedback loops that let the runtime self-optimize without surrendering deterministic behavior. The central innovation is a physics-grounded modeling vocabulary — execution heat, the rolling window of truth — that drives adaptive optimization while preserving reproducibility.

4.1.1 Three-Layer Model

The VM core sits atop a Hardware Abstraction Layer (HAL), which in turn rests on platform-specific implementations. The VM is platform-agnostic and calls the HAL only; it contains no `#ifdef PLATFORM_*` branches.

- **Layer 1 — VM core and physics subsystems.** The FORTH-79 interpreter (`vm.c`), dictionary, data and return stacks, the physics subsystems (heat, window, cache, pipelining), and the heartbeat coordinator.
- **Layer 2 — HAL.** Clean interfaces for timing (`hal_time.h`), interrupts (`hal_interrupt.h`), memory (`hal_memory.h`), console I/O (`hal_console.h`), and CPU control (`hal_cpu.h`).
- **Layer 3 — platform implementations.** Linux (POSIX `clock_gettime`, `malloc`, stdio), L4Re (`L4Re::Clock`, dataspaces, L4Re console), and StarKernel (TSC + HPET + APIC timing, PMM + VMM + `kmalloc`, UART + framebuffer).

4.1.2 Core Components

VM Core

The core (`src/vm.c`) runs the FORTH-79 interpreter loop, manages the dictionary and stacks, executes words, and handles compilation through `:` and `;`.

```
typedef struct VM {
    vaddr_t data_stack[STACK_SIZE];
    vaddr_t return_stack[STACK_SIZE];
    vaddr_t dict_ptr;                   /* Dictionary pointer */
    vaddr_t here;                       /* Compilation pointer */
    DictEntry *latest;                  /* Most recent word */
    HeartbeatState heartbeat;           /* Timing coordinator */
    RollingWindowOfTruth *window;       /* Execution history */
```

```
    /* ... */
} VM;
```

Execution proceeds by fetching the next word from the input stream, searching the dictionary, then executing or compiling it (governed by `WORD_IMMEDIATE`) or, failing a dictionary match, parsing it as a number. Each execution updates the word's execution heat as physics feedback.

Dictionary System

The dictionary is a linked list of `DictEntry` nodes, searched linearly but accelerated by the hot-words cache. Each entry carries a name, code pointer, flags, and physics metadata:

```
typedef struct DictEntry {
    char name[32];
    void (*code_ptr)(VM *vm);
    uint32_t flags;                     /* IMMEDIATE, HIDDEN, etc. */
    float execution_heat;               /* Physics: frequency tracking */
    PhysicsMetadata physics;            /* Window samples, decay state */
    TransitionMetrics *transitions;     /* Pipelining: successor prediction */
    struct DictEntry *next;             /* Linked list */
} DictEntry;
```

Memory Model

VM addresses (`vaddr_t`) are byte offsets, never C pointers. The dictionary occupies the first 2 MB; user blocks begin at block 2048; the heap is allocated through the HAL. All access goes through bounds-aware accessors, `vm_load_cell()` and `vm_store_cell()`.

4.1.3 Physics Subsystems

Six coordinated subsystems implement the adaptive runtime.

- **Execution heat** (`dictionary_heat_optimization.c`) — each execution increments a per-word counter, identifying frequently used words for optimization (positive feedback).
- **Rolling window of truth** (`rolling_window_of_truth.c`) — a fixed-size circular buffer of execution samples, providing deterministic seeding for statistical metrics such as ANOVA and Levene's test.
- **Hot-words cache** (`physics_hotwords_cache.c`) — sorts dictionary entries by heat and moves hot words to the front of the list, accelerating linear search. It yields a 10–30% speedup on realistic workloads, and cache updates are triggered by heat decay rather than execution order, preserving determinism.
- **Pipelining metrics** (`physics_pipelining_metrics.c`) — records the most common successor of each word for future speculative prefetch; transition counts update deterministically.
- **Inference engine** (`inference_engine.c`) — adapts window width and decay slope using deterministic statistical methods (ANOVA early-exit, Levene's test, exponential regression), with no randomness.

- **Heartbeat system** — a centralized, time-driven coordinator that periodically triggers heat decay (Loop #3) and window-width inference (Loop #5).

The heartbeat coordinates the time-dependent loops on a periodic HAL timer:

```
void vm_tick(VM *vm) {
    if (!vm->heartbeat.enabled) return;
    vm->heartbeat.tick_count++;
    if (should_decay_heat(vm))   decay_execution_heat(vm); /* Loop
        #3 */
    if (should_tune_window(vm))  infer_window_width(vm);   /* Loop
        #5 */
}
```

4.1.4 Seven Feedback Loops

Loop	Name	Type	Effect
#1	Execution Heat Tracking	Positive	Increments `execution_heat`
#2	Rolling Window History	Neutral	Captures sample in circular buffer
#3	Linear Decay	Negative	Decays heat over time
#4	Pipelining Metrics	Positive	Increments `transition_count`
#5	Window Width Inference	Adaptive	Adjusts window size (Levene's test)
#6	Decay Slope Inference	Adaptive	Adjusts decay rate (regression)
#7	Adaptive Heartrate	Adaptive	Adjusts heartbeat frequency (future)

Every loop uses deterministic algorithms; the resulting 0% algorithmic variance has been validated experimentally.

4.1.5 Boot Sequence

On hosted platforms the path is `main()` → HAL initialization → `vm_create()` → dictionary population → physics initialization → heartbeat start → REPL → clean shutdown. On StarKernel the UEFI firmware loads `BOOTX64.EFI`, which collects boot information (memory map, ACPI, framebuffer), calls `ExitBootServices()`, and enters `kernel_main()`; the kernel HAL initializes, the VM is created in kernel mode, and Forth runs as the kernel shell.

4.1.6 Data Flow

A line of input is tokenized; each word is located (hot-words cache first, linear search as fallback), then executed — which tracks heat, records the transition, and updates the rolling window — and any output is emitted. The physics feedback cycle runs alongside: execution raises heat and records a sample; the periodic heartbeat tick decays heat (Loop #3), runs window inference (Loop #5), and, in future, decay inference (Loop #6); the hot-words cache then refreshes and the cycle repeats.

4.1.7 Control Flow and Interrupt Context

In execute mode — the default — words run immediately. Compile mode, entered by `:`, instead compiles words into the dictionary, the exception being `WORD_IMMEDIATE` words, which execute even while compiling.

```
: SQUARE   ( n -- n^2 )   DUP * ;
```

Under StarKernel, certain operations are ISR-safe: `heartbeat_tick_isr()`, `vm_tick()` (no `malloc` or blocking I/O), and the lock-free rolling-window updates. Blocking allocation

(`hal_mem_alloc()`), blocking console reads (`hal_console_getc()`), and the non-reentrant dictionary compiler are *not* ISR-safe.

4.1.8 Cross-Subsystem Interactions

The HAL heartbeat timer fires a periodic interrupt that drives `vm_tick()`, which in turn coordinates heat decay and window-width inference. Defining a new word invalidates the hot-words cache, triggering a rebuild that re-sorts the linked list by heat. As word executions accumulate samples and the window fills, inference runs Levene's test and adjusts the window width.

4.1.9 Roadmap

- **Phase 1 — StarForth (done).** FORTH-79 interpreter, physics-driven adaptive runtime, proven 0% algorithmic variance, the full test suite passing, running on Linux and L4Re.
- **Phase 2 — HAL migration (in progress).** Define HAL interfaces, refactor the Linux platform and the VM core onto the HAL, and validate that determinism is preserved; an L4Re HAL is optional.
- **Phase 3 — StarKernel (planned).** Kernel HAL, UEFI loader, PMM, VMM, `kmalloc`, UART and framebuffer console, TSC/HPET/APIC timing, boot to the `ok` prompt on bare metal, and physics validation on hardware.
- **Phase 4 — StarshipOS (future).** Storage drivers, filesystems, networking, a Forth-task process model, a unified device model, and capability/ACL-based security.

4.1.10 Performance Characteristics

Operation	Cycles	Notes
`DUP`	∼5	Stack manipulation (hot path)
`+`	∼8	Arithmetic (optimized)
Dictionary search (hot)	∼20	Cache hit
Dictionary search (cold)	∼200	Linear search
Word call overhead	∼15	Indirect call
Heat tracking	∼3	Increment + branch

Physics overhead averages under 5% on typical workloads.

4.1.11 Diagnostics

Built-in Forth diagnostics include `WORD-ENTROPY` (execution-heat statistics), `.S` and `.R` (data and return stacks), and `WORDS` (dictionary listing). Development builds add `make debug` (`-g -O0`) and `make PROFILE=1` (`gprof`); DoE mode (`./starforth -doe`) runs the experiments and reports metrics with determinism validation.

4.1.12 Key Invariants

- Determinism: 0% algorithmic variance, formally validated.
- Platform-agnostic VM code: zero `#ifdef PLATFORM_*` branches.
- Memory safety: all addresses are `vaddr_t`, bounds-checked under `STRICT_PTR=1`.
- FORTH-79 compliance across all standard words.
- Zero warnings under `-Wall -Werror`.
- Strict ANSI C99 — no GNU extensions, no C++ features.

4.2 Architecture: Subsystem Index

StarForth's architecture documentation covers the core subsystems and the path toward StarKernel and StarshipOS. The subsystems are the Hardware Abstraction Layer (which enables the StarForth → StarKernel → StarshipOS progression), the heartbeat system (the centralized time-based tuning coordinator), the physics engine (the physics-driven adaptive runtime), the pipelining subsystem (speculative execution via word-transition prediction), and the adaptive systems (window inference and decay algorithms).

The architecture rests on three layers — the platform-agnostic VM and physics subsystems, the HAL, and the platform implementation (Linux, L4Re, or kernel). Its defining feature is a physics-grounded, self-adaptive runtime with formally proven deterministic behavior: 0% algorithmic variance across 90 experimental runs. The full treatment, including the four-phase roadmap, appears in the Architecture Overview chapter.

Chapter 5

Interpreter and Memory Model

5.1 The StarForth Initialization System

StarForth loads foundational Forth definitions from `./conf/init.4th` at startup. The system cleanly separates boot-time initialization from runtime operation, zeroing all initialization blocks once execution completes and protecting the resulting dictionary words from accidental removal.

5.1.1 Components

- `./conf/init.4th` — version-controlled initialization file containing block-structured Forth source.
- `INIT` **word** — core initialization primitive, implemented in `src/word_source/starforth_words.c`.
- `(-` **comment word** — dual-purpose: a standard Forth comment at runtime and a tooling metadata marker.
- **Dictionary fence** — prevents `FORGET` from removing any word defined during initialization.
- **Block subsystem** — provides transient block storage for initialization code.

5.1.2 Design Principles

- **Deterministic boot.** The same `init.4th` always produces the same VM state.
- **Transient initialization.** Blocks execute then vanish; their storage is zeroed and returned to userspace.
- **Protected dictionary.** Initialization words cannot be removed with `FORGET`.
- **No runtime dependencies.** After boot the VM has no file system dependencies.
- **Platform agnostic.** The same mechanism works with both filesystem (Linux) and ROMFS (L4Re) backends.

5.1.3 init.4th File Format

Each file consists of one or more numbered blocks. Each block opens with a `Block <n>` header line and, conventionally, a `(-` comment describing its purpose.

```
Block 2048
(- Large Letter F )
: STAR   42 EMIT ;
: STARS  0 DO STAR LOOP ;
: MARGIN CR 30 SPACES ;
: BLIP   MARGIN STAR ;
: BAR    MARGIN 5 STARS ;
: F      BAR BLIP BAR BLIP BLIP ;

Block 3000
(- Add Me to init.4th )
: E      BAR BLIP BAR BLIP BAR ;

Block 3001
(- Add Me to init.4th )
E F CR
```

Format rules:

- Each block starts with `Block <number>`.
- Blocks load in file order (1, 2, 3, ...) regardless of original block numbers.
- `LOAD` references such as `2048 LOAD` are automatically remapped to sequential numbers.
- Newlines within blocks are preserved for correct parsing.
- The `->` word must not appear in `init.4th`; use separate blocks instead to avoid double execution.

5.1.4 INIT Execution Flow

1. Read `./conf/init.4th` (or ROMFS on L4Re).
2. Parse block headers and build a mapping table (`Block 2048` → sequential 1, etc.).
3. Copy block content sequentially, rewriting `LOAD` references to their sequential equivalents.
4. Execute all blocks via `LOAD`.
5. Switch to the `FORTH` vocabulary context.
6. Zero all initialization blocks to reclaim them for userspace.
7. Return to `main.c`, which immediately sets the dictionary fence.

Stack effect: `INIT ( -- )`.

`INIT` is fatal if `init.4th` cannot be opened, if a block allocation fails, or if a block fails to execute. The system cannot enter the REPL until initialization completes successfully.

5.1.5 Platform Support

On Linux, `INIT` opens the file with `fopen`:

```
FILE *fp = fopen("./conf/init.4th", "r");
```

On L4Re, the file will be read from a ROMFS dataspace declared in `modules.list`.

5.1.6 The (- Comment Word

The `(-` word consumes input from the opening marker to the first unmatched closing parenthesis, handling nested parentheses correctly. It has no stack effect and logs at `DEBUG` level. Tooling in `tools/` uses it as a metadata extraction marker; the runtime treats it as an ordinary comment.

5.1.7 Dictionary Fence Protection

After `INIT` returns, `main.c` sets the dictionary fence:

```
/* Set dictionary fence after INIT to protect foundational words
   from FORGET */
vm.dict_fence_latest = vm.latest;
vm.dict_fence_here   = vm.here;
log_message(LOG_INFO,
    "Dictionary␣fence␣set␣-␣init␣words␣protected␣from␣FORGET");
```

Any subsequent `FORGET` that would reach below this fence silently fails, preventing users from destabilizing words that other definitions depend on.

5.1.8 Block Lifecycle

During INIT: Blocks 1–N hold the parsed content of `init.4th`.

After INIT: Blocks 1–N are zeroed. The dictionary retains all defined words, all protected by the fence.

Runtime: Blocks 1–992 are available to userspace programs as a clean slate with no initialization dependency.

5.1.9 Full Boot Sequence

1. Parse command-line arguments.
2. Open the block device (file or RAM backend).
3. Initialize the VM (`vm_init`).
4. Initialize the block subsystem (1 MB RAM blocks 0–1023).
5. Register all FORTH-79 words.
6. Run the `INIT` word.
7. Set the dictionary fence.
8. Start the REPL.

5.1.10 Memory Layout

VM memory (5 MB) Dictionary grows upward: system words, then initialization words (all protected by the fence), then user words.

Block RAM (1 MB) Block 0 is reserved for volume metadata. Blocks 1–992 are user blocks, zeroed after `INIT`. Blocks 993–1023 are reserved.

5.1.11 LOAD Rewriting Algorithm

The second pass of `INIT` walks the copied block content and rewrites any `NNNN LOAD` pattern found in it:

```
/* First pass: build mapping table
   Block 2048 -> Sequential 1
   Block 3000 -> Sequential 2
   Block 3001 -> Sequential 3 */

/* Second pass: copy with rewriting */
while (copying block content) {
    if (found "NNNN␣LOAD" pattern) {
        lookup NNNN in mapping;
        replace with mapped number;
        write "M␣LOAD" to block;
    }
}
```

5.1.12 Performance Characteristics

Initialization completes in under 10 ms for a typical `init.4th`. The transient blocks are zeroed immediately after execution, so the memory overhead is zero at runtime. Dictionary size grows proportionally to the number of definitions in `init.4th`.

5.1.13 Code References

`src/word_source/starforth_words.c:214-495` `INIT` word implementation. File open (234–240), block mapping (272–305), block copy with `LOAD` rewriting (307–418), block execution (447–465), vocabulary switch (468–481), block zeroing (484–493).

`src/main.c:488-491` Dictionary fence setting.

`src/word_source/defining_words.c:530-612` `FORGET` implementation; fence check at lines 559–571.

`src/word_source/block_words.c:191-213` `LOAD` implementation.

`src/word_source/block_words.c:289-309` `->` implementation (block continuation).

5.1.14 Development Workflow

In IDE mode (Linux), edit `./conf/init.4th` directly and rebuild. The file is plain text; diffs are readable in code review.

In the L4Re deployment path, `init.4th` is packed into a ROMFS image at build time and declared as a module in `modules.list`. The resulting binary carries no filesystem dependency at runtime.

5.1.15 Best Practices

- Keep `init.4th` minimal — only foundational definitions.
- Always annotate blocks with `(-` comments.
- Define dependencies before the words that use them; avoid forward references.
- Add blocks incrementally and test after each addition.

- Avoid side effects in initialization blocks; the last block may optionally invoke a self-test.

Chapter 6

Physics-Driven Adaptive Runtime

6.1 The Seven Feedback Loops

StarForth's adaptive runtime is governed by seven configurable feedback loops. Each loop pairs a positive (accumulation) signal that drives the runtime toward optimization against a negative (stabilization) signal that prevents runaway growth and holds the system at equilibrium. Every loop can be toggled independently through a Makefile flag of the form `ENABLE_LOOP_N_*`, and the loops are designed to compose cumulatively so that Design of Experiments (DoE) campaigns can measure each increment in isolation.

The loops borrow thermodynamic language as a modeling convenience. Using execution frequency as a proxy for thermal energy, the runtime tracks "heat" per word, lets it "decay" over time, and reasons about "stability" in statistical terms. These are descriptive tools, not claims of physical thermodynamics.

Loop	Name	Accumulation signal	Stabilization signal
#1	Execution Heat	word execution increments heat	Loop #3 decay; cold words demoted
#2	Rolling Window	every execution recorded to buffer	wrap overwrite; adaptive shrinking
#3	Linear Decay	slope starts at 1/3	decay reduces heat; slope adjusts ±5%
#4	Pipelining	word→word transitions recorded	counts age out; miss-rate feedback
#5	Window Inference	window grows with diversity	Levene's test shrinks to minimum
#6	Decay Inference	regression extracts decay rate	ANOVA early-exit; fit-quality bound
#7	Adaptive Heartrate	tick counter increments	inference cadence falls when stable

Table 6.1: The seven feedback loops, with their accumulation and stabilization mechanisms.

6.1.1 Loop #1 — Execution Heat Tracking

Loop #1 counts word usage to identify hot execution paths. Every execution increments `DictEntry.execution_heat`, and words crossing the promotion threshold migrate into the `hotwords_cache` for constant-time lookup. Stabilization comes from Loop #3, which decays heat over time, and from cache demotion: words falling below `HEAT_CACHE_DEMOTION_THRESHOLD` (10) are evicted. The `PHYSICS-RESET-STATS` word clears all heat counters. Build flag: `ENABLE_LOOP_1_HEAT_TRAC`

6.1.2 Loop #2 — Rolling Window History

Loop #2 captures the execution sequence in a circular buffer (`execution_history`) to seed metrics deterministically. The window grows to `ROLLING_WINDOW_SIZE` (default 4096) before wrapping and becomes "warm" after 1024 executions, the point at which it holds representative data. Stabilization is twofold: oldest entries are overwritten on wrap, and adaptive shrinking reduces `effective_window_size` whenever pattern-diversity growth falls below

ADAPTIVE_GROWTH_THRESHOLD (1%). The check runs every ADAPTIVE_CHECK_FREQUENCY (256) executions, retains ADAPTIVE_SHRINK_RATE (75%) of the window per cycle, and never shrinks below ADAPTIVE_MIN_WINDOW_SIZE (256). Build flag: ENABLE_LOOP_2_ROLLING_WINDOW.

6.1.3 Loop #3 — Linear Decay

Loop #3 ages words to prevent unbounded heat accumulation; it is primarily a negative-feedback loop. Heat decays at DECAY_RATE_PER_US_Q16 (1/65536 heat per microsecond), giving a half-life of roughly six to seven seconds for a 100-heat word. The decay slope, stored as a $\mathbb{Q}_{48.16}$ value (decay_slope_q48, initialized to 1/3), self-adjusts by ±5% against the hot-to-stale word ratio: an excess of hot words increases the slope, an excess of stale words decreases it. A minimum interval DECAY_MIN_INTERVAL (1 μs) guards against over-decay. Build flag: ENABLE_LOOP_3_LINEAR_DECAY.

6.1.4 Loop #4 — Pipelining Metrics

Loop #4 tracks word-to-word transitions to enable speculative prefetch. WordTransitionMetrics records transitions per word and builds context windows of depth TRANSITION_WINDOW_SIZE (default 2) for prediction, while prefetch_attempts and prefetch_hits track accuracy. Transition counts age out over time, and a miss-rate signal drives a binary-chop search over window size through PipelineGlobalMetrics.suggested_next_size. Build flag: ENABLE_LOOP_4_PIPELINING_METRICS.

6.1.5 Loop #5 — Window Width Inference

Loop #5 finds the optimal rolling-window size through statistical testing. The effective_window_size expands toward ROLLING_WINDOW_SIZE when pattern diversity rises above threshold. For stabilization it applies Levene's test for equality of variance: the heat trajectory is split into K disjoint chunks, each chunk's variance is computed independently, and the test statistic W is compared against the critical value ($W \approx 6.5$ at $\alpha = 0.05$). When $W \leq$ critical, the variances are stable and the window has reached its minimum sufficient size, preventing over-capture of redundant pattern data. Build flag: ENABLE_LOOP_5_WINDOW_INFERENCE.

6.1.6 Loop #6 — Decay Slope Inference

Loop #6 extracts the optimal decay rate by exponential regression over the heat trajectory. Fitting the log-linear model

$$\ln(\text{heat}[t]) = \ln(h_0) - \text{slope} \cdot t \tag{6.1}$$

yields a closed-form adaptive_decay_slope. An ANOVA early-exit guards the cost: if has_variance_stabilized() reports that variance changed by less than 5% since the last inference, the computation is skipped and the cached result reused, saving an estimated 5–10k CPU cycles. The slope is bounded by fit quality (slope_fit_quality_q48), and inference_outputs_validate() rejects invalid slopes (zero or greater than 100). Build flag: ENABLE_LOOP_6_DECAY_INFERENCE.

6.1.7 Loop #7 — Adaptive Heartrate

Loop #7 adjusts tick frequency to balance responsiveness against inference cost. tick_count increments every heartbeat; the heartbeat thread wakes at HEARTBEAT_TICK_NS (1 ms default) and samples hot-word count, total heat, and window width. When the system is stable, full inference runs less often, gated by HEARTBEAT_INFERENCE_FREQUENCY (5000 ticks): expensive inference triggers a slower tick cadence. The background HeartbeatWorker thread can be disabled entirely with HEARTBEAT_THREAD_ENABLED=0. Build flag: ENABLE_LOOP_7_ADAPTIVE_HEARTRATE.

6.1.8 Principal Stabilizers

Five mechanisms carry the bulk of the system's negative feedback:

- **Heat decay** (Loop #3) — the primary stabilizer, preventing unbounded heat accumulation.
- **Window shrinking** (Loops #2 and #5) — a diversity plateau triggers size reduction via Levene's test.
- **ANOVA early-exit** (Loop #6) — variance stability below 5% change skips expensive inference.
- **Cache demotion** (Loop #1) — words below the demotion threshold leave the hot-words cache.
- **Slope reversal** (Loop #3) — an inverted hot-to-stale ratio adjusts the decay slope by ±5%.

6.1.9 Configuration

Any loop can be disabled at build time. Setting all seven flags to zero yields the baseline configuration used as the DoE reference point.

```
# Disable a specific loop
make ENABLE_LOOP_3_LINEAR_DECAY=0

# Baseline build (all loops off)
make ENABLE_LOOP_1_HEAT_TRACKING=0 \
     ENABLE_LOOP_2_ROLLING_WINDOW=0 \
     ENABLE_LOOP_3_LINEAR_DECAY=0 \
     ENABLE_LOOP_4_PIPELINING_METRICS=0 \
     ENABLE_LOOP_5_WINDOW_INFERENCE=0 \
     ENABLE_LOOP_6_DECAY_INFERENCE=0 \
     ENABLE_LOOP_7_ADAPTIVE_HEARTRATE=0
```

Knob	Default	Description
`ROLLING_WINDOW_SIZE`	4096	Initial/maximum window size
`ADAPTIVE_SHRINK_RATE`	75	% retained when shrinking
`ADAPTIVE_MIN_WINDOW_SIZE`	256	Floor for window shrinking
`ADAPTIVE_CHECK_FREQUENCY`	256	Executions between diversity checks
`ADAPTIVE_GROWTH_THRESHOLD`	1	Growth rate (%) signalling saturation
`DECAY_RATE_PER_US_Q16`	1	Heat decay per microsecond ($\mathbb{Q}_{48.16}$)
`HEARTBEAT_INFERENCE_FREQUENCY`	5000	Ticks between full inference runs
`TRANSITION_WINDOW_SIZE`	2	Context depth for pipelining prediction

Table 6.2: Tuning knobs for the feedback loops.

The implementation is distributed across `src/rolling_window_of_truth.c` (Loop #2), `src/dictionary_heat_optimization.c` (Loops #1 and #3), `src/inference_engine.c` (unified Loops #5 and #6), `src/physics_pipelining_metrics.c` (Loop #4), and the `vm_tick()` heartbeat dispatcher in `src/vm.c` (Loop #7). Knob definitions live in `include/rolling_window_knobs.h`.

6.2 Feedback Loop Wiring and Self-Optimization Audit

This audit classifies the self-optimization machinery by two independent axes: whether a loop is *wired* into the execution path, and whether its output is actually *utilized* to drive a decision. At the time of the audit the runtime carried six feedback loops, of which three were fully operational, one ran without validation, one was wired but dormant, and one served observability alone.

- **Fully wired and operational.** Hot-words cache promotion (1.78× measured speedup), rolling-window adaptive shrinking (enforcement fixed 2025-11-08), and pipelining transition speculation (speculative prefetch wired 2025-11-08).
- **Wired and utilized but unvalidated.** Linear heat decay — it runs, but its slope is arbitrary and its function choice is unsupported by measurement.
- **Wired but dormant.** Context-aware window tuning — a binary-chop stub awaiting VM-level metric aggregation.
- **Observability only.** Physics-metadata temperature tracking, collected for formal verification rather than optimization.

6.2.1 Hot-Words Cache Promotion

Each dictionary entry carries an `execution_heat` counter incremented per execution. When heat exceeds `HOTWORDS_EXECUTION_HEAT_THRESHOLD` (default 50), `hotwords_cache_promote()` moves the word into a 32-entry LRU cache, dropping subsequent lookups from 20–30 ns in the bucket to 1–2 ns in cache. Metric tracking lives at `src/dictionary_management.c:170`, the threshold check at `src/physics_hotwords_cache.c:155`, and the promotion logic at `src/physics_hotwords_cac` The measured 1.78× speedup on cache-hit paths confirms the loop as fully operational.

6.2.2 Rolling Window Adaptive Shrinking

This loop measures pattern diversity — the count of unique adjacent word transitions in the execution history. When the diversity growth rate falls below `ADAPTIVE_GROWTH_THRESHOLD` (1%), checked every `ADAPTIVE_CHECK_FREQUENCY` (256) executions, the `effective_window_size` shrinks to `ADAPTIVE_SHRINK_RATE` (75%) of its current value, stepping $4096 \to 3072 \to 2304 \to 1728 \to \cdots \to$ `ADAPTIVE_MIN_WINDOW_SIZE` (256).

The loop was repaired on 2025-11-08. Previously the diversity measurement always scanned the full `ROLLING_WINDOW_SIZE` of 4096 entries, so the reported metric shrank but the enforced scan did not. The fix bounds the scan to the effective window when warm and uses circular indexing to scan only recent entries:

```
uint32_t scan_limit = (window->is_warm)
    ? window->effective_window_size
    : ROLLING_WINDOW_SIZE;

for (uint32_t i = 0; i < scan_limit; i++)
{
    /* window_pos is the next write position */
    uint32_t idx = (window->window_pos
        + ROLLING_WINDOW_SIZE - scan_limit + i)
        % ROLLING_WINDOW_SIZE;
    uint32_t current = window->execution_history[idx];
    /* Count unique transitions in recent entries only */
}
```

New data now arrives each tick and stale data falls off after `effective_window_size` ticks rather than always 4096, satisfying the requirement that the window advance continuously.

6.2.3 Linear Heat Decay (Wired, Unvalidated)

Heat decays linearly:

$$H(t) = \max(0,\ H_0 - \text{decay\_rate} \times \Delta t) \tag{6.2}$$

with rate `DECAY_RATE_PER_US_Q16` (default 1, i.e. 1/65536 heat per microsecond) yielding a roughly six-to-seven-second half-life for a 100-heat word. Stale words shed weight automatically, so only frequently executed words stay above the promotion threshold. The decay logic sits at `src/physics_metadata.c:164-203` (`physics_metadata_apply_linear_decay()`) and is invoked before each execution from `src/vm.c:524`.

A linear form was chosen over exponential for integer-only arithmetic, bounded and predictable convergence time, simplicity of formal verification, and a StarshipOS modeling rationale in which a task context switch forces a heat reset. It was explicitly designated a baseline: the team noted it could switch to a half-life model after validating the linear approach.

The loop remains unvalidated, with several open gaps recorded at audit time:

- No empirical evidence that decay improves any outcome.
- The initial slope of $1\,\mu\text{s}^{-1}$ is arbitrary, with no evidence it is optimal.
- The functional form is unverified — exponential, logarithmic, parabolic, or sinusoidal decay might perform better.
- `DECAY_RATE_PER_US_Q16` has no knob in the DoE configurations, so it cannot be tuned experimentally.
- The loop is not closed: decay happens, but nothing measures whether it helped.

6.2.4 Pipelining Transition Speculation

This loop records transition frequencies ($\text{word}_A \to \text{word}_B$) via `transition_metrics_record()`, computes conditional probabilities $P(B \mid A)$ in $\mathbb{Q}_{48.16}$ format, and tracks prefetch accuracy. Speculation fires when confidence exceeds `SPECULATION_THRESHOLD_Q48` (50%) with at least `MIN_SAMPLES_FOR_SPECULATION` (10) observations: after recording a transition the runtime updates the probability cache, checks whether the predicted next word clears the threshold, and if so locates that word's dictionary entry and pre-promotes it to the hot-words cache so the next lookup is already warm. Metric collection is at `src/physics_pipelining_metrics.c:92-109`, probability calculation at lines 112–123, the decision at lines 164–186, and the action wiring at `src/vm.c:544-583`. The loop is fully closed: metrics collected, probability computed, confidence checked, speculative promotion performed, and the prefetch attempt recorded for feedback.

6.2.5 Context-Aware Window Tuning (Prototype)

A second-phase feature measures multi-word context patterns (sequences of two to four consecutive words) and intends to binary-search for the optimal `effective_window_size` via `transition_metrics_`
(lines 340–357). The function exists but has no integration point in the main execution path, and the algorithm is not yet validated.

6.2.6 Temperature Tracking (Observability Only)

The `temperature_q8` field holds an exponential moving average of execution heat. It is used for profiling, debugging, and as the foundation for the Isabelle/HOL physics state-machine proofs — it does not drive any optimization decision and works as intended in that observational role.

6.2.7 Bootstrap Seeding

Two routines run once at startup to prime feedback state. `rolling_window_seed_hotwords_cache()` (lines 258–329) replays POST execution and promotes high-heat words so the cache is warm before user workload begins. `rolling_window_seed_pipelining_context()` (lines 341–415) replays POST sequences to establish a transition baseline, giving the pipelining loop initial pattern knowledge.

6.2.8 Configuration

All loop knobs are tunable from the Makefile.

```
# Hot-words cache promotion
make HOTWORDS_EXECUTION_HEAT_THRESHOLD=75

# Adaptive window shrinking
make ADAPTIVE_SHRINK_RATE=75        # keep 75%, shrink by 25%
make ADAPTIVE_MIN_WINDOW_SIZE=256   # never shrink below 256
make ADAPTIVE_CHECK_FREQUENCY=256   # check every 256 executions
make ADAPTIVE_GROWTH_THRESHOLD=1    # shrink when growth < 1%

# Heat decay
make DECAY_RATE_PER_US_Q16=16384    # decay rate (heat/us)
make DECAY_MIN_INTERVAL=1000000     # min interval between decays (ns)

# Pipelining (disabled by default)
make ENABLE_PIPELINING=1
make SPECULATION_THRESHOLD_Q48=$((50 << 16))  # 50% confidence
make MIN_SAMPLES_FOR_SPECULATION=10
```

Loop	Measured	Threshold	Action	Status
Hot-words promotion	`execution_heat`	> 50	add to cache	validated
Adaptive shrinking	pattern diversity	growth $< 1\%$	shrink window	fixed
Heat decay	time since exec	∼6–7 s half-life	reduce heat	unvalidated
Pipelining transitions	`transition_heat[]`	$> 50\%$ conf.	speculative prefetch	phase 2
Context window tuning	multi-word seqs	—	binary chop	phase 2
Temperature tracking	`temperature_q8`	—	observability	working

Table 6.3: Audit summary of the six feedback loops as of 2025-11-08.

The audit closed with three active loops, two dormant loops, and one observability loop, plus the single rolling-window shrinking bug identified and fixed in the same pass.

6.3 Monitoring Principles

The StarForth VM is built as a physics-driven organism rather than a conventional interpreter, and its monitoring subsystem is governed by a single overriding doctrine: observation must be non-perturbative. Sensors describe the runtime; they never shape it. These rules are canonical for every contributor and every automated tool that touches the physics runtime, the inference engine, or DoE integration.

The runtime carries several continuous, interacting feedback loops — heat accumulation and decay, the Rolling Window of Truth, hot-words cache dynamics, prefetch heuristics, transition-graph formation, adaptive decay-slope tuning, and adaptive heartbeat-interval control. All of these must exist and run before any monitoring can be safely attached, because a sensor that

injects latency, allocations, reordering, or contention would corrupt the very signal it is meant to read.

6.3.1 Physiology Before Instrumentation

Instrumentation must never precede the runtime's physiology. The required order is strict: first build the core loops — heat and decay, window updates, prefetch logic, inference heuristics, the tick/heartbeat loop, and transition metrics; then let the system run until it reaches first-cycle stability; only then attach sensors, DoE metrics, event-bus listeners, and observability points. Before the loops run there is no signal to observe, so premature metrics would either produce garbage or perturb the VM.

6.3.2 Monitoring Begins at $t \approx 0$

The Rolling Window of Truth begins empty at VM startup, and this is required rather than an error. The VM must operate with no history and adapt with whatever it has, allowing inference to form its earliest predictions from sparse data. No artificial seeding or fake signal insertion is permitted. Inference must detect missing history, treat the first values as a baseline, and adapt gradually as the window fills. Early fluctuations are normal and desired; decay-slope and window-size tuning must behave conservatively while history is thin; prefetch heuristics must never assume a non-empty window; and no subsystem may depend on a pre-populated window.

This startup interval is the system's initial learning phase, analogous to the warm-up of a PID controller, the first samples of a strip-chart recorder, or the startup of a biological homeostasis loop. Monitoring begins at the earliest moment when all feedback loops have completed at least one internal cycle but before any long-term adaptation occurs. This is $t \approx 0$ — not literal time zero, but the first physiologically meaningful moment, when the Rolling Window holds its first value, heat has been incremented at least once, cache promotion may have occurred, the decay slope has an initialized baseline, prefetch tracking exists, the transition graph has at least one edge, and the heartbeat loop has run once. From that point forward the VM has true physics to observe. Where seed values are required, they are supplied by the builder at build-configuration time.

6.3.3 Non-Perturbative Observation

Monitoring must never disturb dictionary ordering, bucket heat, window width, inference tuning, lookup strategy, word execution timing, tick interval, or cache hit and miss counts. The absolute rules are: no `malloc` or `free`; no locks (use read-only pointers); no dictionary traversal; no reordering; no I/O to stdout during VM operation; and no contact with any structure that the physics feedback loops mutate. Monitors observe. They never participate.

6.3.4 History Plus Present Yields Inference Yields Adjustment

This is the governing rule for every inference cycle. Every adaptive subsystem — decay-slope tuning, window-width adjustment, prefetch ROI, hot-word promotion heuristics, and adaptive heartbeat pressure — bases its decisions on three inputs and produces one output:

- **History** (the integral component): the Rolling Window of Truth, the previous decay slope, heat momentum, the prefetch hit/miss ratio, transition-graph edges, and the long-term pressure average.
- **Current value** (the proportional component): the current heat spike, the current window entry, instantaneous tick-time pressure, the current transition, and the current hit or miss event.

- **Inference**: smoothing, numerical weighting, confidence scoring, deterministic seeded stochastic biasing, bounded slope tuning, and bounded window expansion or contraction.
- **Adjustment** (the output): an updated decay slope, window width, prefetch heuristic, heartbeat interval, or hot-word promotion threshold.

This mirrors a PID controller tuned for StarForth's information thermodynamics — using execution statistics as a proxy for thermal state, the controller blends an integral term (history) and a proportional term (present) into a bounded adjustment.

6.3.5 Explicit, Side-Effect-Free Sampling Points

Every feedback loop exposes explicit, isolated sampling points — heat updated, window updated, decay slope recalculated, inference cycle completed, prefetch stat updated, transition recorded, heartbeat tick completed. Each sampling point uses only a `const VM*`, takes read-only access to internal structures, never influences scheduling or ordering, never allocates, and never blocks the VM.

6.3.6 Monitoring Feeds DoE, Not the VM

All monitoring output goes to one of three sinks: an optional, listener-based, non-intrusive event bus; the DoE metrics collector, which emits a single deterministic CSV line with no stdout logging; or optional debug logging restricted to stderr, never present in CI/CD and never in DoE mode without an explicit flag. Monitoring informs experimentation; it does not control the VM. The VM's physiology controls itself.

6.3.7 Monitoring Must Not Create New Feedback Loops

The runtime already has its core loops — heat accumulation and decay, hot-word promotion, inference-driven decay-slope tuning, and inference-driven window-width tuning. Monitoring must not introduce a new contaminating loop, logging-based slowdown, mechanical timing noise, heat changes from sampling, or cache busting from instrumentation. Monitoring describes; it does not shape.

6.3.8 Checklist

The mandatory invariants are that physiology is wired before instrumentation; that monitoring begins at the first stable cycle ($t \approx 0$); that monitors are non-perturbative; that metrics follow the history-plus-present-to-inference- to-adjustment pattern; that metrics never modify physics state; and that DoE output is isolated and side-effect-free. The forbidden actions are sampling inside locked dictionary regions, allocating memory during monitoring, reordering the dictionary because of instrumentation, logging to stdout during VM operation, and creating new feedback loops via metrics.

StarForth is not a VM with some metrics bolted on; it is a self-regulating cybernetic organism with measurable behavior. These monitoring principles are doctrinal and apply without modification.

6.4 Mathematical Models of the Steady-State Machine

This section presents performance models derived empirically from the Steady-State Machine (SSM) experimental record: 51,840 runs of the StarForth VM across 128 feedback-loop configurations. Five relationships were extracted from the data with measurable precision. The headline

result is a capacity law, $\Lambda(\text{DoF}) = 4096/(\text{DoF} + 1)$, which holds with a 0.00% coefficient of variation across all stable configurations.

These models are exploratory. Several borrow functional forms and names from physics — the Lorentz factor, the Friedmann equation, the Schwarzschild radius — purely because the data happens to fit those forms. The physics vocabulary is a modeling convenience for describing adaptive-system behavior; it is not a claim that the runtime obeys relativity or cosmology. Throughout, *degrees of freedom* (DoF) denotes the number of enabled feedback loops, ranging from 0 to 7.

6.4.1 Equation 1 — Performance Scaling with Load

Execution time scales with the number of enabled loops according to a Lorentz-like factor:

$$\tau(\text{DoF}) = \tau_0 \cdot \gamma(\text{DoF}), \qquad \gamma = \frac{1}{\sqrt{1-\beta^2}}, \qquad \beta^2(\text{DoF}) = 0.0736\,\text{DoF} + 0.1464, \tag{6.3}$$

with $\tau_0 = 31.67\,\text{ms/word}$ the rest-state performance. The empirical fit is $R^2 = 0.938$. The β^2 term grows linearly with DoF, so each additional loop carries increasing marginal overhead. The structural similarity to relativistic time dilation is offered as a predictive model, not a physical claim.

6.4.2 Equation 2 — Window Capacity Law Λ

The central empirical finding is an inverse capacity law:

$$\Lambda(\text{DoF}) = \frac{W_0}{\text{DoF} + 1}, \qquad W_0 = 4096. \tag{6.4}$$

Equivalently, the product $\Lambda \times (\text{DoF} + 1) = W_0$ is conserved across all stable configurations.

DoF	Ideal Λ	$\Lambda \times (\text{DoF} + 1)$	Stability (CV)
0	4096.0	4096.0	17.34%
1	2048.0	4096.0	16.70%
2	1365.3	4096.0	15.31%
3	1024.0	4096.0	13.91%
4	819.2	4096.0	13.77%
5	682.7	4096.0	13.96%
6	585.1	4096.0	14.33%
7	512.0	4096.0	22.06%

Table 6.4: Window capacity verification. The conserved product holds at 4096.0 ± 0.0, a 0.00% coefficient of variation.

The symbol Λ is borrowed from cosmology for convenience: it denotes the effective window capacity per degree of freedom, with $W_0 = 4096$ bytes an empirically determined constant. As loops are added, proportional window capacity must fall to preserve stability, and the conserved product behaves as a stability invariant. Notably, $W_0 = 4096 = 2^{12}$ was not designed — it emerged from experimental optimization, and may reflect memory page-size alignment, cache-line constraints, or buffer-size effects.

6.4.3 Equation 3 — Expansion Dynamics (Friedmann Form)

The growth in execution time as loops activate fits a Friedmann-like form:

$$H^2(\text{DoF}) \approx c_1\,\rho(\text{DoF}) + c_2\,\Lambda(\text{DoF}) + c_3, \tag{6.5}$$

with the fitted relation

$$H^2 = -2.24 \times 10^{12}\,\mathrm{DoF} \;-\; 4.51 \times 10^{9}\,\Lambda \;+\; 2.11 \times 10^{13}. \tag{6.6}$$

Here H stands in for the performance growth rate, ρ for the degrees of freedom, and Λ for window-capacity pressure. The negative coefficients indicate that both DoF and Λ contribute to deceleration — the system resists unbounded growth through feedback regulation.

6.4.4 Equation 4 — Instability Threshold (Schwarzschild Form)

A Schwarzschild-like radius models the onset of instability:

$$r_s(\mathrm{DoF}) = \frac{\mathrm{DoF}}{W_{\mathrm{norm}}}, \qquad W_{\mathrm{norm}} = \frac{\mathrm{window}}{4096}. \tag{6.7}$$

In this analogy r_s is the instability threshold, DoF the computational load, and W_{norm} the available space. The configurations with the highest coefficient of variation sit near this threshold.

Config	DoF	r_s	CV	Performance
1	1	1.00	26.90%	32.43 ms
84	3	3.00	26.36%	34.94 ms
46	4	4.00	25.70%	50.27 ms
59	5	5.00	25.31%	50.43 ms

Table 6.5: Configurations nearest the instability threshold. Where $r_s \geq \mathrm{DoF}$, the system tends toward instability.

6.4.5 Equation 5 — Energy–Momentum Form

Total execution time fits an energy–momentum relation:

$$E^2 = E_0^2 + p^2, \qquad E = \tau(\mathrm{DoF}), \qquad E_0 = \tau_0 = 31.67\,\mathrm{ms}, \qquad p(\mathrm{DoF}) = 5.20\,\mathrm{DoF} + 6.55. \tag{6.8}$$

Here E_0 is the all-loops-off baseline, p the additional time cost from loops, and E total execution time. Momentum scales linearly with DoF ($R = 0.425$, $p < 10^{-6}$).

Config	DoF	E_{total}	E_{rest}	p
0	0	31.67	31.67	0.00
35	3	31.59	31.67	0.00
55	5	31.91	31.67	3.88
124	5	59.48	31.67	50.35

Table 6.6: Energy–momentum decomposition for selected configurations.

Two configurations are notable. Config #35 carries three enabled loops yet sits at near-zero momentum, suggesting an optimal arrangement where loop interactions cancel. Config #124, the worst performer, carries the same DoF as Config #55 but enormous momentum (50.35) — a destructive combination of loops.

6.4.6 Interpretation and Open Experiments

The recurring observation is that these adaptive-runtime relationships fit functional forms also found in general relativity, thermodynamics, and quantum mechanics. The strongest of them, the capacity law, is exact to within measurement. Whether this reflects a deep property of feedback-driven systems or an emergent coincidence remains open; the SSM data is presented as evidence that the functional forms are useful predictive models.

Several follow-on experiments are proposed:

- **Window-size scaling.** Vary the window across 1024, 2048, 4096, 8192, and 16384 bytes and test whether $\Lambda(\text{DoF}) = W/(\text{DoF} + 1)$ holds, i.e. whether the 4096 constant scales linearly with window size.
- **Critical window capacity.** Reduce the window in steps and locate a W_{crit} below which the selector collapses to the all-off configuration.
- **Maximum degrees of freedom.** Extend beyond seven loops and test whether the capacity law holds up to a $\text{DoF}_{\max}$, with the candidate $\text{DoF}_{\max} \approx \log_2(W_0) = 12$.

6.5 Steady-State Machine: Experimental Validation

Four experimental campaigns, totaling 51,840 runs of the StarForth VM collected over 22–27 November 2025 (roughly 15.2 MB of raw data), validate the Steady-State Machine (SSM) adaptive runtime. The campaigns establish four results: static configuration choice carries an 88% performance spread; the L8 adaptive mode selector converges to a near-optimal configuration; the system is shape-invariant across waveform types to within 0.6% CV; and the runtime exhibits attractor-basin behavior, self-organizing to a stable operating point.

6.5.1 Dataset 1 — Full Factorial DoE

The full factorial swept all $2^7 = 128$ binary combinations of feedback loops L1–L7, with 300 replicates each (38,400 runs) against a fixed Forth benchmark of 4,501 word executions.

Metric	Value
Best config	#35 @ 31.59 ms/word
Worst config	#124 @ 59.48 ms/word
Performance spread	88.3% slower (worst vs best)
Median performance	40.77 ms/word
CV range	13.77% – 26.90%

Table 6.7: Full factorial performance summary.

The best configuration, #35 (binary 0100011), enables only the window, decay-inference, and heartrate loops (L2, L6, L7) and runs at 31.59 ± 4.78 ms (CV 15.13%) with a 0.00% cache hit rate. The worst, #124 (binary 1111100), enables five of seven loops (L1–L5) and posts a 31.24% cache hit rate yet runs at 59.48 ± 10.80 ms — 88% slower. The lesson is that more adaptation does not imply better performance: loop coordination matters more than loop count.

6.5.2 Dataset 2 — Runoff Competition

The eight top DoE finalists were re-run head to head with 30 replicates each (240 runs) to identify a single best static configuration.

The winner, config 0100101, enables the window, window-inference, and heartrate loops (L2, L5, L7), balancing the fastest mean (30.84 ms) against the best stability (12.49% CV).

Config (binary)	Mean (ms)	Std (ms)	CV (%)
0100101	30.84	3.85	12.49
0000000	31.17	4.34	13.93
0010111	31.19	3.90	12.50
0100100	31.52	4.63	14.69
0110111	31.68	4.47	14.11
0010010	31.89	4.36	13.68
0000011	33.90	11.66	34.40
1000101	34.30	5.07	14.78

Table 6.8: Runoff results. The winner is config 0100101.

6.5.3 Dataset 3 — L8 Adaptive Mode Selector

The L8 selector was validated across five workload families (STABLE, TEMPORAL, VOLATILE, TRANSITION, DIVERSE) against eight strategies (L8 adaptive plus seven static configs), 2,400 runs per family (12,000 total). The central result is that L8 converged to config #55 (binary 0110111 — L2, L3, L5, L6, L7 enabled) for all 1,500 adaptive runs across every workload family. Config #55 ranks #6 of 128 on performance (31.91 ms/word) and #73 of 128 on stability (17.53% CV).

Workload family	L8 adaptive (ms)	C0 baseline (ms)	Best static (ms)
DIVERSE	57.89 ± 2.52	57.59 ± 2.61	57.52
STABLE	57.88 ± 2.62	58.28 ± 3.82	57.88
TEMPORAL	57.96 ± 2.51	57.79 ± 2.50	57.79
TRANSITION	57.67 ± 2.62	57.80 ± 2.63	57.67
VOLATILE	57.93 ± 2.59	58.18 ± 2.56	57.75

Table 6.9: L8 adaptive selector versus baseline and best static configuration, by workload family.

L8 matches or beats the static configurations on every family while performing near-zero mode switching — it converges to a single mode and stays there.

6.5.4 Dataset 4 — Shape-Invariant Validation

Four waveform types (baseline, damped sine, square wave, triangle) were run at 300 replicates each (1,200 runs) under the fixed runoff-winner configuration (0100101).

Waveform	Mean (ms)	Std (ms)	CV (%)
baseline	59.01	1.11	1.89
triangle	58.93	1.09	1.84
square wave	59.16	1.30	2.19
damped sine	59.02	1.44	2.44

Table 6.10: Shape-invariance results across four waveform types.

The CV range is 1.84%–2.44%, a spread of only 0.60%, with a max/min mean performance ratio of 1.0039×. The system is shape-invariant — a critical property for unpredictable real-world workloads.

6.5.5 Attractor Surface

Plotting configuration ID (0–127), mean rolling-window size (3900–4300), and coefficient of variation (0.14–0.26) reveals a clear attractor structure. Most configurations cluster at CV

$\approx$ 0.16–0.20; the basin centers on the optimal performance zone; configurations with CV $>$ 0.22 are rare and unstable; and a 10% variation in window size still maintains convergence. This geometric structure is the foundation for formal verification of convergence via Lyapunov stability analysis.

6.5.6 Methodology

Data was collected on x86-64 hardware using high-resolution nanosecond timers against the fixed 4,501-word Forth benchmark, with 30–300 replicates per configuration. Quality controls tracked coefficient of variation on every measurement, applied z-score outlier detection, monitored CPU thermal stability, and held a fixed memory footprint to exclude garbage-collection interference. The validation strategy was sequential: a full-factorial map of the design space, a head-to-head runoff for the best static configuration, the L8 adaptive-versus- static comparison across workload families, and the waveform sweep to demonstrate invariance.

The raw data spans four CSV files — `doe_results_20251123_093204.csv` (11 MB, 38,400 runs), `runoff_results.csv` (67 KB, 240 runs), `l8_validation_results.csv` (3.8 MB, 12,000 runs), and `shape_results.csv` (365 KB, 1,200 runs) — each carrying 68 columns: the L1–L7 configuration bits, performance metrics, state-vector components (heat, entropy, decay, pressure), cache and lookup statistics, window and inference parameters, and hardware thermal/frequency monitoring.

6.6 The Steady-State Machine

A Steady-State Machine (SSM) is a computational model whose operational state is defined not by discrete symbolic transitions but by convergence to an optimal runtime equilibrium. Where a Finite State Machine, pushdown automaton, or Turing Machine makes "state" explicit and symbolic, an SSM derives its behavior from continuous runtime variables. Using execution statistics as a proxy for thermodynamic quantities, the SSM tracks execution heat, entropy (workload diversity), pipeline readiness, heat decay, the steady-state slope, localized temperature gradients, and adaptive strategy selection. The machine observes itself, adapts, and settles into stable performance basins called steady states. In an SSM the attractor defines the machine, not the symbol.

6.6.1 Core Definition

An SSM is a computational system defined by a set of internal metrics, a set of adaptive policies, an attractor-based state space, and a governing dynamic. The metrics are execution heat H, entropy and diversity measures E, pipeline activation thresholds P, a heat-decay profile Θ, and the sliding execution window W. The adaptive policies are lookup strategies, caching regimes, bucket reorganizations, pipeline decisions, and inference-based mode switches. The state space is composed of attractor basins — steady, quasi-steady, transient, and chaotic — rather than symbolic states. The governing dynamic is

$$M_{t+1} = f(M_t,\ \Delta H,\ \Delta W,\ \Delta E), \tag{6.9}$$

where f drives the machine toward a stable basin. The machine is defined by its convergence behavior, not by its instruction sequence.

6.6.2 The Fundamental Insight

In an SSM, performance *is* the state. The system continuously measures how hot words are, how often patterns recur, how pipeline-able a sequence is, how diverse the window is, and how much

the dictionary should reorganize. As these quantities stabilize, the machine settles into a self-maintaining optimal regime, preserved until the environment changes — through new workloads or a diversity spike — at which point the machine passes through a transient or chaotic regime before finding a new equilibrium.

6.6.3 Phases

An SSM moves through four phases:

- **Transient.** The system warms up; execution patterns are sporadic, heat gradients noisy, and the pipeline inactive or unstable.
- **Quasi-steady.** Patterns emerge, lookups begin to bias, hot words form local attractors, and dictionary reorders stabilize.
- **True steady state.** The heat surface is smooth, the pipeline active, the diversity window predictable, and lookups converge to optimal; runtime exceeds the baseline VM and the system remains stable unless perturbed.
- **Chaotic/disruption.** Triggered by a major workload shift, dictionary mutation, an extreme entropy spike, or a cold-cache shock, the SSM falls out of equilibrium temporarily.

6.6.4 Axioms

- **Axiom 1 — Convergence.** Every sustained workload induces the SSM to converge toward a stable attractor unless external entropy forces divergence.
- **Axiom 2 — Locality of Heat.** Execution heat reflects both locality and temporal relevance; locality produces stability.
- **Axiom 3 — Adaptive Reordering.** Reordering is not symbolic mutation but thermodynamic self-optimization.
- **Axiom 4 — Stability Maximizes Throughput.** The steady state is always faster than the cold state and usually faster than the naive baseline VM.
- **Axiom 5 — Perturbation Response.** When disrupted, the SSM seeks a new steady state; it does not thrash indefinitely.

6.6.5 Relationship to Other Computational Models

The SSM is distinguished from established models by its emergent, non-symbolic state. A Finite State Machine occupies discrete symbolic states, whereas the SSM occupies emergent attractors in a metric space. A Turing Machine defines behavior through tape symbols and transitions; the SSM defines it through adaptation toward equilibrium. A Markov chain moves probabilistically between explicit states; the SSM drifts deterministically toward stable basins under observation. A neural network learns weights; the SSM reorganizes runtime structures rather than parameters.

6.6.6 Novelty

The SSM is presented as the first runtime model whose state is emergent rather than explicit: a runtime self-optimization model, an adaptive VM with convergence properties, a load-dependent reordering machine, a self-optimizing dictionary machine, and a phase-shifting execution system. It defines a new category of adaptive computation.

6.6.7 Canonical Example: StarForth

StarForth is the reference implementation of the SSM. It realizes execution heat, rolling diversity windows, phase-based lookup strategies, adaptive dictionary ordering, pipeline activation, a DoE-calibrated inference loop, and a background physics monitor.

6.6.8 Formal Summary

A Steady-State Machine is a computational system in which the operational state is defined by attractor convergence in a runtime metric space rather than by discrete symbolic transitions. An SSM continuously measures workload characteristics, adjusts its internal structures, and stabilizes into performance-optimized steady states; when perturbed, it transitions through transient or chaotic phases before reaching a new equilibrium. This model defines a class of adaptive computational systems with properties distinct from finite state machines, pushdown automata, and Turing Machines.

6.7 Instrumentation Gaps and Tuning Knobs

This section catalogs the metrics the runtime does *not* yet collect — the blind spots that currently prevent certain tuning knobs from being driven by data. It is a working punch list, organized by subsystem and prioritized for the instrumentation campaign internally nicknamed "twisting the dragon's tail."

6.7.1 Dead Code

One routine is defined but never reached. `hotwords_bucket_reorder()` at `physics_hotwords_cache.c:201` implements a bubble sort over `execution_heat` but is never invoked from the lookup path, with no automatic trigger. Whether this is genuine dead code or a forgotten hook remains open. If it were enabled, it would need three metrics: reorder frequency, bubble-sort cost, and a measurement of whether reordering actually improves hit latency.

6.7.2 Stack Metrics (Zero Collection)

No stack instrumentation exists, leaving `STACK_SIZE` tuning blind and tail-call optimization impossible to justify.

Metric	Location	Impact of absence
Data stack peak depth	`stack_management.c`	cannot size the stack to workload
Return stack peak depth	`stack_management.c`	cannot measure colon nesting
Underflow attempts	`arithmetic_words.c:59-62`	fragile paths invisible
Overflow attempts	`arithmetic_words.c`	no exhaustion early warning

Table 6.11: Missing stack metrics.

6.7.3 Dictionary Metrics (Zero Visibility)

Dictionary behavior is opaque. First-character bucketing across 26 buckets may be unbalanced and would directly affect lookup latency, but no histogram exists to confirm. Dictionary growth velocity, `FORGET` usage, lookup failure rate, and word-name length distribution are likewise untracked, leaving fragmentation, "dictionary full" prediction, error-path frequency, and memory efficiency all unknown. Relevant sites include the dictionary lookup in `vm.c`, `memory_management.c`, and `dictionary_words.c`.

6.7.4 Block I/O Metrics (Completely Dark)

The block subsystem reports nothing. Read and write frequency, writeback frequency (`block_subsystem.c:198-20`
RAM cache hit rate, and dirty block ratio are all DoE phase-1 targets; I/O latency and fragmentation ratio are phase-2 targets. Until these land, writeback behavior — a plausible performance killer — cannot be characterized.

6.7.5 Per-Word Execution Metrics

The inner loop in `vm.c` records no per-word execution frequency, latency distribution, word-category breakdown, or colon-versus-native ratio. These metrics are the validation evidence for the cache and window decisions and the basis for hotspot detection and performance-regression tracking.

6.7.6 Heat Dynamics (Phase 2)

Heat dynamics are largely a phase-2 placeholder in `physics_metadata.c`. A heat decay trace is needed to confirm that decay does not destroy useful pattern information. The heat percentile distribution is stored but never dynamically recalculated, and heat concentration ratio (top-10-word heat over total heat), heat velocity, and heat plateau detection are not tracked at all.

6.7.7 Pipelining Metrics (Instrumented, Unused)

Phase-1 pipelining instrumentation exists but feeds nothing. Prediction accuracy by depth is not measured, context transition frequency is counted but not reported, speculation ROI is not aggregated, misprediction cost is measured but not validated, and pipeline stall reduction is not measured — leaving the binary chop over `TRANSITION_WINDOW_SIZE` and the `SPECULATION_THRESHOLD` tuning without inputs.

6.7.8 Cache Pollution and Cycles

The hot-words cache offers no view into its own churn. Eviction and re-promotion cycles (is the cache thrashing?), false negatives (words evicted before promotion, hinting the threshold is too high), eviction candidate value (was LRU the right policy?), bucket scan depth, and promotion wait time are all uncollected as of day one.

6.7.9 Priorities

The instrumentation work is ranked in four tiers:

- **Critical.** Stack peak depth (minutes to add, large insight); block I/O metrics (writeback behavior is a likely performance killer); dictionary bucket load (imbalance drives the latency tail).
- **High (phase-1 DoE).** Per-word execution frequency; heat decay trace; block I/O hit rate.
- **Medium (phase-2 DoE).** Pipelining accuracy by depth; colon nesting depth; dictionary growth velocity.
- **Nice to have (phase 3).** Heat concentration ratio; word-category breakdown; speculation ROI histogram.

Category	Files	Est. LOC	Est. time
Stack metrics	`stack_management.c`, `vm.h`	∼30	30 min
Dictionary metrics	`vm.c`, `memory_management.c`, `vm.h`	∼50	45 min
Block I/O metrics	`block_subsystem.c`, `vm.h`	∼80	1 hr
Per-word metrics	`vm.c`, `physics_metadata.c`, `vm.h`	∼100	1.5 hr
Heat dynamics	`physics_metadata.c`, `vm.h`	∼60	1 hr
Pipelining accuracy	`physics_pipelining_metrics.c`, `vm.h`	∼40	45 min

Table 6.12: Implementation effort estimate per metric category.

6.8 Adaptive Window Shrinking and Decay Slope: Root-Cause Analysis

This section records a root-cause analysis, dated 2025-11-08, of two questions about the adaptive runtime: whether the rolling window grows as well as shrinks, and whether a driving metric feeds an inference mechanism that tunes it. The finding, accurate for its time, is that the window was then unidirectional — shrink-only — with the growth mechanism deferred to the second phase as Loop #5, and with no measurement-error (gauge) study in place.

The thermodynamic and field metaphors used below are modeling language for the runtime's optimization behavior; they describe how the system tunes its rolling window, not a literal physical apparatus.

6.8.1 The Shrink-Only Feedback Loop

Shrinking was implemented and active. Every word execution is recorded into the rolling window. Every 256 executions the adaptive-shrink check runs: it measures pattern diversity as the count of unique adjacent word transitions ($word_a \rightarrow word_b$), computes a growth rate against the previous measurement, and, when growth falls below one percent, multiplies the effective window size by seventy-five percent, never falling below a floor of 256.

$$growth\_rate = \frac{diversity_{\text{current}} - diversity_{\text{last}}}{diversity_{\text{last}}} \tag{6.10}$$

The complementary growth branch did not exist; it was carried as a commented Phase 2 placeholder. The consequence is structural: once shrunk, the window cannot recover, so a pattern that emerges after a contraction may exceed the window's reach.

6.8.2 Why the Window Read Static in Tests

Metrics consistently reported an effective window size of 4096. Three explanations were advanced: pattern diversity never plateaued below the one percent threshold because the harness kept introducing new transitions; the window never reached its warm state, since warming requires roughly 4096 executions and short tests end first; or the metrics, extracted once at the end of a run, missed the dynamic behavior — the window might already have settled at its floor, or shrinking might never have triggered. The recommended resolution was instrumentation: log each shrink decision to see whether the check was contracting the window or merely measuring it.

6.8.3 The Unimplemented Decay Slope

The exported `decay_slope` metric was hardcoded to zero. The underlying linear-decay model nevertheless existed and ran per word: heat declines as $H(t) = \max(0,\, H_0 - d\,t)$, with the decay constant expressed in $\mathbb{Q}_{48.16}$ fixed point as three units per 65536 per microsecond — roughly

4.58×10^{-5} heat per microsecond, giving a hundred-heat word a half-life near two seconds. What was missing was any aggregate: no total-heat trajectory was stored, no start-to-end slope computed, no heat budget tracked. Three candidate definitions were proposed — the constant decay rate read from the fixed-point constant, the observed heat decline divided by runtime, or a per-word average decay — none of which had been implemented.

6.8.4 Toward a Bidirectional Loop

The design question pointed at a bidirectional controller. In the envisioned Loop #5, diversity-driven shrinking would be balanced by growth driven by prefetch-accuracy feedback from the pipelining loop, so the window oscillates toward an optimal size rather than ratcheting monotonically downward. Three components were missing: a growth trigger (falling prefetch accuracy, a sudden rise in diversity, or available memory), a gauge study to judge whether a given contraction helped or hurt (comparing prefetch accuracy and cache-hit rate before and after), and the wiring that lets the pipelining loop's accuracy metric feed the window-tuning loop — a connection that did not then exist.

6.8.5 Disposition

The analysis concluded that the shrink-only behavior was not a defect but a Phase 1 boundary, with bidirectional tuning explicitly deferred. The shrinking that existed worked correctly; the test harness simply may not have exhibited enough diversity plateau to trigger it. The decay-slope export was a placeholder awaiting one of the three candidate definitions.

6.9 Loop #5: Context-Aware Window Tuning — Design Sketch

This section presents a design sketch for Loop #5, the context-aware window tuner. The motivating gap is that the rolling window shrinks on pattern diversity alone, with no evidence that shrinking helps or harms pipelining prediction. The sketch closes that gap by driving window size from the actual prefetch hit rate through a binary-chop search.

6.9.1 Intended Behavior

Loop #5 measures, triggers, adapts, and records. It tracks global pipelining metrics across all words — total speculative prefetch attempts, successful hits, and their ratio. Periodically, when the window is warm, it computes prediction accuracy at the current window size and asks the binary-chop routine for the next size to try. It then shrinks or grows the effective window according to the accuracy trend: if a smaller window improves accuracy it keeps shrinking; if accuracy degrades it grows back; and it converges on the size that maximizes prediction accuracy. Each size tried is recorded with its observed accuracy, building a history that accelerates convergence.

6.9.2 Required Metrics

A new aggregate structure carries the global pipelining state alongside the existing per-word transition metrics.

```
typedef struct {
    uint64_t prefetch_attempts;       /* total speculative prefetch
       calls */
    uint64_t prefetch_hits;           /* word was actually looked up
        next */
    uint64_t window_tuning_checks;    /* tuning checks performed */
    uint32_t last_checked_window_size;
    double   last_checked_accuracy;
```

```
    uint32_t suggested_next_size;      /* binary-chop recommendation
        */
} PipelineGlobalMetrics;
```

The prefetch counter increments when a speculative promotion fires; the hit counter increments when a subsequent lookup matches a word that was recently promoted speculatively. Accuracy is the ratio of hits to attempts.

6.9.3 The Binary-Chop Suggestion

The tuning routine proposes the next window size from the accuracy trend. With no data it holds. On the first check it shrinks by a quarter. Thereafter it compares current accuracy to the last recorded accuracy: an improvement above a one percent threshold continues shrinking, a degradation below minus one percent grows the window by roughly a third, and a plateau holds the current size — all bounded by the configured minimum and maximum.

$$accuracy = \frac{prefetch\_hits}{prefetch\_attempts}, \qquad \Delta = accuracy_{\text{current}} - accuracy_{\text{last}} \tag{6.11}$$

The suggestion is consulted from the adaptive-shrink check, which already runs on the execution path; when warm and with pipelining enabled, it polls the tuner every thousand executions and applies any change, logging the transition and the accuracy that drove it.

6.9.4 Design Decisions

Four choices shape the loop. *Accuracy measurement* counts speculative promotions that were actually used, which requires distinguishing a predicted lookup from a natural one — by flagging speculatively promoted entries, by matching predicted word identifiers against the next lookup, or by reusing the per-word prefetch metrics already collected. *Tuning frequency* is every thousand warm executions, made configurable at build time. *Search strategy* is greedy shrinking with backoff on degradation, converging by oscillating around the sweet spot. *Build knobs* gate the loop behind the pipelining flag and expose the tuning frequency and accuracy threshold.

6.9.5 Validation and Risks

Validation would run the design-of-experiments harness across a baseline, a cache-only configuration, a window-tuning-with-pipelining configuration, and a combined configuration, collecting the converged window size, the final prefetch accuracy, and throughput against baseline. The known risks are the overhead of periodic accuracy checks, oscillation under noisy accuracy, workload-dependent optima, and the possibility that a speculative lookup does not actually correlate with a prediction. The sketch estimates roughly 150 lines of code across the VM struct, the interpreter, the pipelining-metrics module, and the rolling-window check.

6.10 Multivariate Coupled Dynamics: Golden-Configuration Discovery

This section describes the experimental framework, dated 2025-11-20 and in design at the time of writing, for discovering a golden runtime configuration by stability fingerprinting. It is deliberately not a factorial design of experiments. The premise is that the adaptive runtime is a coupled dynamical system in which every metric is at once a measurable signal, a feedback variable that shapes future state, and a coordinate in a high-dimensional attractor landscape. The analysis therefore measures the shape of each configuration's trajectory through metric space rather than isolated main effects.

The dynamical-systems vocabulary — attractors, basins, eigenmodes, coupling — is modeling language for the runtime's tick-by-tick behavior, applied to the domain of configuration selection; it is not a claim of literal physics.

6.10.1 The Coupling Web

Within a single tick the metrics form a feedback web. Cache and bucket hit rates feed bucket load, which feeds heat, which feeds decay, which feeds window width, which feeds prediction accuracy; prediction accuracy and load feed back into the next tick's cache behavior and into the adaptive heartbeat interval. Load intensity shapes the word-access pattern, which shapes the heat distribution, which governs decay effectiveness and the jitter envelope. Each configuration traces a distinct path through this web.

6.10.2 Three Levels of Measurement

The framework records state at three nested scopes. *Per tick*, at roughly one-millisecond cadence, it captures a snapshot of the coupled system: a temporal anchor, the cache and bucket wave functions $\psi_{\text{cache}}(t)$ and $\psi_{\text{bucket}}(t)$, the feedback signals (hot-word count, mean heat, inferred decay, window width), the load response (executions, prediction attempts, prefetch hits), and the stability measures (tick jitter and a composite smoothness index). *Per run*, over roughly a hundred thousand ticks, it distills a stability fingerprint: convergence time and coefficients of variation, steady-state means, dynamics (rise time, settling time, overshoot), coupling strengths including a full cross-correlation matrix, spectral properties (dominant frequency, spectral entropy, dominant eigenvalue), and an overall stability score. *Per configuration*, over 150 or more runs, it aggregates those fingerprints into between-run statistics: the mean and spread of the stability score, convergence reliability, coupling stability, an attractor-basin depth estimate, the jitter envelope and its whiteness, and noise-propagation measures.

$$\mathit{overall\_stability\_score} = \tfrac{1}{3}\,\mathit{jitter\_score} + \tfrac{1}{3}\,\mathit{convergence\_score} + \tfrac{1}{3}\,\mathit{coupling\_score} \tag{6.12}$$

6.10.3 Experimental Design

The design holds five elite configurations carried forward from an earlier stage, runs each 150 to 200 times for statistical power, and runs each for roughly a hundred thousand ticks to capture both convergence and steady state — on the order of seventy-five million ticks in total, some six to eight hours of wall-clock time. Run order is randomized and configurations are interleaved to avoid temporal and thermal confounds, background tasks are minimized, and every tick is captured without subsampling to preserve time-series fidelity.

6.10.4 Analysis Pipeline

Analysis proceeds in three phases. Per-run fingerprinting detects convergence (when the cache hit-rate coefficient of variation falls below five percent), computes steady-state statistics, extracts the transient dynamics, measures coupling with lag search and a full correlation matrix, performs spectral analysis to find dominant modes, and reduces all of it to a stability index. Per-configuration fingerprinting aggregates the run fingerprints into robustness, convergence reliability, coupling strength, basin depth, and noise characteristics. Golden-configuration selection then scores the five candidates on weighted dimensions and takes the argmax.

6.10.5 Interpretation

A strong configuration fingerprint shows a high mean stability score with low spread (reproducibility), full convergence across runs, tight load-to-heartbeat coupling, a low jitter coefficient

Dimension	Weight	Ideal behavior
Stability score	30%	highest mean, lowest spread
Convergence reliability	25%	full convergence, low variability
Coupling strength	20%	strong (0.8+), stable load-to-heartbeat
Attractor basin depth	15%	fast eigenmode decay, noise-resilient
Jitter smoothness	10%	low variation, white noise

Table 6.13: Weighted dimensions for golden-configuration selection.

of variation, and a large dominant eigenvalue indicating a deep attractor basin. A weak fingerprint inverts each of these: a low and widely scattered stability score, incomplete convergence, loose and noisy coupling, high jitter variation, and a shallow, noise-prone basin.

6.10.6 Implementation Roadmap

The build sequence instruments the heartbeat loop with a per-tick circular buffer, implements per-run fingerprinting (convergence detection, coefficients of variation, correlations, eigenvalues, spectral properties), aggregates run fingerprints into per-configuration fingerprints, scores and selects the golden configuration, and finally produces the visual reporting suite — time-series plots, correlation heatmaps, a principal-component projection of runs, and stability-score distributions.

6.11 Window-Width Inference: A Statistically Valid Redesign

This section sets out a redesign, dated 2025-11-19 and approved for implementation at the time of writing, of the window-width inference algorithm used by the adaptive runtime. The original `find_variance_inflection()` measured the wrong quantity — prefix variance — and the redesign replaces it with a hypothesis test for variance stability across disjoint windows.

6.11.1 Why the Original Algorithm Was Invalid

The original algorithm scanned candidate window sizes and, for each, computed the variance of the trajectory prefix from the start through that size, stopping when the change fell below one percent of the full variance. Four statistical defects follow. The samples are not independent: a larger prefix wholly contains every smaller one, so the variance estimates are correlated and biased. The data are not stationary: execution heat decays roughly as $heat(t) = h_0\, e^{-st}$, so early elements are hot and late elements cold, and both mean and variance drift with position. The decision is confounded: a flattening variance cannot be distinguished between the window being wide enough for the workload (the goal) and the prefix simply accumulating low-heat samples that dilute variance (an artifact). And the threshold is arbitrary: one percent is a magic number with no statistical justification and no accompanying confidence measure.

The failure is concrete. On a trajectory that decays through hot, warm, and cold phases, the algorithm halts not where the workload's true pattern width lies but where it runs out of hot data and crosses into the low-variance cold region — a stopping point with nothing to do with the correct window width.

6.11.2 The Redesign: Variance-Stability Testing

The redesign replaces prefix variance with stability testing over disjoint windows. For each candidate size N, the trajectory is divided into non-overlapping chunks of size N, the variance of each chunk is computed independently, and a statistical test asks whether those chunk variances

are equal. The smallest N at which the variances are statistically similar is the sufficient window width.

6.11.3 Levene's Test

The chosen instrument is Levene's test for equality of variance, evaluated in $\mathbb{Q}_{48.16}$ fixed point. The null hypothesis is that all chunk variances are equal. The test statistic is

$$W = \frac{(K-1)\sum_i n_i(z_i - \bar{z})^2}{\sum_i \sum_j (z_{ij} - z_i)^2}, \tag{6.13}$$

where K is the number of chunks, n_i the size of chunk i, $z_{ij} = |x_{ij} - \text{median}_i|$ the absolute deviation from the chunk median, z_i the chunk mean of those deviations, and $\bar{z}$ their overall mean. When W exceeds the critical value at $\alpha = 0.05$ the variances differ significantly and a larger window is needed; when W falls at or below it the variances are similar enough and the window is sufficient. The redesign uses a conservative critical value near 6.5 to accommodate integer arithmetic.

Three edge cases are handled explicitly: a trajectory shorter than the candidate window falls back to the minimum window; reaching the maximum window without passing the test returns the maximum; and a candidate that yields fewer than three chunks is skipped, since reliable testing needs at least three.

6.11.4 Implementation Outline

The Levene statistic is added as a helper over an array of per-chunk variances, supported by median and mean routines in fixed point. The inference routine then scans candidate sizes in steps of 64 from the minimum to the trajectory-bounded maximum, computes per-chunk variances, applies the test, and returns the first size that passes. The inference output structure is extended with diagnostics — the Levene statistic, the number of chunks tested, a pass flag, and the minimum sufficient window — so the result is inspectable from external analysis tooling.

6.11.5 Validation

Unit tests anchor the test against known inputs: four identically-variance chunks must pass with a statistic far below the critical value, four monotonically increasing variances must fail with a statistic well above it, and a synthetic trajectory with a known pattern width must return a window inside the valid range. Regression testing confirms the existing suite still passes, and the old and new algorithms are run side by side on the same trajectories for comparison.

6.11.6 Expected Outcome

The redesign trades an ad-hoc heuristic for a grounded hypothesis test. Where the old algorithm violated independence, confounded decay with sufficiency, leaned on a magic threshold, and could vary run to run, the new one uses disjoint windows, tests within-chunk stability, reports a defensible confidence measure, and is deterministic. The expectation is that window-width estimates stabilize across runs and that the resulting optimization becomes reproducible and defensible. The assessed risk is medium complexity with no breaking interface change and roughly ten to twenty percent more computation, still under one percent of total VM overhead.

6.11.7 References

- Levene, H. (1960). "Robust tests for equality of variances." In *Contributions to Probability and Statistics*, ed. I. Olkin et al. Stanford University Press.

- Brown, M. B., & Forsythe, A. B. (1974). "Robust tests for the equality of variances." *Journal of the American Statistical Association*, 69(346), 364–367.
- NIST/SEMATECH e-Handbook of Statistical Methods, section on Levene's test.

6.12 Hot-Words Cache Integration Guide

The hot-words cache (`ENABLE_HOTWORDS_CACHE=1`, default on) is a frequency-driven dictionary acceleration feature. It uses per-word execution heat to maintain a small LRU cache of the most frequently executed words and reorders dictionary buckets by execution frequency to reduce average search depth.

Expected impact: 2–4× speedup on hot-word lookups (IF, THEN, DO, LOOP, etc.) under typical FORTH workloads.

6.12.1 Architecture

Cache Structure

The `HotwordsCache` struct (32 entries by default) is embedded in the `VM` struct. The lookup path:

1. Check `cache[0..cache_count]` — cache hit returns immediately.
2. Search the bucket for the name — if found and hot, promote to cache.
3. LRU eviction if the cache is full.

Key parameters (in `include/physics_hotwords_cache.h`):

Constant	Default	Effect
`HOTWORDS_CACHE_SIZE`	32	Cache slots; larger = more hits, slower scan
`HOTWORDS_ENTROPY_THRESHOLD`	50	Executions required for cache admission
`HOTWORDS_ENTROPY_DELTA_THRESHOLD`	100	Delta required to trigger bucket reordering

6.12.2 Benchmark Words

After integration, the following FORTH words are available for measurement:

Word	Purpose
`1000 BENCH-DICT-LOOKUP`	Run 1,000 dictionary lookups and show timing
`PHYSICS-CACHE-STATS`	Display hit rate, promotion count, eviction count
`PHYSICS-TOGGLE-CACHE`	Enable/disable cache at runtime for A/B testing
`PHYSICS-RESET-STATS`	Reset all statistics for a clean measurement
`PHYSICS-BUILD-INFO`	Show current build configuration

6.12.3 Before/After Measurement Protocol

```
# Baseline (cache disabled)
make clean && make ENABLE_HOTWORDS_CACHE=0 test

# With cache enabled (default)
make clean && make ENABLE_HOTWORDS_CACHE=1 test
```

In the REPL:

```
PHYSICS-RESET-STATS
100000 BENCH-DICT-LOOKUP
PHYSICS-CACHE-STATS
```

6.12.4 Expected Results

Scenario	Cache Hit Rate	Speedup
Hot-word heavy (IF, THEN, DO, LOOP)	40–60%	2–4×
Balanced workload	20–30%	1.3–1.8×
Cold-word heavy	5–15%	1.1–1.2×

Memory overhead: approximately 512 bytes (pointer array plus stats struct). **CPU overhead:** negligible — array scan is substantially faster than bucket search.

6.12.5 References

- `src/physics_hotwords_cache.c` — cache implementation
- `include/physics_hotwords_cache.h` — constants and struct definitions
- `src/word_source/physics_benchmark_words.c` — benchmark word definitions
- `src/physics_metadata.c` — metrics collection

6.13 Physics Engine Implementation Options (Pre-Implementation Survey)

This document records the three implementation paths considered for extending the StarForth physics engine beyond its Phase 1 infrastructure. It predates the current implementation and is HISTORICAL; the physics engine has since been developed. Preserved as a design rationale record.

6.13.1 State at Survey Time

Phase 1 infrastructure was complete:

- Execution hooks wired at both outer interpreter (`vm.c:533`) and colon word executor (`vm.c:456`)
- Word metadata initialized with a 20-word seed table (`physics_metadata_apply_seed`)
- Temperature via exponential moving average from entropy
- Analytics heap allocated (10 MiB default)
- Host snapshots functional (POSIX comprehensive, L4Re stub)

Missing: periodic capture mechanism, temperature decay model, feedback loop integration.

6.13.2 Option A: Zone 1 Discovery (1–2 hours)

Wire the observable data stream end-to-end to verify metrics flow from word execution to analytics heap. Steps: (1) call `physics_host_snapshot()` at 10 ms intervals in the REPL loop; (2) add a `PHYSICS-HEAP-DUMP` FORTH word to inspect heap contents; (3) add a `PHYSICS-TEMPS` word to display per-word temperature distribution; (4) write an end-to-end test confirming metrics flow.

6.13.3 Option B: Metrics Infrastructure (2–3 days)

Build comprehensive metrics collection to feed scheduling and memory decisions. Steps: (1) per-word call-stack depth and recursion tracking; (2) latency histogram with percentiles (P50, P99, P99.9); (3) call-graph discovery with edge frequencies; (4) composite summary words (`PHYSICS-SUMMARY`).

6.13.4 Option C: Design Questions Resolution (1 day)

Resolve six architectural decisions before committing to a control system:

Scheduling model. Option chosen: hybrid (advisory hints building toward batch enforcement).

Memory placement. Option chosen: logical tiers (advisory labels, not physical movement).

GC vs. spilling. Option chosen: governance-driven policy with per-word `can_forget` and `can_spill` flags.

L4Re specifics. Decision: defer L4Re optimization to Phase 3; complete POSIX path first.

Governance integration. Decision: load governance config at startup; runtime adjustments stay within approved ranges.

Measurement overhead. Estimated additional cost 80–100 ns per decision; acceptable for interactive and high-performance targets.

6.13.5 Recommended Sequence

A → C → B → Phase 2 implementation. Option A validates infrastructure quickly; Option C guides what metrics Option B should prioritize; Option B then provides the data required for Phase 2 weak control.

6.14 Physics Engine Implementation Status: POSIX vs. L4Re Porting Strategy

This document records the implementation status of the physics engine at the time the porting strategy was being finalized. It is HISTORICAL; the current state of the codebase should be verified directly.

6.14.1 What Is Wired

Both word-execution paths call physics metadata updates:

- **Outer interpreter** (`src/vm.c:533`) — direct word execution: calls `physics_metadata_touch()` after each primitive dispatch.

- **Colon word executor** (`src/vm.c:456`) — nested word calls: same call after each colon-definition word dispatch.

Both paths update `temperature_q8` (exponential moving average from entropy) and `last_active_ns` (monotonic timestamp).

Dictionary initialization calls `physics_metadata_init()` and `physics_metadata_apply_seed()`, pre-seeding 20 high-impact words (IF, LOOP, EMIT, BLOCK, SAVE-BUFFERS, etc.) with domain-knowledge priors.

6.14.2 What Is Incomplete

POSIX/L4Re abstraction pattern. The physics runtime uses inline `#ifdef __l4__` conditionals rather than the vtable pattern established by `include/platform_time.h`. Recommended refactoring: extract POSIX code to `src/platform/linux/snapshot.c` and the L4Re stub to `src/platform/l4re/snapshot.c`, with a vtable dispatcher in `src/physics_runtime.c`.

Periodic host snapshot capture. `physics_host_snapshot()` is defined but never called automatically. No heartbeat mechanism invokes it; the analytics heap is allocated but unused in normal operation.

Temperature decay model. `temperature_q8` is updated on execution but never decayed for inactive words. The exponential moving average with configurable half-life remains unimplemented.

6.14.3 POSIX Implementation Status

The POSIX path is production-ready and robust. Captured signals include CPU count (`sysconf`), scheduler policy and priority (`sched_*`), time quantum (`sched_rr_get_interval`), load average (`getloadavg`), PSI metrics (`/proc/pressure/`), `/proc/stat` jiffies, and cgroup v2 (`cpu.stat`, `memory.current`). Missing interfaces result in `flags = 0` for the affected category; no crashes.

6.14.4 L4Re Implementation Status

The L4Re path is a stub: monotonic and real-time timestamps via the platform time abstraction work correctly; CPU count is hardcoded to 1 (KIP query deferred). Scheduler info via `l4_scheduler_info()` and IO server capabilities are planned for later phases.

6.14.5 Recommended Next Steps

1. Verify temperature updates in `starforth_words_test.c`: define a word, execute it N times, inspect `temperature_q8`.
2. Add periodic host snapshot capture to the REPL loop or VM tick.
3. Implement temperature decay (exponential moving average with configurable half-life) in `src/physics_metadata.c`.
4. Refactor to the vtable pattern once the logic is proven.

6.15 Physics-Driven Scheduling Metadata Plan

This document records the planning context for extending `DictEntry` with physics-inspired metadata. It predates the current implementation and is HISTORICAL. The `DictPhysics` struct fields described here (temperature, last-active timestamp, mass, average latency, state flags, and ACL hint) are implemented in the codebase; the Phase 2 fields below remain prospective.

6.15.1 Current Dictionary Signals

The dictionary already provides these free signals for physics modeling:

- `DictEntry.entropy` — global execution counter; coarse temperature reading.
- Profiler timers — per-word call counts and nanosecond timing when profiling is enabled.
- Word flags (`WORD_IMMEDIATE`, `WORD_COMPILED`, etc.) — proxy indicators for control-flow roles.
- VM memory layout — approximates bytecode footprint (`mass`) from the span of a word definition.

6.15.2 Phase 1: Minimum Viable Physics Metadata (Macro Thermodynamics)

Field	Type	Purpose
`temperature_q8`	`uint16_t`	Quantifies thermal state; derived from entropy delta over sliding window
`last_active_ns`	`uint64_t`	Supports decay models and idle detection
`mass_bytes`	`uint32_t`	Word footprint; represents resource inertia
`avg_latency_ns`	`uint32_t`	Rolling average execution cost when profiling is active
`state_flags`	`uint8_t`	Bitfield (I/O, pure, control) derived from flags/registry
`acl_hint`	`uint8_t`	Reserved for ACL controls per word

This MVP fits under 24 bytes. Only a lightweight monotonic timer and a rolling window accumulator are required.

6.15.3 Phase 2: Physics Scheduler Metadata (Prospective)

When the physics-driven scheduler is implemented, additional optional fields capture richer particle behavior. These fields are lazily maintained and incur no cost in baseline builds:

Field	Type	Benefit
`momentum_vec[3]`	`int32_t[3]`	Directional call rates (interpret, compile, test contexts)
`acceleration_q16`	`int32_t`	Derivative of call rate; detects surges
`energy_quanta`	`uint32_t`	`avg_latency_ns` × `temperature_q8`; supports energy-based priority
`coherence_band`	`uint16_t`	Similarity grouping for locality-aware placement
`charge_enum`	`uint8_t`	Categorical: neutral, I/O, memory, control, meta
`spin_state`	`uint8_t`	Word type: scalar, colon, immediate, compile-only
`entropy_half_life`	`uint16_t`	Configurable decay constant per word
`heat_capacity`	`uint16_t`	Upper bound on temperature rise per interval

This expanded footprint stays within 64 bytes, remaining cache-line friendly.

6.15.4 Instrumentation Strategy

- **Sampling cadence:** lazy updates on word execution; global maintenance ticks for decay and half-life.
- **Profiler integration:** reuse `profiler_word_enter/exit` for `avg_latency_ns` and momentum when profiling is active; skip otherwise.
- **Mass:** computed once at definition time by diffing HERE; avoid per-execution recomputation. Recomputed on redefinition.
- **Decay:** exponential moving average with configurable `entropy_half_life`; fallback to global constant.
- **Pinned words:** `WORD_PINNED` bypasses cooling and heating adjustments. Temperature observational data is still tracked.

6.15.5 Bayesian Inference Loop (Design Intent)

Each word ships with seeded priors (temperature floor, expected latency, side-effect flags). A rolling observation buffer accumulates execution events. Periodic Bayesian updates adjust word-specific distributions; the posterior feeds back into temperature and scheduling weights. An adaptive window (ANOVA/Levene's test) keeps inference both responsive and statistically reliable. Summary statistics persist in a fixed-size analytics heap for warm-boot initialization.

6.15.6 Open Questions (At Survey Time)

- Should physics metadata persist across VM restarts, or is it runtime-only?
- Should `state_flags` derive from author annotations rather than heuristics?
- What minimum clock precision is guaranteed on L4Re for `last_active_ns`?
- How does physics scheduling interact with `STRICT_PTR=1` mode?

6.16 Phase 2: Physics Model Extensions — Decay and Freeze

Status at time of writing: Design phase, ready for implementation. **Date:** 2025-11-07.

This document specifies the freeze flag and decay mechanism that extend the Phase 1 physics model. Phase 2a has since been completed; this document is retained as the design rationale record.

6.16.1 Problem Statement

Phase 1 tracks `execution_heat` as a monotonically increasing counter. When the StarForth VM is preempted and a new task begins execution, the old heat values remain in the dictionary and misrepresent the new task's access patterns. Words that were hot for the previous task pollute the hot-words cache, causing cache misses for the new task's actual hot path.

A decay mechanism addresses this by dissipating heat over time, and a freeze mechanism ensures that genuinely cross-context critical words (e.g. `DUP`, `DROP`, `SWAP`) remain cached regardless.

6.16.2 Word Flag Extension

A new flag is added to the `DictEntry` flag byte:

```
1 #define WORD_FROZEN   0x04  /* execution_heat does not decay */
```

The complete flag layout is:

Bit	Flag
0x80	WORD_IMMEDIATE
0x40	WORD_HIDDEN
0x20	WORD_SMUDGED
0x10	WORD_COMPILED
0x08	WORD_PINNED
0x04	WORD_FROZEN
0x02	(reserved)
0x01	(reserved)

6.16.3 Decay Models Considered

Halflife (Exponential) Decay

$$H(t) = H_0 \cdot 2^{-t/\tau}$$

Smooth, asymptotic approach to zero. Requires floating-point arithmetic (`pow()`). Heat never reaches zero exactly. Considered and rejected for Phase 2a.

Linear Decay (Selected)

$$H(t) = \max(0,\ H_0 - d \cdot t)$$

Pure integer subtraction. Heat reaches zero in finite time $t^* = H_0/d$. Predictable and bounded. Selected for Phase 2a.

Criterion	Halflife	Linear
Integer arithmetic	No	Yes
Bounded convergence	No	Yes
Simplicity	No	Yes
Formal verifiability	Harder	Easier

6.16.4 Integration Points

Decay is applied at word-lookup time before heat accumulation (lazy strategy). A check prevents application for elapsed times below `DECAY_MIN_INTERVAL` (default 1 µs) to avoid noise from rapid successive calls.

Cache eviction logic (Phase 2b) must also check the `WORD_FROZEN` flag and skip frozen candidates:

```
if (candidate->flags & WORD_FROZEN) continue;
```

6.16.5 Configuration

All decay parameters are tunable via Makefile override:

```
# Aggressive decay
make DECAY_RATE_PER_NS=2

# Conservative decay
make DECAY_RATE_PER_NS=0.5 HEAT_THRESHOLD=25
```

6.16.6 Formal Verification Targets

Three theorems for Isabelle/HOL proof (Phase 2d):

Theorem 5 — Decay Monotonicity For non-frozen words, heat is non-increasing over time.

Theorem 6 — Freeze Preservation Frozen words maintain constant heat for all time.

Theorem 7 — Convergence Non-frozen words with positive decay rate reach zero in finite time.

6.16.7 Deployment Roadmap

Phase 2a Freeze flag + linear decay (complete as of 2025-11-07).

Phase 2b Cache eviction respects frozen flag; heat demotion.

Phase 2c StarshipOS real-world validation with OS context switches.

Phase 2d Isabelle/HOL proofs for Theorems 5–7.

Chapter 7

Pipelining and Speculative Execution

7.1 Pipelining and Speculative Execution

Pipelining reduces dictionary lookup latency by speculatively prefetching the next word while the current word executes. The mechanism reuses the runtime's existing execution-heat data: the physics model here serves as a predictor of word-to-word transitions, in direct analogy to CPU branch prediction.

7.1.1 Conceptual Model

Without pipelining, the lookup of word B is blocked until word A finishes executing, so transition latency is exposed on the critical path. With pipelining, the lookup of B is started during the execution of A, hiding that latency behind useful work. The predictor is driven by transition heat: if a word is reliably followed by a particular successor, that successor is prefetched.

```
/* decision rule */
IF transition_heat[N -> N+1] > THRESHOLD THEN prefetch_word(N+1)
```

7.1.2 Transition Metrics

Each word accumulates a transition-heat vector over its successors, a total transition count, prefetch attempt/hit/miss counters, and $\mathbb{Q}_{48.16}$ latency accounts for savings and misprediction cost. Transitions are recorded in the inner interpreter at each word boundary. Probability is computed in $\mathbb{Q}_{48.16}$ fixed point to avoid floating point on the hot path:

$$P(\text{from} \to \text{to}) = \frac{\texttt{transition\_heat}[\text{to}]}{\texttt{total\_transitions}}, \tag{7.1}$$

```
int64_t transition_probability_q48(DictEntry *from, DictEntry *to) {
    if (from->metrics.total_transitions == 0) return 0;
    return ((int64_t)from->metrics.transition_heat[to->id] << 16) /
           (int64_t)from->metrics.total_transitions;
}
```

7.1.3 Tuning Knobs

Five compile-time parameters govern speculation. Each trades coverage against accuracy or cost.

Knob	Purpose	Range	Default
`SPECULATION_THRESHOLD`	Min confidence to speculate	0.10–0.95	0.50
`SPECULATION_DEPTH`	Words ahead to prefetch	1–4	1
`MIN_SAMPLES`	Transitions before speculating	1–100	10
`MISPREDICTION_COST`	Penalty for wrong spec (ns)	0–100	25
`MINIMUM_ROI`	Min expected improvement	1.0–2.0	1.10

A low threshold speculates more often at the cost of mispredictions; a high one is conservative. Depth deeper than one or two yields diminishing returns bounded by the rate of instruction-pointer advancement.

7.1.4 Implementation in Phases

The design proceeds in stages so that each is independently verifiable:

1. **Instrumentation.** Record transitions during normal execution and report transition matrices—no prediction yet.
2. **Prediction analysis.** Predict the next word without prefetching and measure how often the prediction would have been correct.
3. **Pipelining.** Add the actual speculative prefetch and a `should_speculate()` decision that combines the threshold, minimum-sample, and ROI tests against expected latency saved.
4. **Knob tuning.** Sweep parameters; explore adaptive and per-word thresholds.
5. **Integration.** Combine with the hot-words cache and harden for production.

7.1.5 Relation to CPU Pipelining

The analogy maps dictionary lookup to instruction fetch, word dispatch to decode, transition prediction to branch prediction, and prefetch to speculative execution. The key difference is that the VM predicts *sequential* transitions rather than conditional branches: sequences are more predictable, recovery from a wrong guess is merely a normal lookup with no pipeline flush, and the prefetch itself is cheap.

7.1.6 Expected Cumulative Speedup

Pipelining is orthogonal to the hot-words cache and stacks with it. Against an unoptimized baseline of 1.0×, the hot-words cache delivers about 1.78×; pipelining is projected to add a further 1.25–1.40×, for a cumulative 2.2–2.5×. The non-goal is to exceed roughly 2× from pipelining alone, which would be unrealistic without a just-in-time compiler.

7.2 Context Window and Binary-Chop Tuning

The context window extends transition prediction beyond the immediate predecessor, and a binary-chop search selects the window depth that best predicts the next word. This addresses a single question: how deep into execution history must the predictor look?

7.2.1 The Trade-Off

A one-word window asks "after `DUP`, what comes next?" and is right roughly 60% of the time, because several successors are plausible. A two-word window asks "after `LOOP DUP`, what comes next?" and the pattern sharpens toward a single answer near 85%. A four-word window improves

further to perhaps 90%, but at twice the per-word memory and a more complex sparse hash, with diminishing returns thereafter.

7.2.2 Knob #6: TRANSITION_WINDOW_SIZE

The window depth is a compile-time knob defaulting to two, chosen as the empirical sweet spot: one word captures only the immediate predecessor, two captures "where we came from" plus "what we are doing," and four or eight rarely justify the overhead.

```
make TRANSITION_WINDOW_SIZE=1   # immediate successor only
make TRANSITION_WINDOW_SIZE=2   # 2-word patterns (default)
make TRANSITION_WINDOW_SIZE=4   # 4-word patterns
```

The window is a per-word circular buffer of word IDs. As each word executes, its predecessor's window records the context-to-successor transition, then shifts the new word in and the oldest word out. With a window of two the overhead is about 16 bytes per word; for 200 words the total is roughly 27 KB, negligible beside the transition-heat arrays.

7.2.3 Binary-Chop Search

Phase 2 finds the optimal depth by testing window sizes 1, 2, 4, and 8 and measuring prediction accuracy at each. The search starts at two and doubles while accuracy improves, then reverts to one to establish the baseline, and converges on the best observed size.

```
uint32_t suggest_window(current_window, accuracy_at_current) {
    if (current_window == 2 && accuracy_improves) return 4;
    if (current_window == 4 && accuracy_improves) return 8;
    if (current_window >= 4 && accuracy_plateaus) return 1;
    /* otherwise converge to the size with best accuracy */
}
```

For each candidate, the harness rebuilds at that window size, runs the `BENCH-DICT` workload to collect transition data, counts transitions whose context-to-successor probability exceeds 70% as predictable, and records accuracy as the predictable fraction.

7.2.4 Expected Outcome

FORTH exhibits strong local patterns, so most decisions depend on about two words of context. The hypothesis, to be confirmed by measurement:

Window	Predicted accuracy
1	∼65%
2	∼82% (expected winner)
4	∼88% (marginal, 2× memory)
8	∼89% (overhead unjustified)

The recommendation is to keep the window at two: 82% accuracy at minimal overhead is the best return on investment.

7.2.5 Interaction With Other Knobs

Window depth shifts the useful settings of the speculation knobs. A deeper, more accurate window produces fewer failures, permitting a lower speculation threshold (more aggressive prefetch) and fewer required samples before a pattern is trusted. A shallow window calls for the opposite: a higher threshold and more samples to compensate for noisier predictions.

7.2.6 Status

The Phase 1 extension—the window knob, the per-word circular buffer, and the context-aware recording functions—is complete and collecting data. Three functions remain stubs awaiting Phase 2: the accuracy-reporting string, the context recording that will build a sparse hash of context-to-successor patterns, and the binary-chop suggestion logic that will react to measured accuracy rather than blindly doubling.

7.3 Phase 1: Pipelining Instrumentation

Phase 1 establishes the observability layer for pipelining: it tracks word-to-word transitions during execution to answer which words typically follow which. It deliberately stops short of speculative execution, collecting the data that later phases will act upon.

7.3.1 Transition Metrics Structure

Each dictionary word carries a pointer to a transition-metrics record holding the per-successor transition-heat array, a total count, prefetch counters reserved for Phase 3, $\mathbb{Q}_{48.16}$ latency accounts, and a cached most-likely successor with its probability.

```
typedef struct WordTransitionMetrics {
    uint64_t *transition_heat;              /* Array[
        DICTIONARY_SIZE] */
    uint64_t  total_transitions;
    uint64_t  prefetch_attempts;            /* Phase 3 */
    uint64_t  prefetch_hits;                /* Phase 3 */
    uint64_t  prefetch_misses;              /* Phase 3 */
    int64_t   prefetch_latency_saved_q48;   /* Phase 3 */
    int64_t   misprediction_cost_q48;       /* Phase 3 */
    int64_t   max_transition_probability_q48;
    uint32_t  most_likely_next_word_id;
} WordTransitionMetrics;
```

The transition-heat array is allocated lazily on the first transition, so words that never execute consume no memory for tracking.

7.3.2 Data Collection

The inner interpreter records one transition per word boundary, gated on `ENABLE_PIPELINING`. The cost is about two cycles per word execution—lost in the noise of the word itself.

```
if (prev_word && prev_word->transition_metrics && ENABLE_PIPELINING)
    {
    transition_metrics_record(prev_word->transition_metrics,
                              word_id, DICTIONARY_SIZE);
}
```

7.3.3 Fixed-Point Probability

All probabilities use 64-bit signed $\mathbb{Q}_{48.16}$ fixed point, which keeps the math deterministic across platforms, free of `libm`, and compatible with formal verification—no floating point appears on the path.

$$P_{\mathrm{Q48.16}} = \frac{\texttt{transition\_heat}[\mathrm{target}] \ll 16}{\texttt{total\_transitions}}. \tag{7.2}$$

For example, 85 of 100 successors gives $(85 \ll 16)/100 =$ `0xD800`, i.e. 0.828125, an 82.8% probability.

7.3.4 Tuning Knobs

Five parameters control speculation behavior in later phases: `SPECULATION_THRESHOLD_Q48` (default 0.50, minimum confidence), `SPECULATION_DEPTH` (default 1, words ahead), `MIN_SAMPLES_FOR_SPECULATION` (default 10, observations before trusting a pattern), `MISPREDICTION_COST_Q48` (default 25 ns, recovery cost), and `MINIMUM_PREFETCH_ROI` (default 1.10, required improvement).

7.3.5 Diagnostic Words

Six FORTH words inspect the collected metrics:

```
1 PIPELINING-STATS                    ( -- )          \ dictionary-wide
      aggregate
2 PIPELINING-SHOW-STATS               ( addr len -- ) \ one word's
      metrics
3 PIPELINING-SHOW-TOP-TRANSITIONS     ( addr len N -- ) \ top N
      successors
4 PIPELINING-ANALYZE-WORD             ( addr len -- ) \ predictability
      assessment
5 PIPELINING-RESET-ALL                ( -- )          \ clear all metrics
6 PIPELINING-ENABLE                   ( -- )          \ report compile-
      time state
```

Hot words such as `EXIT`, `LIT`, `DUP`, `@`, and `!` show hundreds to thousands of transitions with strong preferred successors, while rare words show few, variable transitions and make poor speculation candidates.

7.3.6 Design Decisions

- **Lazy allocation.** Most words execute rarely; allocating a full-width array for every word would waste memory, so allocation is deferred to first use.
- **Fixed point over floating point.** Double precision was rejected because it breaks formal verification on L4Re, accumulates error over long benchmarks, and rounds platform-dependently.
- **Pointer rather than embedded struct.** A pointer permits lazy allocation and optional compilation without enlarging every dictionary entry.

7.3.7 Status and Known Limits

Phase 1 is complete: instrumentation collects transitions automatically, diagnostics report them, the build is non-breaking, and the test suite passes. Default builds collect but do not act on the data (`ENABLE_PIPELINING=0`). Known limitations are an O(DICTIONARY_SIZE) linear scan for word-ID lookup on each transition (a Phase 2 target, addressable with a `word_id` field for an estimated hundred-fold speedup), non-atomic recording unsuited to multiple threads, and manual, non-adaptive knob tuning.

7.4 Pipelining: Wired but Not Utilized

The pipelining infrastructure was asymmetric: the data-collection hooks were wired into the interpreter, but the decision functions that would act on that data were never called. The

result was a feature that appeared active—its build flag could be set, its decision function was exported—yet performed no optimization.

7.4.1 Loop #4: Transition Metrics

Collection executes when `ENABLE_PIPELINING=1`; the matching decision function is defined but never invoked.

```
/* WIRED: data collection in the inner interpreter */
if (prev_word && prev_word->transition_metrics && ENABLE_PIPELINING)
    {
    transition_metrics_record(prev_word->transition_metrics,
                              word_id, DICTIONARY_SIZE);
}

/* NOT UTILIZED: defined, exported, never called */
int transition_metrics_should_speculate(const WordTransitionMetrics
   *metrics,
                                        uint32_t target_word_id) {
    if (!metrics || !metrics->transition_heat) return 0;
    if (metrics->total_transitions < MIN_SAMPLES_FOR_SPECULATION)
       return 0;
    int64_t prob_q48 = transition_metrics_get_probability_q48(
       metrics,
                                                              target_word_id
                                                                 );
    if (prob_q48 < SPECULATION_THRESHOLD_Q48) return 0;
    return 1;
}
```

With the flag set, metrics were collected but never examined, giving a false impression of optimization.

7.4.2 Loop #5: Context Window Tuning

The binary-chop window suggestion is likewise a stub: it doubles the window by rote, is never called, and is not connected to any effective-window tuning. Its own comments concede the placeholder status.

7.4.3 Root Cause

Phase 2 was started but not finished. Phase 1 wired the collection hooks, allocated per-word metrics, and made the counting functions work. The Phase 2 decision logic was stubbed—`transition_metrics_should_speculate()` exists with a "TODO: add ROI check" note—but never integrated: nothing calls it, no prefetch action follows a positive result, and no predictive loading exists in the execution path.

7.4.4 Options

- **A — Disable until ready.** Comment out the half-wired collection in `src/vm.c` so that setting the flag honestly does nothing, reclaiming the wasted per-word memory and removing confusing code from the hot path.
- **B — Wire in the decision logic.** Complete Phase 2: call the decision function, implement a prefetch action, connect the binary-chop suggestion to the effective window, and add a DoE knob to measure the speedup. This is substantial implementation work.

- **C — Remove it.** If pipelining is not planned, delete the metrics source, header declarations, collection hooks, and the build flag.

7.4.5 Recommendation

Option A is the right move: it is honest, low-risk, removes dead code from the hot path, and is reversible when Phase 2 resumes. The disabling should be documented—why it is off, what Phase 2 entails, and how to re-enable it.

Item	State	Note
Data collection hooks	Wired	Active only if `ENABLE_PIPELINING=1`
Metrics structures	Allocated per word	Wastes memory when unused
Decision function	Defined	Never called
Header export	Yes	Appears available to users
Build flag	Default off	`ENABLE_PIPELINING ?= 0`
DoE configuration	No knob	Not tested by Phase 1 DoE

Chapter 8

Heartbeat System

8.0.1 Heartbeat Architecture: Centralised Time-Based Tuning

The heartbeat mechanism coordinates all time-driven VM operations through a single dispatcher, `vm_tick()`. Both Loop #5 (rolling-window tuning) and Loop #3 (heat-decay validation) are time-driven events that belong in one place rather than scattered through execution paths. The same dispatcher is designed to evolve into a background thread for system observability without touching core VM logic.

HeartbeatState

The `HeartbeatState` struct, declared in `include/vm.h` and embedded in the `VM` struct, holds the tick counter and per-plugin cursors:

```
typedef struct {
    uint64_t tick_count;             /* total ticks since VM init */
    uint64_t last_window_tune_tick; /* tick of last window tune */
    uint64_t last_slope_tune_tick;  /* tick of last decay validation
        */
    int      heartbeat_enabled;      /* 1 = active, 0 = disabled */
} HeartbeatState;
```

Dispatcher

`vm_tick()` in `src/vm.c` increments the global tick counter and dispatches each plugin at its configured interval:

```
void vm_tick(VM *vm)
{
    if (!vm || !vm->heartbeat.heartbeat_enabled)
        return;

    vm->heartbeat.tick_count++;

    /* Plugin 1: Window Tuning (Loop #5) */
    if (ENABLE_PIPELINING &&
        (vm->heartbeat.tick_count
            - vm->heartbeat.last_window_tune_tick) >=
               WINDOW_TUNING_FREQUENCY)
    {
        vm_tick_window_tuner(vm);
        vm->heartbeat.last_window_tune_tick = vm->heartbeat.
           tick_count;
    }
```

```
16
17      /* Plugin 2: Heat Decay Slope Validation (Loop #3) */
18      if ((vm->heartbeat.tick_count
19              - vm->heartbeat.last_slope_tune_tick) >=
                  SLOPE_VALIDATION_FREQUENCY)
20      {
21          vm_tick_slope_validator(vm);
22          vm->heartbeat.last_slope_tune_tick = vm->heartbeat.
                tick_count;
23      }
24  }
```

Window Tuner Plugin

`vm_tick_window_tuner()` in `src/physics_pipelining_metrics.c` computes the current prefetch accuracy and delegates to `loop_5_binary_chop_suggest_window()` for the next candidate size:

```
1   void vm_tick_window_tuner(VM *vm)
2   {
3       RollingWindowOfTruth *window  = &vm->rolling_window;
4       PipelineGlobalMetrics *metrics = &vm->pipeline_metrics;
5
6       if (!window->is_warm || metrics->prefetch_attempts == 0)
7           return;
8
9       double accuracy = (double)metrics->prefetch_hits
10                      / (double)metrics->prefetch_attempts;
11
12      uint32_t suggested = loop_5_binary_chop_suggest_window(
13          metrics,
14          window->effective_window_size,
15          ADAPTIVE_MIN_WINDOW_SIZE,
16          ROLLING_WINDOW_SIZE);
17
18      if (suggested != window->effective_window_size)
19          window->effective_window_size = suggested;
20
21      metrics->last_checked_window_size = window->
            effective_window_size;
22      metrics->last_checked_accuracy    = accuracy;
23      metrics->window_tuning_checks++;
24  }
```

Slope Validator Plugin

`vm_tick_slope_validator()` in `src/physics_metadata.c` scans the dictionary and classifies entries by execution heat, using execution frequency as a proxy for thermal energy. It logs the hot-word count, the stale-word ratio, and the mean heat, providing the raw signal needed to assess whether the linear decay function is removing cache pollution at the right rate. A follow-on phase will compare the stale-word ratio trend over consecutive ticks to decide whether a different decay function (exponential, logarithmic) would converge faster.

Integration into the Execution Path

Two integration modes are supported.

Synchronous (current default). The main interpreter increments an execution counter and fires `vm_tick()` every `HEARTBEAT_FREQUENCY` word executions:

```
if (++vm->execution_count % HEARTBEAT_FREQUENCY == 0)
    vm_tick(vm);
```

This adds negligible latency as long as plugin work remains lightweight.

Background thread (future). When `HEARTBEAT_THREAD_ENABLED=1`, a POSIX thread wakes at `HEARTBEAT_INTERVAL_MS` and calls `vm_tick()` independently of the execution hot-path. State shared between threads must be protected by `vm->tuning_lock`.

Configuration Knobs

Knob	Default	Meaning
`HEARTBEAT_FREQUENCY`	256	Executions between ticks
`WINDOW_TUNING_FREQUENCY`	1000	Ticks between window-tuner calls
`SLOPE_VALIDATION_FREQUENCY`	5000	Ticks between decay validations
`HEARTBEAT_THREAD_ENABLED`	0	1 = background thread
`HEARTBEAT_THREAD_INTERVAL_MS`	100	Thread wake interval (ms)

Code Locations

- `include/vm.h` — `HeartbeatState` struct; `vm_tick()` prototype
- `src/vm.c` — `vm_tick()` dispatcher; execution-path integration
- `src/physics_pipelining_metrics.c` — `vm_tick_window_tuner()`
- `src/physics_metadata.c` — `vm_tick_slope_validator()`

8.1 The Heartbeat Rate Is Variable, Not Fixed

A pivotal observation reframed the design of the Phase 2 Design of Experiments (DoE): the StarForth heartbeat is not a fixed one-millisecond tick. It is a *variable* rate that the runtime modulates in response to load. The interval is held in the `tick_ns` field of the heartbeat worker, defaulting to roughly 1,000,000 ns (1 kHz) but adjustable by the inference engine at runtime.

```
/* HeartbeatWorker */
uint64_t tick_ns;   /* default ~1,000,000 ns; modulated under load */

/* heartbeat_thread_main() */
uint64_t tick_ns = worker->tick_ns ? worker->tick_ns :
    HEARTBEAT_TICK_NS;
```

This rate modulation is the physics feedback mechanism itself, and the quality of that modulation is what differentiates one configuration from another.

8.1.1 Why the Earlier Baseline Was Incomplete

The Stage 1 baseline ran with the heartbeat thread disabled (`HEARTBEAT_THREAD_ENABLED=0`). Without the thread there is no `tick_ns` modulation, no adaptive response to load, and no physics in motion. The Stage 1 conclusion—that a handful of configurations were "stable"—measured only the absence of variation in a static tick. It could not speak to the responsiveness of heartrate modulation, because modulation was switched off.

8.1.2 The Intended Physics Story

Here the thermodynamic framing is used as a control-theory metaphor for runtime scheduling. When workload rises, the inference engine lengthens `tick_ns`, slowing the heartbeat and yielding more CPU time to parameter tuning. Physics parameters then converge faster, and the system adapts.

- A good configuration is *responsive* (`tick_ns` grows under load), *smooth* (no wild oscillation), and *efficient* (net performance improves).
- A poor configuration is *unresponsive* (rate stays fixed), *oscillating*, or *inefficient* (adaptation makes matters worse).

8.1.3 What Phase 2 Should Measure

The primary metric is heartrate modulation quality, not jitter in a fixed tick. Per run, the harness should capture:

- **Rate trajectory.** The sequence of `tick_ns` values, converted to frequency as $f_{\text{Hz}} = 10^9/\texttt{tick\_ns}$.
- **Load–heartrate correlation.** The correlation between per-tick workload and heartrate. A negative correlation (slower heartbeat under heavier load, hence more thinking time) is the desired adaptive response; near-zero correlation indicates no adaptation.
- **Stability.** Whether the rate converges to a steady state, how smoothly, and how quickly it settles.
- **Performance impact.** Whether modulation actually shortens execution time.

The golden configuration is not the one with the steadiest heartbeat but the one with the best adaptive response—ideally a strong load–heartrate coupling that settles within roughly one thousand ticks.

8.1.4 The Missing Instrumentation

At the time of this analysis the thread discarded `tick_ns` after sleeping; the trajectory was never captured. The remedy is to log a rate sample per tick into a rolling buffer carried on the VM, recording the tick number, the interval in nanoseconds, the contemporaneous workload, and a timestamp.

```
struct HeartbeatRateSample {
    uint64_t tick_number;
    uint64_t tick_ns;        /* the heartbeat rate */
    uint64_t workload_ops;   /* dictionary lookups this tick */
    uint64_t timestamp_ns;
};
```

Offline analysis then computes the load–heartrate correlation directly from the trajectory, identifying which configuration responds best to changing workload.

8.2 Implementation Plan: Heartbeat Rate Logging

This plan captures the `tick_ns` trajectory—the heartbeat rate as it actually changes during execution—so that each configuration can be analyzed for how well it modulates heartrate in response to workload. The work was scoped as a prerequisite to the Phase 2 DoE and touches four files.

8.2.1 Data Structures

A per-sample record and a buffer carried on the VM hold the trajectory.

```
#define HEARTBEAT_RATE_SAMPLE_MAX 100000

typedef struct {
    uint64_t tick_number;            /* absolute tick count */
    uint64_t tick_ns;                /* heartbeat rate (ns) */
    uint64_t workload_ops_this_tick; /* dictionary lookups this tick
        */
    uint64_t timestamp_ns;           /* wall-clock timestamp */
    uint64_t hot_word_count;         /* physics state */
    uint64_t total_heat;             /* physics state */
} HeartbeatRateSample;

/* in VM */
HeartbeatRateSample *heartbeat_rate_samples;
uint64_t            heartbeat_rate_sample_count;
uint64_t            workload_ops_current_tick;
```

The buffer is allocated with `calloc` in `vm_init()` and released in `vm_cleanup()`.

8.2.2 Capturing a Sample Per Tick

A lightweight helper writes one sample per tick, halting once the buffer is full so sampling never overruns its allocation.

```
static void capture_heartbeat_rate_sample(VM *vm, uint64_t tick_ns,
                                          uint64_t workload_ops)
{
    if (!vm || !vm->heartbeat_rate_samples) return;
    if (vm->heartbeat_rate_sample_count >= HEARTBEAT_RATE_SAMPLE_MAX
        ) return;

    HeartbeatRateSample *s =
        &vm->heartbeat_rate_samples[vm->heartbeat_rate_sample_count
            ];
    s->tick_number            = vm->heartbeat.tick_count;
    s->tick_ns                = tick_ns;
    s->workload_ops_this_tick = workload_ops;
    s->timestamp_ns           = platform_monotonic_ns();
    s->hot_word_count         = vm->hot_word_count_at_check;
    s->total_heat             = vm->total_heat_at_last_check;
    vm->heartbeat_rate_sample_count++;
}
```

The interpreter increments `workload_ops_current_tick` on each successful dictionary lookup; `vm_heartbeat_run_cycle()` latches and resets that counter per tick; and `heartbeat_thread_main()` calls the capture helper immediately before sleeping for `tick_ns`.

8.2.3 Derived Metrics

The DoE metrics structure gains heartrate fields covering rate statistics (mean frequency, standard deviation, coefficient of variation, min/max interval, modulation range), load response (correlation, response direction, adaptation and settling latency), convergence (convergence time, a converged flag, oscillation amplitude), and two composite scores for responsiveness and stability.

The analysis routine `analyze_heartbeat_rate_trajectory()` walks the captured samples to compute these. Frequency is $f_{\text{Hz}} = 10^9/\texttt{tick\_ns}$. The coefficient of variation is

$$\mathrm{CV} = \frac{\sigma_f}{\bar{f}}, \tag{8.1}$$

and the headline figure is the Pearson correlation between per-tick workload and heartrate. A correlation above a small positive threshold marks the configuration as responding in the desired direction. The responsiveness score rewards strong correlation and penalizes high CV; the stability score rewards low CV and convergence. $\mathbb{Q}_{48.16}$ fixed-point is used elsewhere in the runtime, but this offline analysis runs in floating point.

8.2.4 CSV Schema and Downstream Analysis

The CSV writers gain columns for the new heartrate fields so each run emits one augmented row. A revised R analysis then ranks configurations primarily by mean load–heartrate correlation, blended with the responsiveness and stability scores into a single golden score, and renders correlation and responsiveness box plots across configurations.

8.2.5 Build and Verification

After editing `include/vm.h`, `include/doe_metrics.h`, `src/vm.c`, and `src/doe_metrics.c`, the build is `make clean && make fastest`. Verification confirms a clean compile, a non-zero sample count after a run, the presence of the new CSV columns, computed correlation metrics, and an R ranking ordered by load response.

8.3 Per-Tick Heartbeat Instrumentation

This plan captures seven metrics on every heartbeat tick (roughly a one-millisecond interval) without disturbing timing accuracy. The strategy is a lightweight circular buffer with minimal copying, decoupled from the core heartbeat logic.

8.3.1 The Tick Snapshot

Each tick records a fixed-size snapshot. The buffer is sized for one hundred thousand ticks—about one hundred seconds at 1 kHz.

```
typedef struct {
    uint32_t tick_number;           /* sequential counter */
    uint64_t elapsed_ns;            /* since run start */
    uint64_t tick_interval_ns;      /* interval from prior tick */

    /* core metrics */
    uint32_t cache_hits_delta;
    uint32_t bucket_hits_delta;
    uint32_t word_executions_delta;

    /* feedback signals */
    uint64_t hot_word_count;        /* words above heat threshold */
    double   avg_word_heat;         /* mean heat, Q48.16 / 65536 */
    uint32_t window_width;          /* rolling window size */

    /* derived */
    uint32_t predicted_label_hits;
    double   estimated_jitter_ns;   /* deviation from nominal tick
       */
} HeartbeatTickSnapshot;

```

```
#define HEARTBEAT_TICK_BUFFER_SIZE 100000
```

The `Heartbeat` context gains the buffer, its size, a wrapping write index, a monotonic total tick count, and the run-start reference time. The buffer is allocated in `vm_init()` and freed in `vm_cleanup()`.

8.3.2 Capture and Injection

A capture routine computes the elapsed time, derives the tick interval from the prior slot in the circular buffer, and copies state from the most recent heartbeat snapshot. The mean heat is converted from $\mathbb{Q}_{48.16}$ fixed point by dividing by 65536. The call is injected at the end of `vm_heartbeat_run_cycle()`, after the tick, background decay, rolling window service, and snapshot publication.

```
static void vm_heartbeat_run_cycle(VM *vm)
{
    if (!vm || !vm->heartbeat.heartbeat_enabled) return;
    vm_tick(vm);
    vm_tick_apply_background_decay(vm, sf_monotonic_ns());
    sf_mutex_lock(&vm->tuning_lock);
    rolling_window_service(&vm->rolling_window);
    sf_mutex_unlock(&vm->tuning_lock);
    heartbeat_publish_snapshot(vm);
    heartbeat_capture_tick_snapshot(vm);   /* NEW */
}
```

8.3.3 Delta Metrics

Accurate per-tick cache, bucket, and execution deltas require latching counter values at the start of `vm_tick()` and differencing them at the end. This adds two or three counter fields to the VM struct—negligible memory. At authoring time these deltas are left as `%% TODO` stubs in the capture routine pending that tracking.

8.3.4 Export and Storage

After a run, `heartbeat_export_csv()` walks the circular buffer in order and emits one CSV row per tick. Storage is the binding constraint:

- 750 runs × 100k ticks ≈ 75 million rows, roughly 75 GB at about 1 KB per row.
- **Option A**—first and last 10k ticks plus a random sample (∼10 GB, ∼500 MB compressed).
- **Option B**—every tenth tick, 10k rows per run (∼7.5 GB).
- **Option C**—every tick in a binary format (∼2–3 GB).

Option B is recommended for Phase 2 prototyping, with density increased later if warranted.

8.3.5 Overhead

The instrumentation cost is dominated by a single monotonic clock read.

Operation	Cost
Seven field assignments	∼10 ns
One modulo (circular index)	∼5 ns
One monotonic clock read	∼100 ns
Total per tick	∼115 ns

Against a nominal 1,000,000 ns tick that is roughly 0.0115%—negligible, so the instrumentation does not perturb the timing it measures.

8.3.6 Status

The `HeartbeatTickSnapshot` structure and tick buffer are declared in `include/vm.h`, but `heartbeat_export_csv()` is not yet implemented and the delta counters remain to be wired.

8.4 Heartbeat Segfault: Root Cause and Repair

A race condition between the main VM thread and the heartbeat worker thread corrupted the `RollingWindowOfTruth` structure. Both threads wrote to shared state without synchronization, producing index corruption, concurrent writes to the execution-history array, dangling snapshot pointers, and ultimately a segmentation fault under stress tests exceeding ten thousand heartbeat cycles. The fault was systematic rather than incidental: any unlocked access to `rolling_window_record_execution()` would reproduce it.

8.4.1 The Race

The main thread records executions from the inner interpreter while the heartbeat thread services the same window:

```
1 /* main VM thread, inner interpreter */
2 rolling_window_record_execution(&vm->rolling_window, word_id);
3
4 /* heartbeat worker thread */
5 rolling_window_service(&vm->rolling_window);
```

The record routine updated `window_pos`, the `execution_history` array, `total_executions`, the warm-up flag, the snapshot-pending flag, and the adaptive-check accumulator—all without a lock.

8.4.2 Failure Modes

- **Index corruption.** Both threads read the same `window_pos`, write the same slot (one overwriting the other), and each increment the index, losing one execution and desynchronizing the position.
- **Snapshot corruption.** A double-buffer copy in progress on one thread reads partially-modified history as another thread records, yielding a torn snapshot.
- **Dangling pointer.** A snapshot view held on one thread is invalidated when the other triggers adaptive shrinking that frees or reallocates the snapshot buffers; the subsequent dereference faults.

8.4.3 Synchronization Requirements

Six fields require protection: `execution_history`, `window_pos`, `total_executions`, the two snapshot buffers, the adaptive-check accumulator, and the warm flag.

The chosen remedy is a single global lock—the existing `vm->tuning_lock`—guarding every `rolling_window_record_execution()` and `rolling_window_service()` call. The overhead is roughly one hundred CPU cycles per word execution, under one percent of system time, and contention is rare because the heartbeat runs on a separate one-millisecond cadence. A reader–writer lock was considered and deferred; the workload is write-heavy and does not justify the added complexity.

8.4.4 The Fix

Each call site is wrapped in lock and unlock, the lock is confirmed initialized in `vm_init()`, and the API header is annotated to state that `rolling_window_record_execution()` is not thread-safe and must be called under `vm->tuning_lock`.

```
sf_mutex_lock(&vm->tuning_lock);
rolling_window_record_execution(&vm->rolling_window, word_id);
sf_mutex_unlock(&vm->tuning_lock);
```

8.4.5 Validation

The fix is validated against five criteria: no segmentation fault after 100,000 heartbeat cycles; a clean Valgrind run for memory and data races; deterministic physics output across runs; all 936 existing tests passing; and zero compiler warnings under `-Wall -Werror`. The stress regime includes an aggressive 100 μs heartbeat (`make HEARTBEAT_TICK_NS=100000 test`) and a prolonged million-word execution under concurrent heartbeat.

8.4.6 Regression Prevention

A code-review rule requires any future `rolling_window_*()` call to be lock-wrapped, supported by thread-safe unit tests, stress tests in CI, and continuous Valgrind checks.

8.5 Heartbeat CSV Export

The heartbeat subsystem emits real-time multivariate VM metrics to `stderr` in CSV form, exposing StarForth's adaptive runtime to external analysis. Each row carries eleven performance indicators sampled once per heartbeat tick — nominally every millisecond — producing a continuous time series of the running system's internal state.

The design favours streaming over buffering. Metrics are read directly from VM state with no intermediate copy, each row is flushed immediately so that downstream tools see data as it is produced, and the stream is confined to `stderr` to keep `stdout` reserved for test output. The emitting thread is a background `pthread` decoupled from the main interpreter, so capture does not perturb execution timing beyond a fixed, small cost per tick.

8.5.1 CSV Format

The VM emits raw rows only; column names are supplied separately by analysis tooling. The reference header is:

```
tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,
    bucket_hits_delta,
word_executions_delta,hot_word_count,avg_word_heat,window_width,
predicted_label_hits,estimated_jitter_ns
```

The *Status* column below reflects which fields carry live values and which currently emit a placeholder zero pending delta-tracking work.

8.5.2 Capturing Metrics

The simplest capture redirects `stderr` to a file and discards `stdout`:

```
# Capture metrics, discard test output
./build/amd64/fastest/starforth --doe 2>heartbeat.csv 1>/dev/null

```

Table 8.1: Heartbeat CSV columns

Column	Type	Unit	Description	Status
`tick_number`	`uint32`	count	Monotonic tick counter since VM start	live
`elapsed_ns`	`uint64`	ns	Time since VM initialization	live
`tick_interval_ns`	`uint64`	ns	Time since previous tick (actual vs. nominal 1 ms)	live
`cache_hits_delta`	`uint32`	count	Hotwords cache hits since last tick	TODO
`bucket_hits_delta`	`uint32`	count	Bucket-level cache hits since last tick	TODO
`word_executions_delta`	`uint32`	count	FORTH word executions since last tick	TODO
`hot_word_count`	`uint64`	count	Words at or above the heat promotion threshold (≥ 10)	live
`avg_word_heat`	`double`	heat	Mean execution heat across the dictionary ($\mathbb{Q}_{48.16} \to$ double)	live
`window_width`	`uint32`	entries	Current rolling window effective size (adaptive)	live
`predicted_label_hits`	`uint32`	count	Successful pipelining speculation hits	TODO
`estimated_jitter_ns`	`double`	ns	Absolute deviation from the nominal 1 ms interval	live

```
# Keep both streams, separated
./build/amd64/fastest/starforth --doe 1>test_output.txt 2>heartbeat.
    csv
```

Because rows are flushed as they are produced, the stream can be monitored live — for example piped through `csvlook` from `csvkit`, or followed with `tail -f`. For post-processing, prepend the reference header and load the result into R or pandas:

```
{ echo "tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,\
bucket_hits_delta,word_executions_delta,hot_word_count,avg_word_heat
    ,\
window_width,predicted_label_hits,estimated_jitter_ns"; \
  cat heartbeat.csv; } > analysis.csv
```

8.5.3 Analysis Use Cases

The eleven-dimensional series supports three broad classes of analysis. For dynamics modeling, the trajectory admits phase-space reconstruction to locate attractor basins in the adaptive parameter space, Lyapunov-exponent estimation to quantify stability, and mutual-information analysis to discover coupling between metrics. For tuning validation, window-width adaptation can be correlated against prediction accuracy, heat-decay behaviour against stale word accumulation, and jitter against thread-scheduling effects. For workload profiling, the same data yields hot-word churn rate, execution density (`word_executions_delta` over `tick_interval_ns`), and cache efficiency relative to working-set size.

8.5.4 Implementation

Capture is performed by `heartbeat_capture_tick_snapshot()`, which reads the monotonic tick counter, computes elapsed and inter-tick time, walks the dictionary to count hot words and their mean heat, samples the rolling window size, and estimates jitter as the absolute difference between the actual and nominal interval. The walk is linear in dictionary size (on the order of a few hundred words). Emission is performed by `heartbeat_emit_tick_row()`, which formats the eleven values and flushes `stderr` immediately. Both are invoked once per cycle from the background heartbeat thread.

The thread runs without the original 50 ms startup delay, so metrics begin flowing immediately even for short-running DoE workloads. The trade-off is that the first few ticks may show transient values during the word-registration phase.

8.5.5 Known Limitations

Three groups of fields are not yet wired to live deltas and currently emit zero: the cache, bucket, and word-execution counters all require per-tick baseline tracking in the heartbeat state. Separately, the source notes an uninitialized `run_start_ns`, which can produce implausibly large `elapsed_ns` values in the first ticks until the field is seeded at VM initialization, and a missing aggregation of per-word prefetch hits behind `predicted_label_hits`.

The runtime cost is small: emission adds well under one percent CPU at the nominal tick rate, the snapshot is a stack-allocated structure of under a hundred bytes, and the output volume is on the order of a hundred kilobytes per second of execution.

Chapter 9

Word-Level ACL

9.1 Word-Level ACL System

StarForth's word-level access control system combines a thin C infrastructure layer with a pure-FORTH policy capsule, `ACL.4th`. The C layer supplies four `DictEntry` fields for the hot path — a TTL counter, an allow bit, a mode, and a pin — while all policy, all adaptive logic, and all inheritance rules live as colon definitions in `ACL.4th`. A bootstrap superuser, `zuse.4th`, provides the sole authenticated escalation path. The system is implemented through Phase 6; Phase 7 (LithosAnanke parity) remains. The implementation spans `capsules/ACL.4th`, `capsules/zuse.4th`, `src/word_source/acl_words.c`, and `src/test_runner/modules/acl_words_test.c`

9.1.1 Three Enforcement Dispositions

Each dictionary word holds one of three dispositions:

State	Behavior
`STRICT`	ACL re-checked on every execution
`TTL`	ACL cached; re-checked only when the TTL counter reaches zero
`PINNED`	Mode and decision frozen permanently; a one-way ratchet

9.1.2 Statistically Adaptive TTL

The TTL is not a fixed value. It is derived and continuously updated from the existing SSM execution physics. Hotter words (higher `execution_heat`) earn longer TTLs, amortizing check cost over more executions. A sudden shift in caller pattern or access rate, detected through the rolling window, shrinks the TTL aggressively. Decay pulls a quiescent word's TTL back down so the next burst re-validates early. The inference engine (Loops L5/L6) detects an oscillating TTL and stabilizes it, using the same coefficient-of-variation threshold logic already employed for window-width inference. `PINNED` is the asymptote: once the adaptive process converges, the operator thumbtacks it and the accumulator freezes permanently. A security word selects the mode per word:

```
' MY-WORD ACL-STRICT       \ check every execution
' MY-WORD ACL-TTL-MODE     \ use adaptive TTL
' MY-WORD ACL-PIN          \ freeze -- mode and decision immutable
  forever
```

9.1.3 The Pin Ratchet

`ACL-PIN ( xt - )` is a one-way ratchet. It transitions `STRICT` or `TTL` to `PINNED` and never back. Once pinned, a word's `acl_mode` and `acl_ttl` are immutable within that VM context, and any

attempt to alter a pinned word's ACL is silently ignored or errors, depending on policy. Kernel primitive words — `BIRTH`, `EXEC`, `BYE`, and the like — are pinned by Mama at boot before the first `BIRTH` fires.

9.1.4 Inheritance at Birth

When a child VM is born, ACL state propagates downward once:

```
acl_mode     = parent->acl_mode     \ policy propagates (STRICT or TTL)
acl_pinned   = 0                    \ always clear -- child owns its own lock
acl_ttl      = default              \ reset; child has no execution history
acl_decision = ALLOW                \ re-evaluated on first access
```

Two properties govern the design. The pin is contextual, not viral: a parent's pinned words do not force the child to be pinned — the child inherits the mode as a starting point and may tighten, relax, or re-pin freely. The security lattice flows downward at birth only; afterward each VM's ACL state is independent, so a compromised child cannot bootstrap its way back to Mama's pinned ACLs.

9.1.5 Interpreter Hook: Two-Level Check

The C interpreter reads only two fields per `DictEntry`:

```
if (vm->emergency_console) goto execute;   /* 100% bypass -- physical access */
if (entry->acl_ttl-- > 0) goto execute;    /* TTL good -- single decrement   */
acl_recheck(vm, entry);                    /* TTL=0: call FORTH ACL-RECHECK  */
if (!entry->acl_allow) goto reject;
```

The hot path costs one decrement and a branch — essentially free. The cold path calls `ACL-RECHECK` in `ACL.4th`, which recomputes the adaptive TTL, updates `acl_allow`, and resets the counter. The C side never reasons about policy; it only reads the result.

9.1.6 Two-Console Architecture

Two permanent, independent console layers exist at all times.

The **emergency console** is always present. Its `emergency_console` flag (a `uint8_t` in the `VM` struct) is written only by C — no FORTH word sets it — and is checked first in the interpreter hot path, producing a 100% ACL bypass; `acl_recheck()` saves and restores it for re-entrancy protection. The `EMERGENCY_CONSOLE_ENABLED` build flag (default 1) strips the interactive fallthrough when set to 0, calling the weak `vm_fault_handler()` symbol instead (overridable for hardware reset, watchdog, or debug probe). `BYE` returns to the emergency console; only `panic` kills the VM; the prompt is `ok>`.

The **Zuse console** is omnipresent and awaits authentication. Its `zuse_session` flag (also a C-only `uint8_t`) grants a full ACL bypass when a Zuse session is active. The prompt is `zuse)ok>`. The two consoles are completely independent — setting one does not affect the other.

9.1.7 Superuser: Zuse

Named for Konrad Zuse, pioneer of programmable computers, Zuse is the bootstrap superuser — the sole entity that can own the Zuse console and mint user credentials. It is defined in

`capsules/zuse.4th` and loaded by `ACL.4th` at capsule boot. For now it is software-only (no thumbdrive), yet a capsule citizen from day one. Zuse's words are pinned by `ACL-ZUSE-BOOT` at load time. `ACL-BOOT` runs first, then `S" zuse.4th" EXEC`, both from within `ACL.4th` itself (Block 2067). In future, thumbdrive PKI (Ed25519 challenge-response) will replace the software path, with `zuse.4th` becoming the bootstrapper that validates the physical drive.

9.1.8 CA Root

The certificate-authority root is embedded in `ACL.4th` (Block 2066) as the constants `ACL-CA-KEY-LO` and `ACL-CA-KEY-HI`. Because the capsule hash serves as the root-of-trust fingerprint, any change to the CA key changes the capsule hash, and the birth protocol detects the tampering. The key is a zero placeholder until `tools/mkcapsule` embeds the real Ed25519 key at build time.

9.1.9 Security Model and No-Security Condition

Security is opt-in through `init.4th`, gated by a single commented-out line:

```
1 \ S" ACL.4th" EXEC
```

`ACL.4th`	`zuse.4th`	Result
absent	absent	No security — open dev mode
present	absent	ACL enforced; emergency REPL locked until Zuse provisioned
present	present	Full lockdown; Zuse owns the Zuse console

Uncommenting the line enables full lockdown; leaving it commented keeps a development build open.

9.1.10 Self-Activating Capsule

`ACL.4th` is self-activating — `init.4th` needs only `S" ACL.4th" EXEC`. Internally, Block 2066 holds the CA root placeholder constants and Block 2067 calls `ACL-BOOT` and then `S" zuse.4th" EXEC`. `ACL-BOOT` stamps default ACL entries onto every existing dictionary word; after it runs, each subsequent `:` definition receives an ACL entry through the defining-word hook, so no word can exist without one.

9.1.11 ACL.4th: Pure-FORTH Implementation

The ACL table is a `CREATE`d FORTH array indexed by execution token (XT); `'` (tick) yields the XT of any word, and the XT is simply a cell value usable as a table key.

Word	Stack	Description
`ACL-ENTRY`	`( xt - addr )`	O(1) table lookup by XT
`ACL-MODE@`	`( xt - mode )`	Read enforcement mode
`ACL-MODE!`	`( mode xt - )`	Set mode (no-op if pinned)
`ACL-PINNED?`	`( xt - flag )`	Test pin bit
`ACL-PIN`	`( xt - )`	Set pin — one-way, irreversible
`ACL-STRICT`	`( xt - )`	Set STRICT mode (no-op if pinned)
`ACL-TTL-MODE`	`( xt - )`	Set TTL mode (no-op if pinned)
`ACL-TTL@`	`( xt - n )`	Read current TTL counter
`ACL-TTL!`	`( n xt - )`	Write TTL counter
`ACL-ALLOW@`	`( xt - flag )`	Read cached decision
`ACL-ALLOW!`	`( flag xt - )`	Write decision
`ACL-INHERIT`	`( src dst - )`	Copy mode, clear pin, reset ttl+decision
`ACL-RECHECK`	`( xt - )`	Adaptive TTL recompute; update allow + new TTL
`ACL-INIT-PRIMITIVES`	`( - )`	Bulk-initialize entries for all existing words

A typical boot-time pin in `init.4th`:

```
' BIRTH  ACL-STRICT  ' BIRTH  ACL-PIN
' EXEC   ACL-STRICT  ' EXEC   ACL-PIN
' BYE    ACL-STRICT  ' BYE    ACL-PIN
```

9.1.12 Bootstrap Sequence

```
\ In init.4th (one line, opt-in toggle -- uncomment to enable):
S" ACL.4th" EXEC        \ self-activating: ACL-BOOT then loads zuse.4
    th

\ ACL.4th Block 2067 does internally:
ACL-BOOT                \ stamp default ACLs on all existing words
S" zuse.4th" EXEC       \ load and pin Zuse's words

\ Subsequent capsule loads get ACL entries via the : hook
S" doe.4th" EXEC
```

Every `:` definition after `ACL-BOOT` creates its own ACL entry at definition time via the defining-word hook, so no word is ever born without one.

9.1.13 Capsule Namespace

File	Role
`init.4th`	Mama VM personality (default); contains the opt-in toggle
`init-*.4th`	Alternate personalities
`doe.4th`	DoE experiment harness
`ACL.4th`	Word-level ACL subsystem; self-activating; loads `zuse.4th`
`zuse.4th`	Bootstrap superuser; words pinned by `ACL-ZUSE-BOOT`
`std-blob.4th`	Standard library layer (future)

9.1.14 Implementation Status

Phases 1 through 6 are complete on `master`. Phase 1 added the four `DictEntry` fields (`acl_ttl`, `acl_allow`, `acl_mode`, `acl_pinned`), the `emergency_console` and `zuse_session` VM flags, the two-level interpreter check, and `acl_recheck()` with re-entrancy protection. Phase 2 delivered the twelve C primitive words, the XT-indexed table, the field accessors, the pin ratchet, the mode setters, `ACL-INHERIT`, `ACL-RECHECK`, `ACL-INIT-PRIMITIVES`, and the pinning of privileged words. Phase 3 made `ACL.4th` self-activating with the CA placeholder and Zuse load. Phase 4 added the `init.4th` opt-in toggle. Phase 5 contributed POST tests in `acl_words_test.c` (800/800 passing) covering pin monotonicity, inheritance, emergency bypass, STRICT and TTL behavior, the adaptive accumulator, and persistence of boot-time pins. Phase 6 added five Isabelle/HOL proofs:

- `ACL_Pin_Monotone.thy` — the pin bit is set-only; no operation clears it once set.
- `ACL_Inherit_Clears_Pin.thy` — `ACL-INHERIT` always produces an entry with `acl_pinned = 0`, regardless of source state.
- `ACL_TTL_Bounded.thy` — the TTL counter is bounded above by `ACL-TTL-COMPUTE` output and cannot grow unboundedly.
- `ACL_Emergency_Bypass.thy` — when `emergency_console = 1`, the allow/deny decision is never consulted.

- `ACL_No_Escalation.thy` — a child VM cannot produce a pinned entry with higher privilege than its inherited mode.

9.1.15 Remaining Work

Phase 7 (LithosAnanke parity) ports the subsystem to the `lithosananke` branch: verifying that `ACL.4th` loads cleanly in the freestanding kernel context, running `ACL-BOOT` at kernel boot before the first `BIRTH`, wiring `vm->emergency_console` and `vm->zuse_session` to the kernel REPL and Zuse authentication paths, and achieving three-architecture acceptance (amd64, aarch64, riscv64 booting to `ok>` with ACL active and no regressions), with acceptance logs committed. Phase 8 (future) introduces thumbdrive PKI: Ed25519 challenge-response for the Zuse drive, build-time embedding of the real CA key into Block 2066, CA-signed user minting with a home-block image, and a deliberate no-software-recovery policy — lose the drive and the administrator mints a new one.

Chapter 10

Capsule System and Mama Forth

10.1 Mama Forth: The Multi-VM Supervisor

Mama Forth is the governing FORTH VM that orchestrates a multi-VM ecosystem. It departs from conventional supervisor design in one decisive respect: Mama Forth is itself a FORTH VM, a first-class citizen carrying the same physics instrumentation as every VM it governs. The supervisor does not stand outside the runtime model; it participates in it.

The ecosystem forms a tree. Mama Forth sits at the root. Beneath it, child VMs each carry their own physics state. Beneath each child, block devices carry per-device physics. Mama observes all three layers and makes system-level decisions: when to spawn or kill a child VM, how to allocate block devices to children, how to enforce governance across the ecosystem, and how to respond to emergencies such as thermal runaway or memory pressure.

The central design insight is that physics metadata becomes a three-tier hierarchy:

- **Word level** — per-word metrics: entropy, temperature, latency.
- **VM level** — per-VM aggregates: child health, resource usage.
- **Block level** — per-block device metrics: I/O patterns, thermal distribution.

10.1.1 VM-Level Physics Metadata

Today's physics model tracks per-word metrics attached to each `DictEntry`:

```
typedef struct {
    uint16_t temperature_q8;   // Thermal state [0x0000, 0xFFFF]
    uint64_t last_active_ns;   // Last execution timestamp
    uint32_t entropy;          // Execution randomness
    int16_t  entropy_slope;    // Rate of entropy change
    uint32_t avg_latency_ns;   // P50 latency
    /* ... */
} DictPhysics;
```

The proposed extension gives each VM its own aggregate physics record. It rolls up thermal state across all words (warmest word, average temperature, count of hot words above `0x8000`), execution load (cumulative executions, words-per-second rate, recency), memory pressure (dictionary heap usage against capacity), reliability (error count, error rate, recovery count), child VM health, block-device affinity, and lifecycle state (uptime, age, run state, criticality class).

```
typedef struct {
    uint16_t vm_temperature_q8;       // Warmest word temperature
    uint16_t vm_avg_temperature_q8;   // Average across all words
    uint32_t vm_hot_word_count;       // Words with temp > 0x8000
```

```
5     uint64_t total_word_executions;
6     uint64_t execution_rate_per_sec;
7     float    dictionary_pressure_pct; // used / max
8     uint32_t error_rate_per_sec;
9     uint32_t active_child_vm_count;
10    uint8_t  vm_state;                // RUNNING, PAUSED, DRAINING,
          DEAD
11    uint8_t  vm_criticality;          // system, critical, normal,
          background
12    /* ... */
13 } VMPhysics;
```

The record attaches directly to the `VM` struct. Every physics event maintains it: `physics_metadata_touch()` updates the word, then resolves the current VM and updates the VM-level aggregates, recomputing running averages and rates.

10.1.2 Block Device Physics Metadata

Block storage (1024 blocks of 1024 bytes) is managed by `block_subsystem.c` (logical-to-physical mapping) and `blkio_factory.c` (pluggable RAM, file, and L4Re backends). Today there is no observability beyond capacity tracking. The proposal attaches a `BlockPhysics` record to each block, tracking thermal state (access frequency), access patterns (read and write counts, P50 and P99 latencies), ownership (owning VM and word), content-type hints, and health (CRC32 checksum, dirty and corruption flags, error count). A device-level aggregate summarizes total reads and writes, hottest and coldest block temperatures, and fragmentation percentage.

Block accessors maintain these metrics inline. On read, the subsystem times the I/O, updates the block's read count and exponential moving-average latency, recomputes temperature from read frequency, and verifies the CRC32 to detect corruption. On write, it does the symmetric work and refreshes the stored checksum.

10.1.3 Mama's Responsibilities

Mama Forth runs with elevated privileges along three axes:

- **Observe** — all word executions in children, all block-device I/O, all child lifecycle events, all errors and anomalies.
- **Control** — spawn and kill child VMs, allocate blocks to children, enforce governance policies, redirect critical work, scale up and down with load.
- **Coordinate** — share block resources, prevent deadlock, balance load across children, degrade gracefully under stress.

Mama is itself instrumented with a `MamaPhysics` record covering its own supervision load, child-management counts, block-management activity, emergency events (thermal warnings, memory-pressure events, orphan recoveries), and governance metrics (violations detected, enforcement actions, audit events).

10.1.4 The Supervision Loop

Mama runs a periodic control loop. Each cycle observes every child's VM-level physics, flags thermal warnings, memory pressure above 90%, and error surges, then inspects every block device for hot blocks and excessive fragmentation. It then makes load-balancing decisions, enforces governance compliance, and sleeps for an adaptive interval derived from its own decision latency.

```
void mama_supervision_loop(void) {
    while (mama_vm.running) {
        for (int i = 0; i < child_vm_count; i++) {
            VM *child = child_vms[i];
            VMPhysics p = child->physics;
            if (p.vm_temperature_q8 > THERMAL_WARNING)
                mama_handle_hot_child(child);
            if (p.dictionary_pressure_pct > 90)
                mama_handle_memory_pressure(child);
            if (p.error_rate_per_sec > ERROR_THRESHOLD)
                mama_handle_error_surge(child);
        }
        mama_balance_load_across_children();
        mama_check_governance_compliance();
        sf_sleep_ns(mama_vm.physics.decision_latency_ns * 2);
    }
}
```

Mama spawns a new child when average child temperature exceeds `0xD000` and capacity remains, allocating a block range to the newcomer and rebalancing the work queue. It kills a child that is unrecoverably faulted, idle, or under unrecoverable memory pressure — first recovering the orphan's blocks, then tearing the VM down. It rebalances block allocation by grouping each child's hottest blocks together to reduce fragmentation and keep hot data near its consumer.

10.1.5 Three-Tier Feedback Loop

Metrics flow upward. Word execution updates word temperature and latency; the VM aggregates word metrics into VM state; Mama aggregates VM state into system state, which is logged across all three tiers to the analytics heap. Decisions flow downward. A Mama decision ("Child 2 is too hot, spawn Child 3") creates a VM and allocates blocks; the affected child adjusts its batch groups and sampling rate; word-level optimization boosts priority for hot words; and the block device responds by relocating cold blocks, compacting fragmentation, and prefetching predicted accesses.

10.1.6 Worked Example: Thermal Emergency

A concrete sequence illustrates the loop. A child VM runs normally until a `BLOCK_WRITE` word heats to `0xC000` and propagates that temperature to the VM. Mama's next cycle detects the warning together with elevated dictionary pressure and an anomalous error rate of 100 errors per second. Mama responds: it spawns a second child for load shedding, reassigns cold work to it, marks the first child's hot blocks protected, temporarily raises that child's heap, and logs the emergency to governance. Within a few seconds the original child cools and stabilizes; the system keeps the new child because it now carries useful work, retiring it only once the first child cools below the safe threshold.

10.1.7 Implementation Phases

- **Phase 1 — Foundation.** Word-level physics and execution hooks are complete; VM-level aggregation and block-device physics remain.
- **Phase 2 — Single-VM Control.** Word-level scheduling and memory hints are complete; VM-level health monitoring and observe-only Mama supervision remain.

- **Phase 3 — Multi-VM Orchestration.** Child spawn and kill, block reallocation, cross-child load balancing, and emergency response.
- **Phase 4 — Mama Optimization.** Mama's own physics tuning, predictive spawning, guided load balancing, and governance enforcement at scale.

Part III

Platform and Hardware

Chapter 11

Platform Abstraction

11.1 The Platform Abstraction Layer

StarForth's platform abstraction layer provides portable timing and clock functionality across POSIX systems (Linux, macOS, BSD) and the L4Re/StarshipOS microkernel, selected by a single Makefile switch. Following the same vtable pattern as the block I/O subsystem, it achieves zero runtime overhead while keeping platform-specific code cleanly separated from the core VM.

11.1.1 Structure

The core VM is platform-agnostic and speaks only to `include/platform_time.h`. Behind that header, compile-time selection routes to one of two backends: `src/platform/time_posix.c` using `clock_gettime`, or `src/platform/time_l4re.c` using the RTC server and KIP clock. Builds choose the path explicitly:

```
make             # POSIX (default)
make L4RE=1      # L4Re/StarshipOS
make MINIMAL=1   # minimal embedded, no platform layer
```

11.1.2 API

All timing flows through `platform_time.h`. The host calls `sf_time_init()` once at startup, then reads `sf_monotonic_ns()` for performance measurement (nanoseconds since boot) and `sf_realtime_ns()` for wall-clock time (nanoseconds since the Unix epoch). `sf_set_realtime_ns()` sets the clock where privileges allow, `sf_format_timestamp()` renders a human-readable string, and `sf_has_rtc()` reports RTC availability. Helper conversions (`sf_seconds_to_ns`, `sf_ns_to_seconds`, `sf_ns_to_ms`, `sf_ns_to_us`) round out the interface.

11.1.3 Backends

The POSIX backend uses `CLOCK_MONOTONIC` for high-precision monotonic time, `CLOCK_REALTIME` for wall-clock time, and `localtime`/`strftime` for formatting; it always reports an RTC as available. The L4Re backend draws monotonic time directly from the Kernel Info Page clock (`l4_kip_clock_ns(l4re_kip())`) and real time from the RTC server offset added to that clock. It obtains the `"rtc"` capability from the L4Re environment, queries the offset over RPC, and degrades gracefully to epoch time when no RTC server is present; setting time updates the offset over RPC and requires a write capability. The selector chooses at compile time on the `__l4__` define:

```
void sf_time_init(void) {
#ifdef __l4__
```

```
3       sf_time_init_l4re();
4       sf_time_backend = &sf_time_backend_l4re;
5   #else
6       sf_time_backend = &sf_time_backend_posix;
7   #endif
8   }
```

11.1.4 Migration from Direct POSIX Calls

The abstraction replaces scattered platform calls in the core. Where the profiler once called `clock_gettime(CLOCK_MONOTONIC, ...)` directly, it now calls `sf_monotonic_ns()`; where the logger called `time`, `localtime`, and `strftime`, it now calls `sf_realtime_ns()` and `sf_format_timestamp()`. The same source compiles unchanged on both platforms.

11.1.5 Benefits

The layer keeps the core free of `#ifdef` clutter; inline functions compile to direct calls for zero overhead; the vtable gives strong typing; new platforms are easy to add; and the vtable can be swapped for mocks in testing.

11.1.6 StarshipOS Integration and Future Platforms

Copied into `StarshipOS/l4/pkg/starforth`, the build automatically selects the L4Re backend by compiling `time_l4re.c` and `platform_init.c`, linking `librtc`, and defining `__l4__=1`; the loader starts the RTC server first and grants the VM the `rtc` capability. The design anticipates further backends — bare-metal hardware timers, WebAssembly via `performance.now()`, RTOSes such as FreeRTOS and Zephyr, and other microkernels such as seL4. Adding one requires a new `time_<platform>.c`, its backend vtable, a detection branch in `platform_init.c`, and a Makefile flag.

11.2 The Hardware Abstraction Layer: Orientation

The Hardware Abstraction Layer (HAL) is the architectural seam that lets a single StarForth source base run as a hosted virtual machine, as a microkernel guest, and as the bare-metal StarKernel without forking the interpreter. This section orients the reader to the HAL document set and the roles it serves.

11.2.1 Reading Order

The HAL documentation is layered. Each document answers a distinct question and builds on the one before it.

- **Overview** — the problem the HAL solves and how it positions StarForth within the StarForth → StarKernel → StarshipOS progression. It covers the three-layer model, the design principles (VM purity, contract-first interfaces, testability), the HAL subsystems, and the relationship between the HAL and the physics subsystems.
- **Interfaces** — the contract. Precise function signatures, guaranteed semantics, error handling, performance envelopes, and the concurrency model (ISR-safe versus thread-safe) for each subsystem: time and timers, interrupts, memory, console, CPU, and panic.
- **Platform Implementations** — how to satisfy the contract on a concrete target, with complete worked examples for hosted Linux and freestanding StarKernel, plus a platform testing strategy and common pitfalls.

- **Migration Plan** — the staged refactor that moves the existing code base onto the HAL with zero functional regressions.

- **StarKernel Integration** — kernel-specific detail: the UEFI boot sequence, the free-standing C environment, hardware bring-up, and the path to a working `ok` prompt.

11.2.2 Roles

The document set serves four audiences. A VM developer reads the overview and the interfaces, then calls HAL functions and never touches a platform API directly. A platform author adds a new target reads through the implementation guide and satisfies every interface, validating against the full VM test suite. A core developer performing the migration follows the staged plan and tests after each step. A kernel developer reads everything, with emphasis on the integration guide, and builds incrementally from UEFI boot to the REPL prompt.

11.2.3 Governing Principles

The HAL rests on five principles. *VM purity*: interpreter code is platform-agnostic and never learns which platform it runs on. *Contract-first*: interfaces are contracts with precise semantics, not convenience wrappers. *Testability*: kernel-bound code is developed and tested on Linux before it ships to bare metal. *Zero overhead*: optimized builds inline HAL calls to direct hardware access. *Fail-fast*: platform assumptions are validated at initialization, not during execution.

11.2.4 Success Criteria

The HAL is judged successful when the VM core carries zero platform-specific code, all of the more than nine hundred regression tests pass on every platform, the runtime's zero algorithmic variance is preserved across platforms, no measurable performance regression is introduced, StarKernel boots to its `ok` prompt, and the physics subsystems behave identically everywhere.

11.3 HAL Architecture Overview

The Hardware Abstraction Layer is the architectural foundation that lets StarForth evolve from a hosted virtual machine into StarKernel while preserving the physics-driven adaptive runtime's deterministic behavior on every platform. It is not an afterthought bolted onto the interpreter; it is the linchpin that makes the StarForth → StarKernel → StarshipOS progression possible without compromising the experimental integrity of the runtime.

11.3.1 The Problem

StarForth runs on Linux as a hosted POSIX process, on L4Re/Fiasco.OC as a microkernel guest, and, experimentally, on bare metal. Each platform brings its own timing, interrupt handling, memory allocation, and I/O. Without an abstraction, platform-specific code bleeds into the interpreter, the physics subsystems, and the word implementations, producing fragile conditional compilation, platform-specific bugs in nominally portable code, an inability to test kernel code on a hosted platform, and a standing risk to the deterministic guarantees that the runtime depends on. With the HAL, the interpreter and physics subsystems are platform-agnostic, the platform code is isolated and testable, new platforms can be added without touching the core, and determinism is guaranteed by the HAL contract rather than by platform quirks.

11.3.2 Three-Layer Model

The architecture stacks in three layers. At the top, the VM core and physics subsystems call only HAL interfaces. In the middle, the HAL declares the platform-agnostic contract across six subsystems: time, interrupts, memory, console, and CPU, plus panic. At the bottom, each platform supplies a concrete implementation — Linux mapping onto `clock_gettime`, timers, `malloc`, standard I/O, and threads; L4Re mapping onto its clock, IRQ, dataspace, console, and thread services; StarKernel mapping onto a calibrated TSC with HPET and APIC, the IDT and APIC interrupt path, a physical and virtual memory manager with `kmalloc`, a UART and framebuffer console, and SMP bring-up.

11.3.3 Design Principles

VM purity: the core never knows its platform; all platform awareness lives in HAL implementations, replacing conditional-compilation anti-patterns with a single portable call. *Contract-first design*: each HAL function has precise semantics, defined error handling, a performance envelope, and a stated concurrency model. *Testability on hosted platforms*: kernel-bound code, such as the heartbeat ISR, is developed and tested on Linux before it reaches bare metal, using the same interface and the same VM code over a different platform layer. *Zero overhead where possible*: optimized builds inline HAL calls to direct hardware access rather than paying for function-pointer indirection. *Fail-fast validation*: a platform validates its assumptions at initialization — calibrating its timer and confirming monotonicity, for instance — and panics immediately rather than failing silently during execution.

11.3.4 The HAL and the Physics Subsystems

The adaptive runtime is the HAL's primary beneficiary, and notably it touches the abstraction only lightly. Of the physics subsystems, execution heat tracking, the hot-words cache, and the pipelining metrics depend on no HAL service at all — they operate purely on VM and dictionary state. Only the rolling window and the inference engine call `hal_time_now_ns()`, and only the heartbeat depends on the timer and interrupt interfaces. That just two of six subsystems reach the HAL, and only through clean interfaces, is precisely what preserves deterministic behavior while enabling kernel deployment.

11.3.5 The HAL and StarKernel

StarKernel is not a fork of the VM; it is a new platform implementation of the HAL. It implements the HAL functions, the UEFI boot loader, and the device drivers, and it modifies neither the interpreter core, nor the physics subsystems, nor the word implementations. That clean separation is the proof that the abstraction works: StarKernel is a platform layer, not a VM variant.

11.3.6 The HAL and StarshipOS

StarshipOS builds on StarKernel by adding a process model of Forth tasks, a filesystem, a networking stack, a unified device model, and a security model grounded in capabilities and Forth-based access control. Each of these still rests on the HAL for low-level access — the filesystem on memory and interrupts, networking on interrupts and time, the device model on the HAL as a common substrate. The HAL is therefore not merely a kernel-bootstrapping tool; it is the foundation for the entire operating system.

11.3.7 Success Criteria

The HAL succeeds when the VM core carries zero platform-specific code, the full test suite passes on Linux, L4Re, and StarKernel, zero algorithmic variance is maintained across platforms, no measurable performance regression is introduced by the abstraction, StarKernel boots to its `ok` prompt and runs the REPL, the heartbeat behaves identically everywhere, and new platforms can be added without touching VM code.

11.4 HAL Interface Specifications

The HAL interfaces define the contract between StarForth's platform-agnostic core and the platform layer beneath it. The interfaces are not optional convenience wrappers; they are the only sanctioned path by which interpreter and physics code reach platform resources. Each subsystem must be implemented with identical semantics on every platform.

11.4.1 Time and Timers (`hal_time.h`)

The time subsystem supplies monotonic time, periodic and one-shot timers, and calibration for the physics subsystems. `hal_time_init()` runs first; it calibrates a high-resolution source, validates monotonicity, and panics if no suitable source exists. `hal_time_now_ns()` returns nanoseconds since an arbitrary epoch and must be monotonic, ISR-safe, and thread-safe, with a target cost under fifty cycles on x86-64. `hal_time_delay_ns()` is a busy-wait, not a sleep. The timer family — `hal_timer_oneshot()`, `hal_timer_periodic()`, and `hal_timer_cancel()` — schedules callbacks that run in interrupt context and must complete quickly. The periodic timer is the primary mechanism for the heartbeat ISR, so the platform must minimize jitter to preserve experimental validity; the contract allows up to ten percent drift and targets under one percent.

```
1 uint64_t hal_time_now_ns(void);
2 int hal_timer_periodic(uint64_t period_ns, hal_timer_callback_t cb,
     void *ctx);
3 int hal_timer_cancel(int timer_id);
```

Platform notes: Linux maps the clock to `clock_gettime(CLOCK_MONOTONIC)` and the periodic timer to `timer_create()` with a real-time signal, accepting microsecond-scale signal jitter. L4Re uses the kernel info page clock and an IPC-based timer with an IRQ object. StarKernel reads a calibrated TSC and drives the local APIC timer.

11.4.2 Interrupts (`hal_interrupt.h`)

The interrupt subsystem enables and disables IRQs, registers ISRs, and reports interrupt context. `hal_irq_disable()` returns the prior state so a caller can restore it after a critical section; `hal_irq_register()` binds one ISR per IRQ. `hal_in_interrupt_context()` must be cheap (under ten cycles) because it gates ISR-only behavior. On Linux these map onto signals and a thread-local flag; on StarKernel they map onto `sti`, `cli`, the IDT, and the local APIC, with context tracked by an interrupt nesting count.

11.4.3 Memory (`hal_memory.h`)

The memory subsystem allocates and frees heap memory, allocates contiguous physical pages, and maps physical memory into the virtual address space. `hal_mem_alloc()` returns zero-initialized, at-least-eight-byte-aligned memory and is thread-safe but not ISR-safe. `hal_mem_map()` carries flag bits for read, write, execute, and uncached MMIO mappings. On Linux the heap calls wrap `malloc`, page allocation falls back to the heap, and mapping is a no-op under the

identity-mapping assumption. On StarKernel the calls bind to the physical memory manager, the virtual memory manager, and `kmalloc`.

11.4.4 Console (`hal_console.h`)

The console subsystem provides character I/O for the REPL and diagnostics: `hal_console_putc()` (blocking, ISR-safe for panic output), `hal_console_puts()`, the blocking `hal_console_getc()`, and the non-blocking `hal_console_has_input()`. Linux uses standard I/O; StarKernel drives a 16550 UART and, where available, a framebuffer.

11.4.5 CPU (`hal_cpu.h`)

The CPU subsystem reports identity and count, and exposes relax and halt hints. `hal_cpu_id()` and `hal_cpu_relax()` are ISR-safe; `hal_cpu_halt()` parks the processor until the next interrupt and is not ISR-safe. The relax hint compiles to `pause` on x86, to `yield` on ARM, and to `sched_yield()` on Linux.

11.4.6 Panic (`hal_panic.h`)

`hal_panic()` prints a message and never returns. On a kernel it disables interrupts and halts; on a hosted platform it writes to standard error and aborts. It is ISR-safe and reserved for unrecoverable errors.

11.4.7 Concurrency Summary

Subsystem	Key function	ISR-safe	Thread-safe
Time	`hal_time_now_ns()`	yes	yes
Time	`hal_timer_periodic()`	no	yes
Interrupt	`hal_in_interrupt_context()`	yes	yes
Interrupt	`hal_irq_disable()`	yes	yes
Memory	`hal_mem_alloc()`	no	yes
Memory	`hal_mem_map()`	no	platform-dependent
Console	`hal_console_putc()`	yes	platform-dependent
Console	`hal_console_getc()`	no	no
CPU	`hal_cpu_id()`	yes	yes
CPU	`hal_cpu_relax()`	yes	yes

Table 11.1: ISR and thread safety of the principal HAL entry points.

11.5 HAL Platform Implementation Guide

This section explains how to implement the Hardware Abstraction Layer for a new target. It walks through the hosted Linux reference and the freestanding StarKernel implementation, then sets out a testing strategy and the pitfalls that most often defeat a new platform.

11.5.1 Directory Layout and Build Integration

Each platform lives under its own directory and supplies one source file per subsystem — time, interrupt, memory, console, CPU, and panic — plus an optional initialization file. The build selects a platform and compiles only its sources. The freestanding kernel target additionally adds the flags that strip the hosted runtime.

```
PLATFORM ?= linux     # linux | l4re | kernel

PLATFORM_SOURCES = \
    src/platform/$(PLATFORM)/hal_time.c \
    src/platform/$(PLATFORM)/hal_interrupt.c \
    src/platform/$(PLATFORM)/hal_memory.c \
    src/platform/$(PLATFORM)/hal_console.c \
    src/platform/$(PLATFORM)/hal_cpu.c \
    src/platform/$(PLATFORM)/hal_panic.c

ifeq ($(PLATFORM),kernel)
    CFLAGS  += -ffreestanding -nostdlib -mno-red-zone
    LDFLAGS += -nostdlib -static
endif
```

11.5.2 The Linux Reference Platform

Linux is the reference platform, chosen for its rich debugging environment. Its implementation is thin. Time maps onto `clock_gettime(CLOCK_MONOTONIC)`, with the periodic timer built from a POSIX timer and a real-time signal whose handler dispatches the registered callback. Memory wraps `malloc` and `free`, zero-initializing every allocation to satisfy the contract, and treats page allocation as a heap allocation under an identity-mapping assumption. The console wraps standard I/O, flushing after each write and polling standard input for the non-blocking check. Interrupts are emulated: enable and disable are no-ops because signals cannot be masked through this path, and interrupt context is reported through a thread-local flag set inside the signal handler. CPU identity is fixed at zero for the single-threaded case, relax calls `sched_yield()`, and panic prints to standard error and aborts.

```
uint64_t hal_time_now_ns(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;
}
```

11.5.3 The StarKernel Freestanding Platform

The kernel platform is freestanding: no C library, no operating system, direct hardware access. Time reads the TSC, calibrated against the HPET at initialization, validates monotonicity, and panics if calibration fails; the periodic timer programs the local APIC timer, and its ISR invokes the callback and issues an end-of-interrupt. Interrupts are real: enable and disable issue `sti` and `cli`, the disable path captures and returns the flags register for later restoration, ISR registration writes the dispatch table and routes the IRQ through the IOAPIC, and interrupt context is tracked by a nesting counter incremented and decremented around each dispatch. Memory binds to the physical memory manager, the virtual memory manager, and the `kmalloc` heap, translating the HAL mapping flags into page-table flags. The console drives a 16550 UART and a UEFI GOP framebuffer in parallel. CPU identity comes from the local APIC, relax issues `pause`, and halt issues `hlt`.

```
uint64_t hal_time_now_ns(void) {
    uint64_t tsc = rdtsc();
    return tsc_to_ns(tsc) - tsc_offset_ns;
}
```

11.5.4 Testing Strategy

A new platform is validated at three levels. First, per-platform unit tests exercise each HAL function in isolation — confirming, for example, that `hal_time_now_ns()` is monotonic and that a requested delay elapses. Second, the cross-platform VM test suite must pass identically on every target. Third, determinism is validated by running the design-of-experiments harness on each platform and confirming that the output is identical across them, which demonstrates zero algorithmic variance.

11.5.5 Common Pitfalls

Four failure modes recur. *Non-monotonic time*: a TSC that runs backward under multi-core scheduling or frequency scaling corrupts the rolling window; the remedy is `rdtscp`, cross-core synchronization, or an HPET fallback. *Timer jitter*: signal latency on Linux or interrupt latency on a kernel widens heartbeat intervals and inflates metric variance; minimize ISR work, use a high-priority interrupt, and disable frequency scaling. *Memory leaks*: allocations without matching frees, often in error paths; track them with Valgrind on Linux and by hand on the kernel. *Interrupt-context violations*: calling a blocking HAL function from an ISR; guard blocking functions with an assertion that the caller is not in interrupt context.

11.6 HAL Migration Plan

This section sets out the staged refactor that moves the existing StarForth code base onto the Hardware Abstraction Layer. The migration is incremental by design: one subsystem at a time, with a passing build and a passing test suite required after every step, and platform parity maintained throughout. The governing constraint is zero functional regression — all regression tests must continue to pass and the runtime's zero algorithmic variance must be preserved.

11.6.1 Phase 1: Define the Interfaces

The first phase writes the HAL headers and nothing else. The header directory is created, the six interface headers are authored against the published contract, the build is taught to find them, and a headers-only compile confirms the tree is sound. No interpreter code changes, so the risk is low.

11.6.2 Phase 2: Implement the Linux HAL

The reference platform is implemented next. An audit locates existing POSIX calls — timing, allocation, and console I/O scattered through the tree — and each is folded behind a HAL function. Timing wraps `clock_gettime`; allocation wraps `malloc` and `free` with mandatory zero-initialization; the console wraps standard I/O; interrupts are emulated with signals; panic writes to standard error and aborts. The VM still builds against its old paths at the end of this phase — the HAL exists but is not yet called.

11.6.3 Phase 3: Migrate the VM Core

The interpreter core, the external API, the dictionary allocator, and the VM header now adopt the HAL. Direct allocation and timing calls are replaced with their HAL equivalents, HAL initialization is added to startup in dependency order, and platform conditionals are deleted in favor of single portable calls.

```
1 /* before */
2 struct timespec ts;
3 clock_gettime(CLOCK_MONOTONIC, &ts);
```

```
uint64_t now = ts.tv_sec * 1000000000ULL + ts.tv_nsec;

/* after */
uint64_t now = hal_time_now_ns();
```

This is the highest-risk phase. Compilation, link, and initialization-order errors are expected and are fixed incrementally. The phase is complete when the core builds against the HAL, the full test suite passes, and the interpreter contains no platform-specific code.

11.6.4 Phase 4: Migrate the Physics Subsystems

The physics subsystems — heat tracking, the rolling window, the hot-words cache, the pipelining metrics, the inference engine, and the heartbeat — adopt the HAL for all timing. The most consequential change is moving the heartbeat from a thread-and-sleep loop onto a HAL periodic timer whose callback runs in interrupt context. That callback must be ISR-safe: no allocation, no blocking I/O, only lock-free ring-buffer updates. Determinism is the acceptance gate here. A design-of-experiments run before and after the change must produce identical output, confirming zero algorithmic variance survives the migration.

11.6.5 Phase 5: Migrate the REPL and I/O Words

The read-eval-print loop and the I/O words (`EMIT`, `KEY`, and their relatives) move from standard I/O onto the console HAL. This phase is low risk; the success test is that the REPL behaves exactly as before.

11.6.6 Phase 6: Implement the L4Re HAL (Optional)

Implementing a second hosted platform is optional but strongly recommended, because it proves the abstraction actually abstracts. The L4Re HAL binds time to the kernel clock, memory to dataspaces, console to the L4Re console service, and interrupts to IRQ objects. Success means the VM runs on L4Re without source changes and passes its tests there.

11.6.7 Phase 7: Validate Determinism

The final phase is validation. The design-of-experiments harness is run repeatedly on Linux; every run must be byte-identical, and identical to the pre-migration baseline, demonstrating the HAL added no algorithmic overhead. The full test suite must pass and the benchmark must show under five percent performance delta from baseline.

11.6.8 Rollback and Cleanup

The migration proceeds on a dedicated branch with one commit per passing phase, so any failed phase reverts cleanly. Where a clean cut-over is risky, a feature flag can gate the HAL path against the legacy path. After validation, superseded platform code is removed, documentation is updated, and build artifacts for all platforms are ignored.

11.6.9 Timeline

After a successful migration the next work is the StarKernel platform itself: the UEFI boot loader, the freestanding HAL, and the path to an `ok` prompt on QEMU with OVMF.

Phase	Duration	Cumulative
1. Define interfaces	1–2 days	1–2 days
2. Linux HAL	3–5 days	4–7 days
3. VM core	2–3 days	6–10 days
4. Physics subsystems	2–3 days	8–13 days
5. REPL and I/O words	1–2 days	9–15 days
6. L4Re HAL (optional)	3–5 days	12–20 days
7. Validation	1 day	13–21 days

Table 11.2: Estimated migration schedule, two to four weeks depending on whether the optional L4Re phase is included.

11.7 StarKernel HAL Integration

This section gives the kernel-specific detail for the HAL platform layer that becomes StarKernel: the UEFI boot sequence, the freestanding C environment, hardware bring-up, and the path to a working `ok` prompt.

11.7.1 Boot Architecture

The kernel boots in four handoffs. UEFI firmware initializes hardware, exposes boot services — the memory map, the ACPI tables, and the graphics output protocol — and loads the boot image. The StarKernel boot loader collects that information into a `BootInfo` structure, sets up initial page tables, exits boot services, and jumps to the kernel proper. The kernel HAL stage initializes the CPU structures, the memory subsystem, the time subsystem, and the console. Finally the StarForth VM is created, its physics subsystems are initialized, the REPL starts, and the `ok` prompt appears.

11.7.2 UEFI Loader

The loader is the UEFI entry point. It collects the memory map (over-allocating to leave room for the descriptor that `ExitBootServices` will add), locates the ACPI 2.0 RSDP among the configuration tables, queries the graphics output protocol for the framebuffer base and geometry, exits boot services, and transfers control to the kernel. It never returns.

```
typedef struct {
    uint64_t memory_map_addr;
    uint64_t memory_map_size;
    uint64_t memory_map_descriptor_size;
    uint64_t acpi_rsdp_addr;
    uint64_t framebuffer_addr;
    uint32_t framebuffer_width;
    uint32_t framebuffer_height;
    uint32_t framebuffer_pitch;
} BootInfo;
```

11.7.3 Kernel Entry Point

The kernel entry point parses `BootInfo` and brings the machine up in order: it initializes the serial console first so that every later step can emit debug output, then the descriptor tables, then the physical and virtual memory managers, then the kernel heap, then the framebuffer, and finally the HAL subsystems. With the HAL ready it calls the standard `main()` to start the VM, and panics if `main()` ever returns.

11.7.4 HAL Implementation Notes

The kernel HAL rests on a handful of hardware facilities. Time calibrates the TSC against the HPET, checks for an invariant TSC, records a boot timestamp, and drives periodic interrupts from the local APIC timer.

```
uint64_t hal_time_now_ns(void) {
    uint64_t tsc = rdtsc() - boot_tsc;
    return (tsc * 1000000000ULL) / tsc_hz;
}
```

Interrupts populate a 256-entry IDT, mask the legacy PIC in favor of the local APIC and IOAPIC, and track nesting depth so that `hal_in_interrupt_context()` is exact. Memory layers a bitmap physical allocator over the parsed UEFI memory map, a four-level page-table virtual manager, and a `kmalloc` heap that zero-initializes to honor the contract. The console drives a 16550 UART at 115200 baud in polling mode and, where present, a linear GOP framebuffer with software text rendering. CPU support reports the local APIC identity and exposes the relax and halt primitives.

11.7.5 Build System

The kernel is built with a freestanding toolchain: a C compiler invoked with `-ffreestanding` and floating point disabled, a linker driven by a custom script, and `objcopy` to emit a PE32+ executable for UEFI. The linker script loads the image at the one-megabyte mark and lays out text, read-only data, data, and BSS while discarding the exception-handling frame sections.

11.7.6 Testing on QEMU

The kernel is exercised under QEMU with OVMF as the UEFI firmware. The image is copied into an EFI system partition as `BOOTX64.EFI` and booted with a serial console routed to standard output. A healthy boot prints its progress through the memory, heap, and HAL initialization stages and ends at the VM banner and the `ok` prompt.

```
qemu-system-x86_64 \
    -bios /usr/share/ovmf/OVMF.fd \
    -drive file=fat:rw:esp/,format=raw \
    -serial stdio -m 512M -enable-kvm
```

11.7.7 Debugging

Three techniques carry most of the load. Early serial output writes directly to the UART before any higher-level console exists. GDB attaches to QEMU's remote stub for source-level debugging from the entry point onward. The panic handler disables interrupts, prints its message to the console, and halts every CPU.

11.7.8 Roadmap to the `ok` Prompt

The bring-up advances through seven milestones: boot with serial output and no triple fault; working memory with a functioning physical manager, virtual mapping, and heap; a fully initialized HAL; a VM that creates and allocates its dictionary without crashing; a REPL that prints `ok`, echoes input, and executes simple words; operational physics subsystems with a firing heartbeat; and finally the full test suite passing on the kernel with zero algorithmic variance on bare metal.

Beyond the prompt lies StarshipOS: storage drivers and a filesystem, networking, a Forth-task process model, a unified device model, and a capability- and ACL-based security model — all built, as before, on the HAL.

Chapter 12

Linux and POSIX

Chapter 13

L4Re / Fiasco.OC

13.1 L4Re Integration

This guide covers running StarForth on the L4Re runtime environment over the Fiasco.OC microkernel: building it as an L4Re package, managing memory through dataspaces, communicating between VMs by IPC, and the path toward StarshipOS integration.

13.1.1 L4Re Fundamentals

L4Re is a user-level infrastructure layered on the L4 microkernel. It supplies memory management, task and thread management, inter-process communication, device drivers, and runtime libraries. A handful of concepts recur throughout the integration: a *task* is an address space plus its threads; a *dataspace* is a memory object (file-like or anonymous); a *capability* is a reference to a kernel object; an *IPC gate* is a communication endpoint; the *region manager* handles virtual memory; and the *name server* provides service discovery.

13.1.2 Package Structure

StarForth installs into the L4Re source tree as `l4/pkg/starforth`, holding a `Control` metadata file, a top-level `Makefile`, a `server/` directory with the L4Re entry point and VM wrapper, a `lib/` directory wrapping the existing `src/` files, the existing `include/`, and an `examples/` client. The `Control` file declares that the package provides `starforth` and requires `libc`, `libstdc++`, and `l4re-core`.

13.1.3 Build System

The top-level `Makefile` recurses into `lib`, `server`, and `examples`. The library Makefile builds `libstarforth` from the full set of VM and word-source files plus an L4Re-specific C++ wrapper, selecting architecture flags from `ARCH` (`-march=x86-64-v2` for `amd64`, `-march=armv8-a+crc+simd` for `arm64`) and compiling C with `-std=c99 -O3 -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -DL4RE_BUILD=1`. The server Makefile builds a static `starforth_server` program requiring `libstarforth`, `l4re_c`, `l4re_c-util`, `libstdc++`, and `libc`.

13.1.4 Memory Management via Dataspaces

Instead of `malloc`, the L4Re wrapper class `L4VM` backs VM memory with a dataspace. Initialization allocates a capability slot (`l4re_util_cap_alloc`), allocates a dataspace of `VM_MEMORY_SIZE` (`l4re_ma_alloc`), and maps it into the address space (`l4re_rm_attach` with `L4RE_RM_F_SEARCH_ADDR | L4RE_RM_F_RW`). The mapped region becomes `vm_.memory`, after which the wrapper performs the usual VM setup — zeroing stack pointers and allotting the `SCR`, `STATE`, and `BASE` cells.

Cleanup detaches the region and frees the capability; the dataspace is reclaimed when its capability is released.

13.1.5 IPC Interface

The server exposes a small opcode protocol: `OP_INTERPRET`, `OP_PUSH`, `OP_POP`, `OP_GET_STATE`, and `OP_RESET`. It is implemented as an `L4::Epiface_t` whose `op_dispatch` reads the opcode from the IPC stream and routes to a handler. `OP_INTERPRET` copies an inbound code buffer, runs `vm_interpret`, and returns the VM error status; `OP_PUSH` and `OP_POP` marshal a single cell; `OP_GET_STATE` returns the stack pointers and error/halt flags; `OP_RESET` tears down and re-initializes the VM. A matching client wraps these calls behind a small C++ class, and registration with the name server makes the server reachable under the `starforth` capability.

13.1.6 Multi-VM Architecture

A typical deployment runs several Forth VMs under a root task (Ned), which acts as name server and VM manager. Distinct VMs serve distinct roles — one hosting Forth tasks, one a request/response server, one an interactive REPL. Ned's `modules.list` wires each VM's capabilities, granting a client the server's IPC channel and giving the server its own fresh channel.

13.1.7 Kernel-Context Forth

Running Forth inside the kernel is supported for narrow uses — runtime tunables, device-driver hotpatching, and interactive kernel debugging — and demands extreme care. Kernel-context VMs have no `malloc/free` (they use a static memory pool), no syscalls, limited stack, and must be interrupt-safe and non-blocking. The kernel module registers only a minimal, side-effect-free word set (arithmetic, logical, stack) and explicitly omits I/O and blocking words. An entry point executes trusted code through `vm_interpret`, and a polling-serial REPL supports live debugging.

13.1.8 StarshipOS, Building, and Operations

On StarshipOS the natural integration points are a boot service that starts the StarForth server, system configuration expressed in Forth, runtime hot-patching, and an interactive debugging REPL; a boot script such as `/boot/init.fth` can register services and set kernel parameters during startup. Packages build per-architecture with `make ARCH=amd64` or `ARCH=arm64` (with `CROSS_COMPILE=aarch64-linux-gnu-` for cross-builds), and boot images assemble through a `modules.list` and run under QEMU. Performance work centers on huge-page-aligned dataspaces, shared memory for large transfers in place of inline IPC, and CPU affinity for cache locality. Debugging spans GDB over QEMU's stub, the Fiasco JDB kernel debugger, and L4Re's `Dbg` logging. Security rests on L4Re's capability model: VMs receive only the capabilities they need and run sandboxed under Ned configuration, deliberately withholding scheduler and allocator access from untrusted code.

13.2 Dictionary Allocation on L4Re

StarForth currently allocates dictionary entries from the host C library. The `vm_create_word()` routine calls `malloc()` for each `DictEntry`, and `FORGET` releases them with `free()`. This is correct and well-tested on hosted systems, but it ties the dictionary to a full libc heap — an awkward dependency in a microkernel context.

13.2.1 Why It Matters for L4Re

The libc-heap approach carries four liabilities under L4Re: it requires a complete `malloc` implementation in the microkernel environment; heap fragmentation makes allocation behavior non-deterministic; dictionary entries live outside the VM's own memory model in a separate address space; and the VM cannot be introspected or snapshotted as a single contiguous region.

A VM-based allocator answers each point. It removes the external dependency, a simple bump allocator behaves deterministically, everything lives inside one 5 MB VM memory region, the entire VM state can be serialized at once, and the model aligns naturally with L4Re dataspaces.

13.2.2 Proposed Approach: Conditional Compilation

A compile-time flag selects between the two strategies, leaving the hosted path untouched:

```
#ifdef STARFORTH_VM_DICT_ALLOC
    DictEntry *entry = dict_alloc_from_vm(vm, total);   /* L4Re/
        embedded */
#else
    DictEntry *entry = (DictEntry *) malloc(total);     /* hosted */
#endif
```

In VM-based mode the 5 MB region is laid out with a 256 KB dictionary-header area at the base (growing upward as entries are allocated), then user data and compiled code (growing upward from `HERE`), with block storage filling the remainder. The allocator itself is a cell-aligned bump pointer:

```
static DictEntry *dict_alloc_from_vm(VM *vm, size_t total_bytes) {
    size_t align = sizeof(cell_t);
    total_bytes = (total_bytes + align - 1) & ~(align - 1);
    if (vm->dict_headers_used + total_bytes > DICT_HEADERS_MAX) {
        vm->error = 1;
        return NULL;
    }
    DictEntry *entry = (DictEntry *)(vm->memory + vm->
        dict_headers_used);
    vm->dict_headers_used += total_bytes;
    return entry;
}
```

Because allocation is a bump pointer, `FORGET` becomes a rewind rather than a loop of frees: it sets `dict_headers_used` back to the target entry's offset. VM initialization starts `dict_headers_used` at zero and offsets `HERE` to `DICT_HEADERS_MAX`. The fast-lookup cache is unaffected — its bucket arrays may still use `malloc` while the entries themselves live in VM memory.

13.2.3 Scope of Change

The switch touches only three functions plus one new field. The new field is `size_t dict_headers_used` in the `VM` struct. The functions are `vm_create_word()` (use the VM allocator), the `FORGET` implementation (rewind instead of freeing), and `vm_init()` (initialize the offset and `HERE`). Estimated effort is roughly six hours: four to implement and two to test.

13.2.4 Current Status

The present decision is to retain `malloc()` and document the abstraction point clearly. The hosted implementation is stable and well-tested, the two approaches show no performance differ-

ence, and the abstraction is simple enough that the L4Re port can flip the switch when needed. The action items for that port are to define `STARFORTH_VM_DICT_ALLOC` in the L4Re build, implement `dict_alloc_from_vm()`, convert `FORGET` to a rewind, run the full test suite, and verify no leaks in the microkernel environment.

13.3 Block I/O Porting Endpoints for L4Re

The `blkio` subsystem maps cleanly onto L4Re because its northbound API never changes. Porting to the microkernel means supplying new backends — dataspace-based or IPC-based — behind the same six-function interface; the VM and Forth block words keep calling exactly what they call today.

13.3.1 The Stable Northbound API

Every caller goes through this fixed surface, which constitutes the subsystem ABI:

```
int  blkio_open (blkio_dev_t *dev, const blkio_vtable_t *vt, const
   blkio_params_t *p);
int  blkio_close(blkio_dev_t *dev);
int  blkio_read (blkio_dev_t *dev, uint32_t fblock, void *dst);
int  blkio_write(blkio_dev_t *dev, uint32_t fblock, const void *src)
   ;
int  blkio_flush(blkio_dev_t *dev);
int  blkio_info (blkio_dev_t *dev, blkio_info_t *out);
```

13.3.2 In-Process Dataspace Backend

When the VM can attach a dataspace directly, this is the fastest path — no IPC involved. The backend exposes `blkio_l4ds_state_size()`, `blkio_l4ds_init_state(...)`, and `blkio_l4ds_vtable()`, and holds a small state record carrying the dataspace capability, the attached virtual address, the mapped length, the block size, the block count, and a read-only flag. The operations are direct: `open` attaches the dataspace with `l4re_rm_attach()` and derives the block count; `read`/`write` are `memcpy` against the mapped region; `flush` is a no-op (or a dataspace operation if required); and `close` detaches with `l4re_rm_detach()`.

13.3.3 Out-of-Process Block Server Backend

When storage is owned by another task — an AHCI, NVMe, or eMMC server — the client-side backend exposes `blkio_l4svc_state_size()`, `blkio_l4svc_init_state(...)`, and `blkio_l4svc_vtable()`, talking to the server over a small opcode protocol:

```
enum bsvc_op {
  BSVC_INFO  = 1,
  BSVC_READ  = 2,
  BSVC_WRITE = 3,
  BSVC_FLUSH = 4,
  BSVC_PING  = 5
};
```

Two transports are available: a shared dataspace mapped by both server and client (preferred), or inline IPC carrying the block payload in the message (simpler but less efficient). The operations map onto the protocol directly — `read` sends the opcode and block number and the server fills the shared buffer, `write` copies into the shared buffer before sending, and `info` queries block size, count, and flags.

13.3.4 Factory Helpers and CLI Semantics

Backend selection can be hidden behind factory overloads — `blkio_factory_open_l4ds(...)`, `blkio_factory_open_file(...)`, and `blkio_factory_open_l4svc(...)` — so `main.c` chooses a backend from its arguments and environment. On L4Re the CLI follows the same logic: `-disk-img=<path>` resolves either to a file server returning a dataspace (the l4ds backend) or to a block server (the l4svc backend); `-ram-disk=<MB>` allocates a RAM dataspace and uses the l4ds backend; and with no disk image the system falls back to a RAM disk from the allocator.

13.3.5 Composite Backends

The intended layout dispatches by block number: blocks 0–1023 go to a RAM backend and blocks 1024 and above go to a disk backend, both wrapped in a composite vtable that routes on `fblock`. A later enhancement adds an outer wrapper that packs three 1 KiB blocks plus 1 KiB of metadata into 4 KiB pages for device alignment — still presenting the same five operations.

13.3.6 Summary

The L4Re port introduces two new backends (the in-process dataspace backend and the IPC client/server backend) and, optionally, two factory helpers. Nothing else moves: the VM continues to call `blkio_read`, `blkio_write`, and the rest. Backends are swapped, not call sites.

13.4 L4Re Integration: Platform Abstraction Layer

The StarForth platform abstraction layer was complete and tested for standalone POSIX builds at the time this document was written. The following steps remained to integrate it into the StarshipOS L4Re BID build system. This record is HISTORICAL; the integration status should be verified against the current codebase.

13.4.1 Pre-Integration Checklist

Item	Status
Platform abstraction layer implemented	Done
POSIX backend tested	Done
L4Re backend code written	Done
Standalone builds with `make L4RE=1`	Done
Platform files copied to StarshipOS tree	Pending
BID Makefile updated	Pending
L4Re build tested in StarshipOS tree	Pending

13.4.2 Integration Steps

Step 1: Copy Platform Files

Copy `src/platform/platform_init.c` and `src/platform/time_l4re.c` to `l4/pkg/starforth/server/sr` in the StarshipOS tree. Do not copy `time_posix.c`. Copy `include/platform_time.h` to `server/include/`.

Step 2: Update BID Makefile

Add `platform/platform_init.c` and `platform/time_l4re.c` to `SRC_C`. Add `librtc` to `REQUIRES_LIBS`. The BID system sets `-D__l4__` automatically (lines 77–78 in the server Makefile), so platform selection is compile-time automatic.

Step 3: Update Core Sources

In `server/src/log.c`: add `#include "platform_time.h"` and replace `get_timestamp()` with `sf_realtime_ns()` and `sf_format_timestamp()`. In `server/src/main.c`: add `sf_time_init();` before any logging.

Step 4: Verify RTC Library

Confirm `l4/pkg/rtc/` exists and `librtc.a` is present in the build tree. If absent, build with `make` in `l4/pkg/rtc/`.

Step 5: Loader Configuration

Add the RTC server to the loader script and provide the `rtc` capability to StarForth. StarForth runs without RTC capability (timestamps show 1970 epoch), but the POSIX profiler path uses the KIP clock independently of RTC.

13.4.3 Success Criteria

- StarForth builds cleanly in the L4Re tree with no linker errors
- REPL starts without crashes
- Logging shows timestamps (even if epoch)
- Profiler works with the `-profile` flag
- Tests pass with `-run-tests`

13.4.4 Rollback

Remove platform files from `SRC_C`, remove `librtc` from `REQUIRES_LIBS`, revert changes to `log.c` and `main.c`, delete the `platform/` directory, and rebuild. The L4Re build reverts to direct POSIX calls via libc.

Chapter 14

Raspberry Pi 4 / ARM64

14.1 Building StarForth on Raspberry Pi 4

This guide covers building and tuning StarForth for the Raspberry Pi 4 with its ARM64 assembly optimizations. The board pairs a Broadcom BCM2711 (quad-core Cortex-A72 at 1.5 GHz, ARMv8-A) with 32 KB of L1 instruction and data cache per core, 1 MB of shared L2, and LPDDR4-3200 memory; NEON and CRC32 are both present.

14.1.1 Building

The Pi 4 can be built natively or cross-compiled. A native build installs `build-essential`, clones the repository, and compiles with the Cortex-A72 tuning flags:

```
make␣CFLAGS="-std=c99␣-O3␣-march=armv8-a+crc+simd␣-mtune=cortex-a72␣
    \
␣␣-DUSE_ASM_OPT=1␣-DUSE_DIRECT_THREADING=1␣-Iinclude␣\
␣␣-Isrc/word_source␣-Isrc/test_runner/include"
```

Cross-compiling from x86_64 Linux installs `gcc-aarch64-linux-gnu`, builds with `CC=aarch64-linux-gn` and `LDFLAGS="-static"`, and copies the static binary to the board over `scp`. The repository provides convenience targets: `rpi4` for a native build, `rpi4-cross` for a static cross-build, and `rpi4-perf` for the maximum-performance build with direct threading and LTO.

14.1.2 Architecture Detection and Code Integration

A detection header (`include/arch_detect.h`) selects the right optimization headers and primitive macros at compile time — x86_64, AArch64, or a portable fallback with a warning. Stack management, arithmetic words, and dictionary lookup each keep an assembly path guarded by `USE_ASM_OPT` alongside a C path, so the architecture-agnostic `vm_push_opt`/`vm_pop_opt` macros resolve to the best available implementation.

14.1.3 Performance Tuning

Three techniques dominate on the Cortex-A72. Aligning the `VM` struct to the 64-byte cache line and grouping hot data (stacks and pointers) away from cold data reduces false sharing. Prefetching the next dictionary entry while comparing the current one hides traversal latency. And NEON block copy moves 64 bytes per iteration for bulk memory operations, falling back to scalar copy for the remainder.

14.1.4 Benchmarking

Reliable measurement requires pinning the CPU to performance mode and watching temperature. The governor is set with:

```
echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/
   scaling_governor
watch -n 1 vcgencmd measure_temp
vcgencmd get_throttled        # 0x0 means no throttling occurred
```

A benchmark script builds an `-O2` baseline and the optimized `rpi4-perf` binary, runs a million iterations each of stack, arithmetic, and logic loops, and reports the timing difference along with final temperature and throttling status. The `perf` tool supplies cache-reference, cache-miss, branch, and branch-miss counters for deeper analysis.

14.1.5 Thermal and Power

Sustained workloads need cooling. A passive aluminum or copper heatsink covering the entire CPU with a 1 mm thermal pad is the minimum; a 5 V 30 mm fan, optionally GPIO-controlled, handles continuous load. Power draw runs 2–3 W idle, 4–5 W under light load, and 7–8 W with all cores busy, plus 2–3 W for peripherals — the official 5 V/3 A supply is recommended. Overclocking through `/boot/config.txt` is possible but requires adequate cooling and may void the warranty.

14.1.6 Storage and Deployment

For the SD card, choose UHS-I or better with an A2 application-performance rating; brands with strong random I/O include Samsung EVO Plus, SanDisk Extreme, and Kingston Canvas React. For production, a systemd service unit runs StarForth on boot with automatic restart on failure; boot time can be trimmed by disabling unused features and the splash screen in `/boot/config.txt`; and standard hardening (system updates, a UFW firewall, and key-only SSH) rounds out a deployment.

14.1.7 Troubleshooting

- *Illegal instruction at runtime* — the build exceeded the host's feature set; rebuild with the conservative `-march=armv8-a`.
- *Cross-compiled binary will not run* — confirm it is statically linked (`file build/starforth`); rebuild with `-static` otherwise.
- *Slower than expected* — verify the governor is `performance` rather than `ondemand` and check `vcgencmd get_throttled` for thermal throttling.
- *Out of memory* — reduce `VM_MEMORY_SIZE` or add swap.

14.2 ARM64 Assembly Optimizations

StarForth's ARM64 optimizations target the AArch64 instruction set, validated primarily on the Cortex-A72 of the Raspberry Pi 4. The work spans four headers: `include/vm_asm_opt_arm64.h` (core optimizations), `include/vm_inner_interp_arm64.h` (direct-threaded interpreter), and `include/arch_detect.h` (automatic architecture detection), with a full build guide in the Raspberry Pi chapter.

14.2.1 Architectural Advantages

ARM64 offers several structural wins over x86_64 for a stack machine. With 31 general-purpose registers against 16, the implementation keeps the VM pointer, instruction pointer, and both stack pointers in registers and still has room to cache the top of stack (TOS) in `x23` — eliminating a memory access on every operation — with `x24`–`x28` held in reserve. Most ARM64 instructions carry conditional variants (`csel`, `cneg`, `cinc`) where x86_64 offers only `CMOV`, removing branches and the pipeline stalls they cause. Load and store with auto-increment fuse a memory access and a pointer bump into a single instruction, shrinking code and easing i-cache pressure. NEON SIMD compares sixteen bytes at once, accelerating string comparison and bulk memory work.

Feature	x86_64	ARM64 (Cortex-A72)
General-purpose registers	16	31
TOS caching	Limited	Excellent (`x23`)
Conditional execution	`CMOV` only	Most instructions
Load/store	Complex modes	Post-increment
SIMD width	256-bit (AVX2)	128-bit (NEON)
Power envelope	15–25 W TDP	7–8 W

Table 14.1: Architectural comparison relevant to the StarForth inner loop.

14.2.2 Expected Gains

Optimization	x86_64 speedup	ARM64 speedup
Stack operations	2–3×	2.5–4×
Inner interpreter	3–5×	4–6×
Arithmetic	1.5–2×	1.8–2.5×
Dictionary lookup	2–3×	2–3×
String operations	2–3×	2–3× (NEON)

Table 14.2: Projected speedups by optimization class.

14.2.3 TOS Caching in Practice

Because the top of stack lives in `x23` and the data-stack pointer `x21` is a true pointer rather than an index, a push collapses to a single auto-incrementing store:

```
; TOS already in x23, no load needed
str     x23, [x21, #8]!      ; store TOS, advance DSP in one
   instruction
```

Dictionary traversal benefits from prefetching the next entry while comparing the current one. Each entry carries roughly 100–200 ns of latency; a prefetch hides 50–80 ns of it, yielding a 30–50% speedup on cold searches.

14.2.4 Raspberry Pi 4 Target

The Cortex-A72 runs four out-of-order cores at 1.5 GHz with a 4096-entry branch-prediction buffer. Each core has 32 KB of L1 instruction and 32 KB of L1 data cache; 1 MB of L2 is shared. Memory bandwidth on LPDDR4-3200 is roughly 12 GB/s theoretical and 8–10 GB/s measured. Practical guidance follows from the cache hierarchy: keep hot code under 32 KB to fit L1I,

align critical loops to cache lines, prefetch predictable access patterns, structure data on 64-byte boundaries, and minimize memory traffic by keeping working data in registers.

14.2.5 Build Configuration

Architecture detection selects flags from `uname -m`: `-march=native` on x86_64, and `-march=armv8-a+crc+simd -mtune=cortex-a72` on AArch64, each with a matching `ARCH_*` define. The optimized profile adds `-O3 -DUSE_ASM_OPT=1`; the performance profile adds `-DUSE_DIRECT_THREADING=1 -flto`. Cross-compilation uses `aarch64-linux-gnu-gcc` with static linking.

14.2.6 Benchmark Results

Measured on a Raspberry Pi 4 Model B (4 GB) running 64-bit Raspberry Pi OS, kernel 6.1.21-v8+, GCC 12.2.0.

Implementation	Time	Speedup
C baseline (`-O2`)	285 ms	1.0×
C optimized (`-O3`)	198 ms	1.4×
ARM64 ASM	68 ms	4.2×
ARM64 + direct threading	42 ms	6.8×

Table 14.3: One million stack operations.

For recursive Fibonacci(30), direct threading eliminated roughly 1.7 billion branches (an 83% reduction), cut instruction count by 60%, and reduced cache misses by 75%, bringing the C baseline of 1250 ms down to 245 ms. Dictionary lookup over 1000 words across 100k searches fell from 89 ms (45k cache misses) to 41 ms (25k misses) with combined prefetch and NEON `strcmp`.

14.2.7 Energy Efficiency

ARM64 optimizations raise instantaneous power slightly through higher utilization but complete work far faster, lowering energy per operation. One million stack operations cost roughly 1.2 J on the C baseline (285 ms × 4.2 W) versus 0.19 J optimized (42 ms × 4.5 W) — a 6.3× improvement in energy efficiency. Sustained workloads warrant at least a passive heatsink; without cooling the Cortex-A72 throttles to 1.2 GHz near 80 °C.

14.2.8 Known Limitations

- The NEON string compare assumes alignment and may fault on unaligned input; an alignment check or unaligned loads would resolve it.
- `vm_mul_double` produces correct low and signed-high 64-bit halves, but unsigned 128-bit division is unimplemented; `*/MOD` falls back to software division.
- Cache-line zeroing via `dc zva` requires an aligned address and may be disabled by the hypervisor or kernel.
- The assembly is validated on Cortex-A72; other ARM64 cores (A53, A76, Apple M-series) may need retuning and should be benchmarked on the target.

14.2.9 Portability and Future Work

The same ARM64 code runs on Apple M1/M2 (wider execution and a much larger L2, expected 2–3× over the Pi 4, built with `-mcpu=apple-m1`), AWS Graviton (`-mcpu=neoverse-n1`), and Android devices via the NDK toolchain. Future directions include NEON parallel stack operations, the Scalable Vector Extension on ARMv9, and the ARMv8.3+/8.5 security features Pointer Authentication and Branch Target Identification.

Chapter 15

Performance Profiling

15.1 The Built-In Word Profiler

StarForth ships a lightweight word-execution profiler that tracks how often each word runs and turns that data into optimization guidance. Its purpose is to surface hot paths — frequently executed words — that are candidates for inline assembly. At its basic level the profiler adds well under 1% overhead, making it suitable for always-on use.

15.1.1 Quick Start

Profiling is enabled with two flags: `-profile` sets the detail level and `-profile-report` prints the report on exit.

```
./build/starforth --profile 1 --profile-report < script.fth
```

The report lists global statistics and the most frequently called words by execution count — typically dominated by `LIT`, `EXIT`, `(LOOP)`, `I`, and the core stack and arithmetic primitives.

15.1.2 Profiling Levels

Level	Overhead	Adds
0 — `PROFILE_DISABLED`	0%	Nothing (production default)
1 — `PROFILE_BASIC`	<1%	Frequency & lookup counts
2 — `PROFILE_DETAILED`	5–10%	Per-word timing (ns)
3 — `PROFILE_VERBOSE`	15–20%	Stack & memory access tracking

Table 15.1: Profiling levels and their cost.

Basic profiling tracks word frequency and dictionary lookups at negligible cost. Detailed profiling adds nanosecond-precision execution timing with average, minimum, and maximum per word. Verbose profiling further records stack operation counts and memory read/write volumes.

15.1.3 Reading the Report

High call counts mark hot paths. The detailed view reports total time per word in microseconds alongside average and maximum nanoseconds, exposing words that are individually slow as well as those that are merely frequent. A typical workload concentrates 80–95% of all executions in its top ten words, so a small set of targeted optimizations affects nearly all runtime.

15.1.4 Hot-Word Analysis

The `profiler_print_hotspots()` routine ranks words by share of total executions and attaches a recommendation to each.

Share of total	Recommendation
$\geq 5.0\%$	High priority — inline-assembly candidate
$\geq 2.0\%$	Consider assembly optimization
$\geq 1.0\%$	Monitor for optimization
$\geq 0.5\%$	Defer to profile-guided optimization
$< 0.5\%$	Low priority

Table 15.2: Optimization priority by execution share.

15.1.5 Development Workflow

The intended cycle is: profile a representative workload at the basic level, identify hot words, then optimize. Words above 5% of executions warrant a hand-written assembly path under a `USE_ASM_OPT` guard; words in the 0.5–5% band are better left to profile-guided optimization, where the compiler inlines and arranges them from the same data. Gains are confirmed by benchmarking before and after with `-benchmark`.

Best practice is to profile real programs rather than toy examples, run enough iterations for statistical significance (1000+ word executions), and start at the basic level. Profiling should avoid `LOG_DEBUG` (logging overhead skews results), focus on normal rather than error paths, and treat a top-ten coverage below 80% as a sign the workload is unrepresentative.

15.1.6 Implementation

The profiler instruments execution in two places: the outer interpreter (`vm_interpret_word()`), which sees words run directly from the REPL or scripts, and the inner interpreter (`execute_colon_word()`), which sees words run from compiled threaded code. Per-word statistics are held in a compact record:

```
typedef struct {
    const DictEntry *entry;     // dictionary entry
    uint64_t call_count;        // executions
    uint64_t total_time_ns;     // DETAILED+
    uint64_t min_time_ns;       // DETAILED+
    uint64_t max_time_ns;       // DETAILED+
} WordStats;
```

Frequency tracking is a single counter increment with O(1) lookup for known entries and O(n) only on a word's first call, with capacity for 256 unique words (expandable; primitive words are always tracked). It performs no timing calls, which is what keeps the basic level effectively free.

15.1.7 C API

```
int  profiler_init(ProfileLevel level);
void profiler_shutdown(void);
void profiler_word_count(const DictEntry *entry);   // lightweight
void profiler_word_enter(const DictEntry *entry);   // timing
void profiler_word_exit(const DictEntry *entry);
void profiler_generate_report(void);
```

```
7 void profiler_print_hotspots(void);
8 void profiler_reset(void);
```

15.1.8 Integration with External Tools

The built-in profiler complements rather than replaces system profilers. It reports hot *Forth* words; `perf` reports hot *C* functions; Valgrind/Callgrind supplies exact instruction counts; and `gprof` provides function-level timing. The recommended combination runs the StarForth profiler at the basic level continuously and reaches for the detailed level or an external tool only for targeted investigation. Typical failures are simple: no data usually means one of the two flags is missing, all-zero counts mean no Forth code executed before `BYE`, excessive overhead means a level above basic is in use, and missing words mean the 256-entry capacity was reached.

15.2 x86_64 Assembly Optimizations

StarForth's x86_64 assembly optimizations serve both standard Linux environments and L4Re microkernel deployments. They live in two headers: `include/vm_asm_opt.h` (basic stack and arithmetic primitives) and `include/vm_inner_interp_asm.h` (the direct-threaded inner interpreter).

15.2.1 Performance Impact

Optimization	Speedup	Use case
Stack operations	2–3×	Every word execution
Inner interpreter	3–5×	Colon definitions
Arithmetic operations	1.5–2×	Math-heavy code
Dictionary lookup	2–3×	Compilation/interpretation
String operations	1.5–2×	Text processing

Table 15.3: Measured impact by optimization class.

15.2.2 Enabling the Optimizations

The optimizations are gated by preprocessor flags. A standard build enables both stack/arithmetic primitives and direct threading; an L4Re build typically targets a conservative microarchitecture level:

```
1 # Standard
2 CFLAGS += -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -O3 -march=native
3
4 # L4Re/StarshipOS
5 CFLAGS += -DUSE_ASM_OPT=1 -march=x86-64-v2 -O3
```

Each optimized word keeps both an assembly and a C path behind a `USE_ASM_OPT` guard, so a build with the flag disabled falls back to the portable implementation. Stack operations, arithmetic words, and dictionary lookup all follow this pattern — the C path provides correctness and the assembly path provides speed.

15.2.3 Direct-Threaded Inner Interpreter

This is the highest-impact and most intricate optimization. A naive C interpreter dispatches each word through a function call:

```
void execute_colon_word(VM *vm) {
    cell_t *ip = vm->ip;
    while (*ip) {
        DictEntry *word = (DictEntry*)*ip++;
        word->func(vm);          // call/return overhead on every
            word
        if (vm->exit_colon) break;
    }
}
```

The call and return per word are expensive, branch prediction suffers, and registers spill. The direct-threaded path instead loads the instruction pointer and stacks into registers once, lets each primitive do its work and jump straight to the next, and saves registers only on exit. Primitives are written as thin macro bodies followed by `NEXT_ASM()`:

```
void forth_dup_fast(void)  { PRIM_DUP();  NEXT_ASM(); }
void forth_plus_fast(void) { PRIM_PLUS(); NEXT_ASM(); }
void forth_fetch_fast(void){ PRIM_FETCH();NEXT_ASM(); }
```

15.2.4 Benchmarking

A small Forth harness exercises the hot paths — `DUP`/`DROP`, arithmetic, and multi-item stack churn — over a million iterations each. The recommended methodology builds three binaries (baseline `-O2`, assembly `-O3`, and assembly plus direct threading), times each, and uses `perf record`/`perf report` and `perf stat -r 10` for detailed and repeated measurement.

15.2.5 L4Re / StarshipOS Integration

For the microkernel, the assembly optimizations build with `-march=x86-64-v2` and, in kernel context, `-fno-stack-protector` and `-mno-red-zone`, linking against `l4re-util` and `l4sys`. VM memory is allocated from an L4Re dataspace (`l4re_ma_alloc` plus `l4re_rm_attach`) rather than the host heap, and inter-VM communication packs stack data into IPC message registers via `l4_ipc_send`.

15.2.6 Debugging and Correctness

Assembly can be disabled wholesale with `-DUSE_ASM_OPT=0 -O0 -g` for debugging; under GDB the relevant state lives in `r12`–`r15`. Correctness is verified by running both the C and assembly paths on identical inputs and asserting equal results and stack pointers. The assembly sets `vm->error` on overflow or underflow but does not log, for performance; debug builds add logging through a macro that compiles away in release.

15.2.7 Platform Compatibility

Platform	Stack ops	Arithmetic	Direct threading
Linux x86_64	Yes	Yes	Yes
L4Re x86_64	Yes	Yes	Yes
StarshipOS	Yes	Yes	Yes (kernel & user)
ARM64	—	—	— (separate path)
RISC-V	—	—	— (future work)

Table 15.4: x86_64 assembly optimization support by platform.

15.2.8 Safety and Tuning

Guard pages around the data and return stacks (`mprotect` with `PROT_NONE`) provide hardware backstops beyond the in-line overflow checks. Further gains come from profile-guided optimization, cache-line alignment of the `VM` struct, dictionary-entry prefetching, and huge pages for VM memory. Common failures map to clear fixes: an illegal-instruction fault usually means `-march=native` exceeded the host's feature set (drop to `-march=x86-64-v2`); crashes in assembly typically trace to stack misalignment (the ABI requires 16-byte alignment) or incorrect clobber lists; and a lack of improvement calls for `perf` to locate the true bottleneck and a check that the optimizations are actually enabled.

15.3 Profile-Guided Optimization

Profile-guided optimization (PGO) feeds runtime profiling data back into the compiler to direct code generation. StarForth's PGO pipeline exercises every major code path, producing a binary tuned to real-world execution patterns. For typical workloads PGO yields a **5–15% improvement** over a standard `-O3` build by predicting hot paths correctly, packing hot code for instruction-cache locality, inlining frequently called functions more aggressively, and laying out code to match common execution order.

15.3.1 Quick Start

A complete PGO build runs from a single target:

```
1 make pgo
```

The target executes a six-stage pipeline: clean the environment, build with instrumentation (`-fprofile-generate`), run the comprehensive profiling workload, collect the resulting `.gcda` files, rebuild with the profile data (`-fprofile-use`), and clean up temporary artifacts. Build time is roughly two to three minutes depending on hardware.

15.3.2 PGO Targets

- `make pgo` — maximum performance for typical workloads. Runs the full profiling workload and produces an optimized binary with assembly optimizations and direct threading enabled, then removes profile data automatically.

- `make pgo-perf` — everything in the standard build plus `perf` capture during workload execution, retaining frame pointers for accurate stack traces. Produces `pgo-perf.data` for call-graph and hotspot analysis. Requires `sudo` and `linux-tools-generic`.

- `make pgo-valgrind` — runs Callgrind during the benchmark workload, collecting instruction counts, cache misses, and branch mispredictions, and builds an optimized binary retaining debug symbols. Produces `pgo-callgrind.out`. Callgrind imposes a 10–100× slowdown, so the workload is limited to 100 benchmark iterations.

15.3.3 Benchmark Comparison

The `make bench-compare` target builds both a regular and a PGO binary, runs identical benchmarks, and reports the timing difference. A representative run shows the regular build at 0.45 s elapsed and the PGO build at 0.38 s — roughly a 1.18× speedup.

15.3.4 The Profiling Workload

The workload script (`scripts/pgo-workload.sh`) drives seven major code paths: the full unit test suite (over 400 cases across all word categories), stress tests (deep call stacks, stack exhaustion, large definitions), integration tests (complete Forth programs and real-world usage), benchmarks (5000 iterations of hot-path operations), an interactive REPL workload, block I/O operations, and lightweight word-frequency profiling for hot-word identification. The workload can be run independently against any binary:

```
./scripts/pgo-workload.sh ./build/starforth
```

15.3.5 Profile Data

GCC emits two file types. The `.gcno` coverage notes are written at compile time and removed before the optimization build; the `.gcda` coverage data is written at runtime and consumed by `-fprofile-use`. Data lands in the directory where the instrumented binary runs; the build system searches `./`, `src/`, `src/*/`, and `build/`. A healthy run produces 100 or more `.gcda` files, one per source file that executed; substantially fewer indicates unexercised code paths.

15.3.6 Compiler Flags

The instrumentation stage uses moderate optimization to keep the profiling build fast:

```
CFLAGS="-O2 -DUSE_ASM_OPT=1 -fprofile-generate"
LDFLAGS="-fprofile-generate -lgcov"
```

The optimization stage applies maximum optimization, direct threading, and graceful handling of inconsistent profile data:

```
CFLAGS="-O3 -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 \
        -fprofile-use -fprofile-correction -Wno-error=coverage-
    mismatch"
LDFLAGS="-fprofile-use"
```

For `perf` analysis, `-fno-omit-frame-pointer` preserves the frame-pointer register at a 2–3% performance cost in exchange for accurate call graphs.

15.3.7 Troubleshooting

- *No profile data found* — the workload failed to exercise the relevant code, the `.gcda` files are missing, or the source changed between stages. Expand the workload and rebuild with `make pgo`, which cleans first.
- *Coverage mismatch* — source was modified between instrumentation and optimization. Run `make pgo` from scratch.
- *Permission denied writing .gcda* — the working directory is not writable; restore write permissions and rebuild.
- *PGO slower than the regular build* — the profile does not match real usage, or the workload was unrealistic. Customize the workload and confirm with `make bench-compare`.

15.3.8 Best Practices

Profile representative workloads that cover the bulk of real usage rather than trivial operations or rarely executed error paths. Re-run PGO after major code changes (more than roughly 10% of the codebase) and always verify gains with `make bench-compare`. PGO composes well with

`-march=native`, link-time optimization, direct threading, and hand-written assembly — `make pgo` enables all of these automatically.

15.3.9 How PGO Works

During instrumentation the compiler inserts counters at control-flow edges; the runtime increments them and writes `.gcda` files on exit. During optimization the compiler reads the profile, separates hot from cold paths, and makes informed decisions: inlining hot functions, packing hot code for I-cache locality, arranging branches for correct prediction, devirtualizing calls where the profile fixes the type, and unrolling hot loops. Hot code is laid out sequentially while cold code is outlined to a separate section.

Heuristic	Default	With PGO
Inline threshold	600 units	Adjusted per call site
Loop unroll factor	4	Up to 8 for hot loops
Branch prediction	Static (50/50)	Dynamic (from profile)
Function outlining	Disabled	Cold code outlined
Register allocation	Balanced	Favors hot paths

Table 15.5: GCC optimization heuristics with and without profile data.

15.3.10 Build Target Comparison

Target	Optimization	Speed	Build Time	Use Case
`debug`	`-O0 -g`	1.0×	30 s	Development
`all`	`-O2`	3.5×	45 s	Default, balanced
`fast`	`-O3` + ASM	5.2×	60 s	Production, no LTO
`fastest`	`-O3` + ASM + DT + LTO	6.8×	90 s	Maximum
`pgo`	fastest + PGO	7.5–8.0×	3 m	Absolute maximum

Table 15.6: Relative performance and build cost across targets (ASM: assembly optimizations; DT: direct threading; LTO: link-time optimization).

15.3.11 Platform Notes

On x86_64 all PGO features are available, with best results under `-march=native`; `perf` hardware counters and Valgrind are fully supported. On ARM64 (Raspberry Pi 4, Apple Silicon) PGO is fully supported — use `-march=armv8-a+crc+simd -mtune=cortex-a72` on the Pi 4; `perf` support varies by kernel and Valgrind support is limited on some platforms. PGO requires native execution to profile, so cross-compilation must run the instrumented binary on target hardware, transfer the `.gcda` files back, and cross-compile the optimized binary with that data.

Part IV

Specifications

Chapter 16

HOLA Analytics Protocol

16.1 The HOLA Shared-Memory Protocol (Phase 1)

Phase 1 establishes the Host Observation & Logistics Adapter (HOLA) as the stable contract between the StarForth VM and external analyzers. The runtime exposes a fixed-size analytics heap — 10 MiB by default — implemented in `physics_runtime.c`. This section specifies the shared-memory schema and command flow that Phase 1 tooling expects.

16.1.1 Memory Layout

The analytics heap divides into four regions. The header carries single-producer/single-consumer metadata identified by `HOLA_SHARED_MAGIC`; the event ring is a lock-free buffer of physics events; the summary region holds aggregated statistics and Bayesian posteriors; and the scratch region is reserved for governance-approved features.

Region	Offset	Size	Notes
Header	`0`	`sizeof(header_t)`	Producer/consumer metadata
Event ring	`ring_offset`	`ring_bytes`	Lock-free event ring
Summary	`summary_offset`	`summary_bytes`	Stats & posteriors
Scratch	`scratch_offset`	`scratch_bytes`	Reserved

Table 16.1: HOLA analytics heap regions.

The header mirrors the C structure in `include/physics_runtime.h` and must remain packed and aligned exactly as defined. All multi-byte fields use the VM's native endianness. Consumers must validate `magic`, `version_major`, and `version_minor` before touching the heap.

16.1.2 Event Records

Each ring entry begins with a fixed header:

```
struct physics_analytics_event_header {
    uint32_t channel;         // logical stream id
    uint16_t payload_bytes;   // raw payload size
    uint16_t reserved;        // alignment padding (zero)
    uint64_t timestamp_ns;    // producer timestamp (sf_monotonic_ns)
};
```

Payloads are padded to eight bytes. The producer updates `produce_seq` atomically after writing each record; consumers advance `read_offset` and bump `consume_seq` once an event is fully processed. Overflow is signalled by incrementing `dropped_events` while leaving the ring untouched, and consumers must treat the next readable event as the resynchronization point.

16.1.3 Channels

Channel	Purpose
`0x00000001`	Word execution samples (entropy, latency, dictionary id)
`0x00000002`	Host snapshot deltas (`physics_host_snapshot_t`)
`0x0000FF00`–`0x0000FFFF`	Governance-reserved control/alert events

Table 16.2: Recommended HOLA channels.

Channel `0x00000002` carries a binary `physics_host_snapshot_t` in native endianness. Phase 1 adds Pressure Stall Information averages (`avg10/60/300` scaled by 1000), `/proc/stat` totals (`cpu_total_jiffies`, `cpu_idle_jiffies`) for consumer-side delta calculation, cgroup v2 readings (`cgroup_cpu_usage_us` from `cpu.stat`, `cgroup_memory_current_bytes` from `memory.current`, each zero when unavailable), and a `flags` bitmask (`PHYSICS_HOST_FLAG_*`) indicating which groups were populated. Consumers should honor the flag bits to distinguish kernels that expose PSI, cgroup v2, or `/proc/stat`.

16.1.4 Command Protocol

Commands travel through the summary region; its first 256 bytes form a simple request/response mailbox:

```
struct hola_command {
    uint32_t opcode;       // see table
    uint32_t arg0;         // optional argument / status
    uint64_t arg1;         // optional payload offset from heap base
};
```

Opcode	Direction	Semantics
`HOLA_NOP`	Either	No-op; producer clears when idle
`HOLA_REQUEST_SNAPSHOT`	Consumer → VM	Publish snapshot on channel `0x2`
`HOLA_RESET_RING`	Consumer → VM	VM zeroes cursors when safe
`HOLA_STATUS_OK`	VM → Consumer	Command completed
`HOLA_STATUS_BUSY`	VM → Consumer	Deferred; retry next tick
`HOLA_STATUS_UNSUPPORTED`	VM → Consumer	Opcode not understood

Table 16.3: HOLA command opcodes.

The VM polls the mailbox at the end of each observation-window hop and writes responses back into the same structure before the next poll. Phase 1 implements only `HOLA_REQUEST_SNAPSHOT`; the remaining opcodes are reserved.

16.1.5 Synchronization Rules

- **Single producer**: the VM is the sole writer to the ring; multiple producers must funnel through the runtime shim.
- **Single consumer**: Phase 1 assumes exactly one analyzer; multi-consumer support will require an indirection table in the summary region.
- **Memory ordering**: producers write payload, then header, then bump `produce_seq`; consumers read `produce_seq` before pulling events and bump `consume_seq` after advancing `read_offset`.

- **Failure semantics**: when `dropped_events` increases, the consumer treats the last coherent boundary as unknown and resynchronizes from the first complete record after the drop.

16.1.6 Artefacts

The runtime implementation lives in `include/physics_runtime.h` and `src/physics_runtime.c` (`physics_runtime_init`, `physics_host_snapshot`, `physics_analytics_publish_event`). The formal model linkage is in `Physics_StateMachine.thy` and `Physics_Observation.thy`, which capture the observation-window semantics this protocol assumes. Future revisions will document multi-node governance flows and captable wiring for L4Re tasks once Phase 2 moves the analyzer into a dedicated microkernel service.

Chapter 17

Formal Verification

17.1 VM Formalization Roadmap

This roadmap records the staged approach to formally proving the StarForth virtual machine in Isabelle/HOL. It grows alongside the `.thy` theory files in the formal proof tree and serves as the architecture map for VM-level verification.

17.1.1 Objectives

The plan pursues three goals. It preserves room for architectural refactoring while the proofs evolve. It treats the physics-metadata proofs — `Physics_StateMachine` and `Physics_Observation` — as the first pillar, onto which additional modules snap within the same Isabelle session. And it keeps proof obligations discoverable from the C headers, documenting invariants close to the runtime code they constrain.

17.1.2 Session Layout

The verification workspace is organized as a layered set of theories. The lower layers are scaffolded and build independently of the physics layer; the physics theories remain fluid; the upper layers are planned.

Theory	Purpose (current status)
`VM_Core`	VM state record, pointer accounting, control-flag helpers (scaffolding).
`VM_Stacks`	List-backed stack model locale and depth lemmas (scaffolding).
`VM_Words`	Abstract wrappers for `>R`/`R>` transfers (scaffolding).
`VM_Register`	Dictionary model for `register_word` (scaffolding).
`Physics_StateMachine`	Ring-buffer and mailbox invariants for HOLA (fluid).
`Physics_Observation`	Per-word physics updates, frame composition (fluid).
`VM_Dictionary`	*Planned:* dictionary lifecycle, physics wiring.
`VM_Interpreter`	*Planned:* interpreter state machine, semantics.
`VM_IO`	*Planned:* host capability shims, scheduler obligations.

The `VM_Formal` session captures `VM_Core` through `VM_Register` and builds without the physics layer. `Physics_Formal` extends it, keeping the observation machinery available while acknowledging that those theories remain fluid. Each planned theory imports the prior layers so the whole workspace stays incrementally checkable via `isabelle build`.

17.1.3 Development Guidelines

- **Pair source and proof** — when a C module gains an invariant, add or update the matching theory and cross-reference it in file-level comments.
- **Prefer locales** — define subsystem properties as locales and extend them in follow-up theories rather than rewriting existing ones.
- **Document assumptions** — every lemma states the runtime assumptions it relies on, such as stack-depth bounds or profiler hooks.
- **Keep proofs fast** — favor algebraic lemmas over brute-force tactics so developers can replay the session during normal builds.
- **Track TODOs** — leave `text` blocks or FIXME notes in the theory files marking which invariants still need encoding.

17.1.4 Next Actions

The immediate agenda is to land a green `VM_Formal` build before layering physics obligations; flesh out `VM_Core` with stack-depth and dictionary invariants once the C helpers stabilize; introduce a `VM_Stacks` theory capturing data and return-stack well-formedness and pointer safety (aligned with `STRICT_PTR`); model dictionary entries against `DictEntry` and connect the physics-metadata lemmas to the compiler pipeline; decide the C-to-Isabelle modeling path (AutoCorres versus manual abstraction) before tackling the interpreter loop; and wire a Makefile target to `isabelle build` so CI can gate on proofs as they expand.

17.1.5 Multi-Day Proof Checklist

The longer-horizon checklist proceeds in stages. A **session baseline** records the theory order in the project `ROOT` and snapshots current failures. **VM core invariants** expand `VM_Core` with pointer and limit invariants and error-flag rules, each phrased as a lemma inside the `vm_core` locale for reuse. **Stack model helpers** add peek/push/pop transfer lemmas for each stack direction, proving both success and guard-failure variants alongside runtime versions that expose the `dsp`/`rsp` relationship. **Word semantic wrappers** split per-module theories that import `VM_Words` and derive success and no-op lemmas in one or two lines. **Dictionary registration** centralizes lemmas in `VM_Register`, guarding reserved names. A **module-march template** repeats, for each of the roughly nineteen word modules, the cycle of identifying the C helper, confirming an Isabelle helper, creating the wrapper lemma, registering the word, and rerunning the session, with progress tracked in a status table. **IO and external hooks** stub locale assumptions for side-effecting words, and **housekeeping** rebuilds the session nightly, treating a green build as the commit gate. The **stretch goal** is a consolidated theorem: that all registered words satisfy their claimed semantics, combining the lookup lemmas with the per-word proofs.

17.1.6 VM-First Verification Roadmap

The proof order works upward from primitives. It begins with the **stack primitives** (`vm_push`, `vm_pop`, `vm_rpush`, `vm_rpop`), proving they refine the abstract transfers and raise `vm_error_active` precisely on overflow and underflow. It then proves the **return-stack words**, the **data-stack transfer words** (`DROP`, `DUP`, `SWAP`), the **arithmetic and logic modules**, the **dictionary and lookup words** (`CREATE`, `FIND`, `IMMEDIATE`), the **control-flow words**, the **compiler and interpreter infrastructure** (proving the interpreter halts on `vm_error_active` and honors abort and exit flags), and finally the **system and I/O words**, stubbing the required capabilities.

17.1.7 Status Snapshot

Stack and return-stack semantics are complete: `vm_push_sem`, `vm_pop_sem`, `vm_rpush_sem`, and `vm_rpop_sem` carry success, overflow, and underflow lemmas in `VM_StackRuntime.thy`. The parameter-stack words `DROP`, `DUP`, `?DUP`, `SWAP`, `OVER`, `ROT`, `-ROT`, `DEPTH`, `PICK`, and `ROLL` have proven abstract semantics, runtime correspondence, and wired dictionary registration. The return-stack words `>R`, `R>`, and `R@` have established abstract semantics and runtime equality. The model now exposes `cell_to_int`/`int_to_cell` conversions, and the `VM_Formal` session builds cleanly with the theories ordered `VM_Core` → `VM_Stacks` → `VM_StackRuntime` → `VM_DataStack_Words` → `VM_ReturnStack_Words` → `VM_Words` → `VM_Register`.

The next milestones introduce an instruction-pointer and memory abstraction so `(BRANCH)` and `(OBRANCH)` can be specified without referencing C internals; model loop frames on the return stack (`?DO`, `DO`, `LOOP`, `+LOOP`, `LEAVE`, `I`, `J`); formalize the compile-time control-flow stack so `IF`, `ELSE`, and `WHILE` can be reasoned about at registration time; and then extend the dictionary lemmas toward the arithmetic and logical modules.

Achieving these steps positions the project to claim that StarForth is, to the authors' knowledge, the first open-source FORTH VM accompanied by a full proof corpus of its core primitives in Isabelle/HOL, enabling proof-carrying reference builds and machine-verified stack discipline.

Appendix A

Physics Design Notes

A.1 The Cone of Influence

The cone of influence is a feedback-control architecture for the physics engine. It is named for a geometric metaphor used purely as a modeling device: the region of possible execution futures is treated as a cone that narrows over time. As the runtime accumulates observations, asynchronous and unpredictable publish/subscribe messaging is progressively constrained into deterministic, ordered information flow.

The driving idea is a closed loop. Observables from the early phase inform statistical models. Those models adjust runtime knobs. The knobs change behavior. The changed behavior generates new observables. Iterated, the loop moves the system from high-uncertainty asynchronous communication toward low-uncertainty deterministic ordering.

A.1.1 The Model

The cone occupies a conceptual three-axis space: a time axis (past to future), a scope axis (narrow word-level interactions to wide global scope), and a determinism axis (asynchronous fire-and-forget, through semi-deterministic retry-with-hints, to deterministic causal ordering). The cone narrows along three dimensions simultaneously:

- **Temporal** — past observations constrain present behavior.
- **Spatial** — an initially global scope resolves to specific word interactions.
- **Modal** — asynchronous uncertainty resolves to deterministic guarantees.

A.1.2 Three Operational Zones

Zone 1 — Wide and Async (Discovery). During the first N observations the system bootstraps. Many execution paths remain possible, entropy variance is high, latency is unpredictable, and scope is global. Publish events fire-and-forget into a ring buffer with no ordering guarantees. The goal is simply to discover which topics words publish to. Overhead is minimal, on the order of 200 ns. Observables available: entropy changes, temperature jumps, host snapshots, error rates.

Zone 2 — Narrowing and Semi-Deterministic (Learning). After M observations, entropy variance stabilizes, temperature reaches equilibrium, and the latency distribution narrows. Scope contracts to module level. Publish events now carry causal ordering hints: each message is tagged with a Lamport clock and a likely-destination set, and related messages are batched. The goal is to learn the dependency graph and subscriber patterns. Overhead rises to roughly

250 ns. New observables: entropy slope, temperature stability, latency percentiles (p95, p99), and the causal dependency graph.

Zone 3 — Narrow and Deterministic (Enforcement). In steady state, entropy plateaus, temperature is bounded by throttling, and latency carries guarantees against a service-level objective (SLO). Scope is specific: a word sends only to known targets. Publish events are ordered deterministically — buffered and sorted by causal order number — and a batch deadline of at most T ms is enforced. The p99 latency SLO is guaranteed, with backpressure applied on violation, and a subscriber whitelist eliminates surprises. Overhead settles at roughly 300 ns. The full observable set adds latency variance, thermal slope, and execution-pattern periodicity.

A.1.3 Metrics Inventory

The engine already captures a substantial baseline. Per-word metrics include `DictEntry.entropy` (raw execution count), and within `DictEntry.physics`: `temperature_q8` (EMA hotness), `last_active_ns` (recency), `mass_bytes` (footprint), `avg_latency_ns` (rolling average), and `state_flags`. Two fields, `acl_hint` and `pubsub_mask`, are reserved stubs.

Host-level snapshots feed the analytics heap from real POSIX and procfs sources:

Signal	Source	Status
`monotonic_time_ns`	`clock_gettime(CLOCK_MONOTONIC)`	Real
`cpu_count`	`sysconf(_SC_NPROCESSORS_ONLN)`	Real
`scheduler_policy`	`sched_getscheduler(0)`	Real
`scheduler_quantum_ns`	`sched_rr_get_interval()`	Real
`load_avg_milli`	`getloadavg()`	Real
`psi_cpu/io/mem_*_milli`	`/proc/pressure/*`	Real (4.20+)
`cpu_total/idle_jiffies`	`/proc/stat`	Real
`cgroup_cpu_usage_us`	`/sys/fs/cgroup/cpu.stat`	Real (cgroup v2)
`cgroup_memory_current`	`/sys/fs/cgroup/memory.current`	Real (cgroup v2)

Profiler data (call frequency, total time, latency distribution, memory read/write patterns) and test-infrastructure outcomes round out the current set.

Several signals are not yet captured but are reachable: stack and return-stack depth at execution, dictionary lookup latency, block I/O queue depth, the data-flow graph of word-to-word calls, and the error rate. Useful ML features remain on the horizon: execution-pattern history, latency percentiles, entropy and thermal slopes, instruction-count estimates, and cache-miss rates where performance counters exist. Some quantities are out of reach by definition — future execution patterns require prediction, optimal scheduling is NP-hard, and ground-truth causality runs into observer effects.

A.1.4 The Control Layer: Runtime Knobs

Per-word knobs include execution priority (tuned by temperature), cache affinity (by entropy slope), temperature target, entropy half-life (cooling rate), latency limit (the SLO), stack-depth limit (recursion guard), and ACL level. System-level knobs include sampling cadence, analytics-heap size, profiler mode, pub/sub batch size and timeout, temperature smoothing alpha, and entropy decay rate. Each knob is driven by a specific tuning signal — for example, the latency limit responds to p99 measurements, and the sampling cadence responds to latency variance.

A.1.5 The Pub/Sub Gateway: Three Phases

The gateway is where words publish observations and where the control system acts. Its behavior tracks the three zones.

In **Discovery**, a word calls `physics_analytics_publish_event(topic, payload, size)` and the event is hash-inserted into the ring buffer; consumers poll when ready, with no ordering.

```
// Zone 1: fire-and-forget
physics_analytics_publish_event(PUBSUB_TOPIC_X, payload, size);
```

In **Learning**, the gateway identifies topic dependencies, learns subscriber sets, stamps each message with a Lamport clock, and tags it with a subscriber hint.

```
// Zone 2: learning phase
message.causal_order_num = ++g_causal_clock;
message.subscriber_hint  = learned_targets;  // "probably {WordY, WordZ}"
physics_analytics_publish_event(PUBSUB_TOPIC_X, message, sizeof(message));
```

In **Enforcement**, causal ordering is enforced by buffering and sorting on the order number; a batch deadline flushes pending messages; the p99 SLO is verified; delivery is restricted to a whitelist; and a full queue triggers backpressure on the publisher.

```
// Zone 3: enforcement phase
message.causal_order_num   = ++g_causal_clock;
message.subscriber_targets = learned_whitelist;   // exact set, no surprises
message.deadline_ns        = now_ns + SLO_LIMIT_NS;
```

A.1.6 The Feedback Loop

The control law maps observed state, through a detection rule, to a knob adjustment. A very hot word with rising entropy lowers execution priority, shortens the entropy half-life, tightens the stack limit, and raises the sampling cadence. A p99 latency that persistently exceeds the SLO tightens the stack-depth limit, the batch timeout, and the batch size. A reached entropy plateau — low slope, low variance — raises cache affinity, the temperature target, and execution priority. Excessive latency variance raises sampling cadence and profiler detail. An error-rate spike restricts the ACL level and lowers priority. Scheduler noise from core migration raises cadence and pins affinity.

A hot-word emergency illustrates the loop end to end: a word saturates at `0xFFFF` with a climbing entropy slope; the controller lowers OS priority, accelerates cooling, caps recursion, raises sampling, and isolates the word via the ACL level; the next cycle checks whether the temperature stabilized, restoring normal priority on success or escalating to throttling and governance alert on failure; offline analysis later mines the event for better detection heuristics.

A.1.7 ML and Statistical Feedback (Future)

Once the three zones operate and history accumulates, the design anticipates four further phases: predictive models for latency, hotness, and optimal batch size; anomaly detection for execution-pattern shifts, phase changes, SLO violations, and early thermal runaway; optimization of core affinity, batching strategy, priority ordering, and profiler mode; and governance integration with machine-checked proofs in Isabelle — observable invariants, a cone-narrowing convergence theorem, verified control loops, and proven SLO bounds.

A.1.8 Open Design Questions

The design records several unresolved choices, each with a working recommendation:

- **Zone transition** — time-based, confidence-based, or hybrid. Recommendation: hybrid (start time-based, add confidence).
- **Ordering strength** — causal only, total, or causal-within-module and total-globally. Recommendation: hybrid.
- **SLO scope** — global, per-word, or governance-derived per category. Recommendation: derived.
- **Loop style** — open-loop, closed-loop, or hybrid. Recommendation: hybrid, with closed-loop reserved for critical knobs.
- **ML training** — in-system, offline, or hybrid. Recommendation: hybrid, with online anomaly response and offline refinement.

A.1.9 Glossary

- **Causal ordering** — messages ordered by dependency ($A \to B$ if A must precede B).
- **Lamport clock** — a scalar clock that respects causality.
- **Cone of influence** — the region of possible execution futures, narrowing as confidence grows.
- **Zone** — an operating region (async discovery, semi-deterministic learning, deterministic enforcement).
- **Knob** — a runtime-adjustable parameter affecting behavior.
- **SLO** — service-level objective (a latency guarantee).
- **Backpressure** — flow control that slows publishers when a queue fills.
- **Entropy** — execution count per word, an implicit hotness measure.
- **Temperature** — the exponential moving average of entropy, an explicit hotness measure.
- **Half-life** — the time for temperature to decay to half its peak.

A.2 From Observability to Control

The physics engine answers the question of what the system is doing. Scheduling and memory management answer the question of what it should do differently. The physics engine supplies observability — entropy slope, temperature trend, p99 latency. Scheduling and memory management supply control: execution placement and data placement respectively, joined by feedback-control loops.

Three subsystems consume physics signals. The scheduling system governs word priority, core affinity, preemption hints, latency SLOs, and throttling. The memory-management system governs cache tier, block placement, garbage-collection triggers, and eviction policy. Between them, feedback loops close observation onto action.

A.2.1 The Scheduling System

StarForth executes on top of a host scheduler (L4Re or Linux). The VM runs largely on a single thread, cannot directly reorder word execution within a definition, and influences the host only through priority hints (`setpriority`, `sched_setparam`); the return stack dictates nesting order.

Within these limits, physics can still control scheduling. At the OS level it sets per-word priority (hotter words receive lower nice values), CPU affinity (hot, cache-sensitive words pinned together; interfering words separated), and preemption hints (hot, rapidly growing words made non-preemptible for batching).

```
typedef struct {
    int os_priority;       // nice level or sched_priority
    int scheduling_class;  // SCHED_OTHER, SCHED_FIFO, SCHED_RR
    int cpu_affinity_mask; // which cores allowed
} sched_hint_t;

// hotter = lower nice = higher priority
word->sched_hint.os_priority = (word->temperature_q8 > 0x8000) ? 5 :
    10;
```

Even without host control, a VM-level scheduler can respect physics. Inside `execute_colon_word()`, hot words on a critical path can be executed with affinity or batched with related words, while cold words are deferred or batched for idle time. Adaptive batching groups words that physics observes always run together — `IF`/`THEN`, for instance — to minimize cache misses.

A latency-SLO mechanism times each execution against a per-word limit and, on violation, adjusts knobs: it boosts priority, lowers the stack-depth limit, and shortens the batch timeout, logging the breach.

A.2.2 The Memory-Management System

StarForth uses a single 5 MB dictionary heap (statically allocated, no fragmentation), block storage of 1024 blocks of 1024 bytes, and two 1024-cell stacks. There is no explicit memory manager; allocation is implicit in the dictionary.

Physics can drive placement across a logical memory hierarchy. A tier is chosen from word temperature and entropy slope:

```
typedef enum {
    MEM_TIER_L1, MEM_TIER_L2, MEM_TIER_L3,
    MEM_TIER_DRAM, MEM_TIER_BLOCK
} mem_tier_t;

mem_tier_t choose_tier(DictEntry *word) {
    if (word->physics.temperature_q8 > 0xE000) return MEM_TIER_L1;
    if (word->physics.temperature_q8 > 0x8000) return MEM_TIER_L2;
    if (word->physics.temperature_q8 > 0x4000) return MEM_TIER_L3;
    if (word->physics.entropy_slope > 0)       return MEM_TIER_DRAM;
    return MEM_TIER_BLOCK;  // cold: evict to block storage
}
```

Cache coloring uses the learned call graph: words that frequently co-execute are placed in different cache sets to avoid conflict. Garbage collection is triggered not by a simple capacity threshold but by memory pressure (drawn from the host PSI snapshot) combined with the presence of cold, stale words to reclaim. Eviction replaces pure LRU with a temperature-weighted score — age multiplied by inverse temperature — so a candidate is hot only when both old and cold:

```
uint64_t score = (age / 1000000) * ((0xFFFF - temperature_q8) + 1);
```

```
// older + colder => higher score => evicted first
```

Block storage absorbs the coldest words. A word spills to disk when it is cold, has not run for several minutes, sits outside any SLO-critical path, and memory pressure is high; it is lazily reloaded on next use, with its physics re-touched on access.

A.2.3 Integration Points

Physics observes (entropy, temperature, recency, latency, call graph, stack depth, error rate, memory pressure) and feeds the scheduler, which decides priority, affinity, batch group, preemption, SLO, throttling, and stack limits. Execution respects those hints and generates new metrics, which feed the memory manager's decisions about cache tier, block placement, GC trigger, eviction, spilling, and prefetch. Placement then affects the next iteration's execution, closing the loop.

Two narrow APIs connect the components. The physics-to-scheduling interface (`sched_control_t`) carries priority, affinity mask, batch group, preemptibility, latency limit, and stack limit; `sched_update_word()` applies adjustments and `sched_report_execution()` reports observed latency. The physics-to-memory interface (`mem_control_t`) carries preferred tier, cache-set preference, spill-age threshold, evictability, and eviction priority. Scheduler and memory manager also coordinate directly: a batch decision notifies the memory manager so it can cache-color the group, and an eviction of a critical-path word warns the scheduler before disabling that word's fast path.

A.2.4 Implementation Phases

- **Phase 1 — Instrumentation (current).** Physics observes and publishes metrics; scheduling and memory read them advisorily.
- **Phase 2 — Weak Control.** Scheduling adjusts OS priority hints; memory becomes cache-tier aware; still advisory.
- **Phase 3 — Strong Control.** Scheduling enforces the latency SLO; memory enforces GC and eviction; the feedback loop is closed.
- **Phase 4 — ML Integration.** A model predicts latency, memory, and scheduling outcomes; governance approves knob adjustments.

A.2.5 Worked Examples

Hot word going critical. An I/O word heats during a file scan. Physics reports rising temperature and entropy slope and alerts both subsystems. Scheduling boosts the word's priority, enables I/O batching, caps recursion, and sets a 10 μs SLO. Memory promotes the word to L2, prefetches related words (`BLOCK_WRITE`, `BUFFER`, `UPDATE`), and protects it from eviction. Execution then meets the SLO at p99 $\approx$ 8.5 μs while the temperature stabilizes, after which both subsystems can act more aggressively on the now-learned pattern.

Cold word emergency eviction. Under 95% dictionary use and 96% cgroup memory, GC triggers. Physics scores eviction candidates by age times inverse temperature, selecting the coldest, oldest word first while protecting a recent hot word on the critical path. The chosen word spills to block storage, is marked `WORD_SPILLED`, and reclaims space. Much later, when the user invokes it again, the executor detects the flag, reloads it from storage in roughly 50 μs, runs it, and lets physics re-learn its pattern.

A.2.6 Critical Design Decisions

- **Scheduling granularity** — per-word priority, coarse word batches, or a hybrid that starts fine-grained and learns optimal batches. Recommendation: hybrid.
- **Memory placement accuracy** — static placement, dynamic migration, OS-hint delegation, or logical tiers with no physical movement. Recommendation: logical tiers as scheduling hints.
- **GC versus spilling** — always compact, always archive, or a governance-driven per-word policy. Recommendation: governance-driven.

A.2.7 Open Questions

The design leaves several questions for resolution: whether the VM should implement its own scheduler or defer entirely to the host; whether dictionary words can be physically relocated without breaking pointers, or only hinted logically; whether block storage serves persistence or active-memory overflow; how scheduling and memory decisions should interact with L4Re IPC deadlines and capability delegation; who owns eviction and spilling policy; and whether the added control overhead (roughly 100 ns atop the existing 200–300 ns per event) is acceptable for target latencies.

A.3 Physics Signal Inventory

This inventory enumerates the telemetry the physics-driven scheduler can sense today, drawn from two sources: the L4Re microkernel (through the StarshipOS loader stack) and the in-process StarForth VM. It defines the input stage of the scheduler. The guiding metaphor is an operational amplifier: the positive input ties to real host signals, the negative input ties to VM-internal counters, and a Bayesian inference loop stabilizes the gain. The metaphor is a modeling device for the control architecture, not a literal circuit.

A.3.1 L4Re and Microkernel Signals

Kernel Interface Page (KIP). The KIP exposes a high-resolution monotonic clock (`l4_kip_clock_ns()`) for timing when no RTC is present, a kernel build fingerprint for correlating inference data with kernel revisions, the scheduling granularity (an upper bound on how fine-grained the physics scheduler can react), physical memory descriptors (how much RAM can be earmarked for in-RAM inference buffers), and platform/SMP information. SMP availability doubles as a noise hook: work can be scheduled on sibling cores to inject controlled jitter and keep the model honest.

Scheduler object. `l4_scheduler_info()` reports the online CPU bitmap and maximum CPU count, bounding how many physics loops run in parallel. `l4_sched_param_t` (priority and quantum) records the loader's configured timeslice as a prior per VM. The scheduler-class flags reveal whether fixed-priority or weighted-fair-queue scheduling is active, which shapes how aggressively words may be heated or cooled. `l4_sched_cpu_set()` pins the main or physics worker thread to a core subset during experiments.

Loader and environment capabilities. The loader script (`quark.lua`) wires named capabilities the runtime can tap: `vbus_storage` for block-device enumeration and driver statistics (queue depth, error counts); `ahci_chan` for completion-interrupt rates and per-port error registers; `rtc_ns` for wall-clock sync; `cons_mux` for console input cadence as a proxy for interactive

heat; `L4.Env.log` for scheduler and hardware warnings that should bias temperature decay; and `L4.Env.vesa` for framebuffer refresh/blit latency.

Further kernel APIs. `l4_thread_control()` and `l4_thread_ex_regs()` expose per-thread UTCB pointers and instruction pointers for correlating VM stalls with kernel scheduling; `l4_ipc_error()` surfaces IPC failure codes that could feed entropy penalties for fault-prone words; and virtual IRQ devices can be counted to derive environmental noise injected into the VM.

A.3.2 StarForth VM Signals

Core runtime state. `DictEntry.entropy` is the per-word execution counter, the baseline for macro temperature. `DictEntry.physics` holds the tunable knobs (temperature, last-active timestamp, mass). The data and return stack pointers (`VM.dsp`, `VM.rsp`) report stack depth, whose steep excursions imply turbulent execution and feed variance estimates. Mode and state fields reveal interpret-versus-compile state and the currently executing word. The `error` and `halted` flags are binary fault signals that spike entropy decay when they flip. Log volume and severity mix are themselves telemetry: heavy warnings can cool risky words.

Profiler and instrumentation. `WordProfile` supplies per-word total time, call count, and min/avg/max latency — a direct feed for `avg_latency_ns` and Bayesian likelihoods. `MemoryProfile` supplies read/write counts and bytes as a mass-energy proxy for memory-thrashing words. The profiler counters (VM cycles, dictionary lookups, stack ops, allocations) seed default temperatures and heat capacities. `profiler_word_count()` stays available even without detailed profiling, and `vm_debug_dump_state()` provides a structured post-mortem the Bayesian tool can parse to reset priors after a crash.

Block and storage subsystem. Block-subsystem globals report working-set size, dirty-block counts, and block-allocation-map churn. `blkio_info()` exposes device geometry and the read-only bit, informing whether a cooling word should migrate to RAM or disk tiers. `blkio_read/write` return codes give immediate error feedback. Cache slots track hit/miss rate and write-back frequency to infer subsystem momentum.

A.3.3 Op-Amp Signal Flow

The signal flow follows the amplifier metaphor in five stages:

1. **Positive input (microkernel)** — real-world noise: CPU availability, I/O latency, RTC drift, IRQ storms.

2. **Negative input (VM)** — internal state: entropy, latency, stack tension, storage dirty set.

3. **Amplifier** — the Bayesian inference loop adjusts priors and updates word physics.

4. **Output** — updated `DictPhysics` structs and scheduler hints that modulate execution order and block placement.

5. **Feedback** — the observation window width is adjusted by variance (a gauge study) and the cycle repeats.

A.3.4 Messaging and IPC Considerations

The pub/sub backbone is shared-memory-first: a ring buffer with sequence counters inside a dedicated analytics heap (10 MiB by default). Producers write events, flip a counter, and continue, with no blocking semantics inside the VM. On L4Re, where IPC is synchronous, IPC serves only as a notification channel — a publisher pokes a notification thread that drains the ring and forwards to subscribers; virtual IRQs offer an alternative non-blocking wakeup. On Linux the same API is backed by condition variables or eventfd behind a common shim, keeping the VM path identical across platforms. All state stays resident in a fixed-size analytics heap with no dynamic expansion. That heap is separate from the 5 MiB VM arena (`VM_MEMORY_SIZE`), so the dictionary, stacks, and block-subsystem budgets remain untouched.

A.3.5 Host Snapshot Shim and Analytics Heap

Phase 1 ships a concrete implementation. `physics_runtime_init()` reserves the analytics heap (10 MiB default) whose layout the HOLA contract documents, publishing the header and region descriptors HOLA consumes. `physics_host_snapshot()` abstracts POSIX and L4Re scheduler probes and feeds ring-buffer events on channel `0x00000002` via `physics_analytics_publish_event()`. The heap header, event records, and mailbox schema live in `include/physics_runtime.h`; the runtime lives in `src/physics_runtime.c`. The interface is deliberately ABI-stable so governance tooling can mirror it without pulling in C sources. On the POSIX path, Phase 1 additionally captures Linux PSI values mapped into `psi_*_avg{10,60,300}_milli`, `/proc/stat` total and idle jiffies, and cgroup v2 CPU and memory usage where available; flag bits advertise which sources were populated.

A.3.6 Conventions and Constraints

Several conventions govern the inference loop. The observation window combines a time-based heartbeat with event-count triggers: events act as "excitement" that boosts entropy, while publish and decay operations cool at roughly half that rate (tunable). All computation uses 64-bit fixed-point integers. Physics snapshots can optionally be persisted into FORTH block storage and reloaded during `(INIT)` to simulate a warm boot or replay a training sequence. Descriptor inheritance flows from module, vocabulary, and VM defaults down to individual words, so sensible priors seed at multiple levels; every tier exposes the same attribute schema — temperature, latency, mass, state flags, ACL hints, pub/sub mask, and pinned flag. VM-level rollups mirror per-word metrics and define operating bands (`COLD`, `WARM`, `HOT`, `CRITICAL`) that governance and the scheduler shim can respond to. Isabelle captures the formal state machine, invariants, and IPC-handshake proofs; HOLA defines the shared-memory layout and control protocol consumed by both the VM and external analyzers.

A primitive seed table (`physics_metadata_apply_seed()`) installs initial priors for high-impact primitives — control flow, I/O, block subsystem, save-system — so temperature and latency estimates do not start at absolute zero; governance tooling can extend or override the table once the Bayesian loop is in place.

A.3.7 Immediate Research Tasks

1. Prototype a thin KIP/scheduler shim exposing the listed signals to userland C with no filesystem: on L4Re via `l4re_kip()`, on POSIX via a stub that feeds monotonic time and scheduler defaults.

2. Inventory the IO-server (vbus) protocol to pull queue-depth and error counters for AHCI, NVMe, and virtio backends.

3. Define the shared-memory layout between the VM and the Bayesian analyzer, defaulting to a 10 MiB heap (header, ∼6 MiB event ring, ∼3 MiB summary/scratch, padding), honoring the platform split between POSIX and L4Re messaging.

4. Extend the profiler to snapshot `MemoryProfile` deltas without enabling full verbose mode, keeping overhead low.

5. Derive initial priors for key primitives (control, I/O, block) from handcrafted knowledge plus the loader's priority and quantum configuration.

Appendix B

Messaging Architecture

B.1 The StarshipOS Messaging Field

The StarshipOS messaging subsystem is modeled as a thermodynamic field rather than a passive bus. In this framework, message propagation and processing behave as a physical system: discrete messages behave analogously to quanta (photons), and words — the fundamental executable units of the StarshipOS runtime — behave as particles, or loci of interaction. The thermodynamic vocabulary is a modeling tool for message dynamics; it grounds prioritization and scheduling, credit allocation and flow control, thermal regulation of load, entropy-driven policy and ML feedback, and instrumentation of message flow.

B.1.1 Conceptual Model

The message field. The field is the logical substrate mediating all communication. It comprises endpoints (boundaries between word spaces and the field), channels (directed pathways: virtual channels, pub/sub topics, RPC routes), and field properties (dynamic parameters such as message density, propagation latency, and entropy). The field is not centralized; it emerges from the coordinated behavior of the broker/router (`sf_msg-srv`), the ABI provider (`sf_msg-drv`), and the participant library (`libsfmsg`) over shared memory rings and control IPC.

Messages as quanta. Messages are modeled as energy quanta with four core properties:

Property	Description
Energy	Encodes priority, TTL, QoS, and thermodynamic state
Momentum	Routing vector through the field
Cross section	Probability of absorption by a word
Entropy	Thermodynamic measure of the flow's state space

Messages may be absorbed, emitted, or scattered by words and services, by analogy with photon–atom interactions.

Words as particles. Each word is a particle in the field — a locus of interaction rather than an isolated routine — with intrinsic mass (computational cost), charge (degree of side-effect or state mutation), and cross-section (the message types to which it responds). When a message reaches a word, the interaction may excite the word (trigger execution), alter its state, or pass through unabsorbed; executed words may emit new messages, either deterministically or in a burst (stimulated emission).

B.1.2 Mathematical Formulation

Maxwellian entropy. The system adopts a Maxwell–Boltzmann entropy model, treating each endpoint or channel as a thermodynamic micro-system. For a given flow f, the flow entropy $S_f(t)$ is a scaled function of the variance of inter-arrival intervals $\sigma_t^2(f)$, the normalized queue occupancy $Q(f) \in [0, 1]$, the credit-utilization ratio $U(f)$ (credits consumed over granted), the delivery fan-out $R(f)$, and the normalized message TTL $T(f)$, with scaling constant k (typically 1):

$$S_f(t) = k \cdot g\big(\sigma_t^2(f),\, Q(f),\, U(f),\, R(f),\, T(f)\big) \tag{B.1}$$

High entropy indicates unpredictable, high-energy flows approaching saturation; low entropy indicates stable, structured behavior.

System temperature and pressure. Temperature Θ is the density-weighted average energy of messages in the field. Pressure Π represents backlog intensity, aggregated over flows from queue occupancy and credit utilization:

$$\Pi = \sum_f Q(f)\, U(f) \tag{B.2}$$

Aggregate system entropy Σ sums the per-flow entropies:

$$\Sigma = \sum_f S_f \tag{B.3}$$

Together these quantities give real-time metrics of system health, suitable as inputs to control algorithms.

B.1.3 Header and State Extensions

Messages carry thermodynamic metadata alongside conventional routing and control information. A per-message header extension encodes a composite energy scalar derived from QoS class, TTL, and policy weighting; a Maxwellian entropy estimate computed at the sender or broker; and an optional producer-side temperature hint for expected burstiness:

```
struct sfm_hdr_thermo {
    uint16_t energy_q8;    // priority/TTL/entropy composite (Q8.8)
    uint16_t entropy_mx;   // Maxwellian entropy (Q8.8)
    uint16_t temp_hint;    // optional producer temperature hint
    uint16_t reserved;
};
```

Endpoints maintain continuously updated thermodynamic state — current entropy, temperature, pressure, credit utilization, and a last-refresh timestamp:

```
struct sfm_endpoint_state {
    float    entropy_current;
    float    temperature;
    float    pressure;
    float    credit_utilization;
    uint64_t last_refresh_tsc;
};
```

B.1.4 Scheduler and Credit Integration

Credits as energy quanta. Credits represent the available energy budget for message emission along a channel. High-entropy flows receive smaller, more frequent credit refreshes for tight regulation; low-entropy flows receive larger, batched credits for looser regulation. The recycling policy — immediate, piggyback, or batched — is selected dynamically from the flow's entropy and temperature.

Thermodynamic scheduling. Scheduling priority is a function of QoS class, deadline, and flow entropy:

$$P_{\text{sched}} = f(\text{QoS}, \text{deadline}, S_f) \tag{B.4}$$

Low-entropy real-time flows are scheduled deterministically; high-entropy bulk flows are throttled or coalesced; aging and anti-starvation mechanisms apply within entropy bands. This prevents high-entropy flows from destabilizing the system while letting low-entropy flows achieve predictable latency.

B.1.5 Monitoring and Control Loop

A control loop in `sf_msg-srv` maintains field stability in four steps:

1. **Sampling** — entropy, temperature, and pressure are sampled periodically from endpoints and channels.

2. **Aggregation** — system-wide Σ, Θ, and Π are computed.

3. **Policy evaluation** — control laws or ML bandits adjust credit windows, scheduling weights, and routing from the sampled state.

4. **Actuation** — the scheduler and credit allocator apply the updated parameters.

The loop runs with fixed thresholds or adaptive policies; ML components treat entropy as the order parameter for optimization.

B.1.6 Implications for the Runtime

Because words are already explicit entities with well-defined entry points, modeling them as particles interacting via message photons integrates cleanly with the existing FORTH execution model. The runtime gains a unified abstraction for computation and communication, fine-grained control over execution dynamics through entropy, a principled basis for prioritization in place of ad-hoc heuristics, and thermodynamic instrumentation for debugging and analysis.

B.1.7 Future Work

Four directions extend the model: a compact formal policy language for entropy-based routing and scheduling; distributed thermodynamic fields spanning multi-node topologies, where message photons propagate across network transports; entropy-driven garbage collection and memory tiering that couple message entropy with VM memory placement; and visualization tooling that renders message flow as a dynamic field.

B.1.8 Reference Header

A draft reference header, `include/sfm_thermo.h`, accompanies the model as design reference rather than production code (C99, released CC0 1.0 / public domain). It provides fixed-point helpers for Q8.8 and Q16.16 conversion; the `sfm_hdr_thermo` header extension; a per-flow state structure (`sfm_flow_state_t`) tracking Welford inter-arrival statistics, EWMA queue occupancy and credit utilization, fan-out and TTL trackers, and derived entropy, temperature, and pressure; a field snapshot structure; and a credit-decision structure. Its API initializes a flow with watermarks and entropy thresholds (`sfm_thermo_init_flow`), updates state on enqueue (`sfm_thermo_on_enq`), samples and decides a credit allocation (`sfm_thermo_sample_and_decide`), aggregates flows into a field snapshot (`sfm_thermo_aggregate`), and fills a message's thermodynamic header (`sfm_thermo_fill_hdr`).

B.1.9 References

- J. C. Maxwell, *Illustrations of the Dynamical Theory of Gases*, Phil. Mag., 1860.
- L. Boltzmann, *Weitere Studien über das Wärmegleichgewicht unter Gasmolekülen*, 1872.
- StarshipOS Internal Messaging Architecture Specifications.